UNIX and Shell Programming

Exam Notes — BCA 3rd Year, 6th Semester

Techno India (Hooghly Campus), Department of Computer Applications
MAKAUT · Bachelor of Computer Applications

Course Overview

This course covers the fundamentals of the UNIX operating system and shell programming. Topics include the UNIX architecture, file system, essential commands, shell scripting with Bash, process management, inter-process communication, text processing utilities, regular expressions, and the VI editor.

Faculty: Prof. Anik Patra | M.C.A, M.Tech · Assistant Professor | Academic Year 2025–26

Copyright Notice

© 2026 Amitendu Bikash Dhusiya

All rights reserved.

You may use, copy, modify, and share this material for any purpose.
Please acknowledge the author: **Amitendu Bikash Dhusiya**.

Contents

Unit 1: Introduction to UNIX 9

Introduction to UNIX 9

What is UNIX? . . . 9

A Brief History of UNIX . . . 9

The UNIX Philosophy . . . 9

UNIX Characteristics . . . 10

Basic UNIX Commands . . . 10

POSIX Standard 10

What is POSIX? . . . 10

POSIX Standards Overview . . . 11

What POSIX Defines . . . 11

POSIX System Call Example . . . 11

POSIX Compliance Today . . . 12

Introduction to Linux 12

What is Linux? . . . 12

The Birth of Linux . . . 12

Key Characteristics of Linux . . . 12

Checking Linux Kernel Information . . . 13

Most Popular Distributions of Linux 13

What is a Linux Distribution? . . . 13

Major Linux Distributions . . . 13

Comparison: UNIX vs Linux 15

Side-by-Side Comparison . . . 15

When to Choose UNIX vs Linux . . . 15

UNIX System Architecture 15

Layered Architecture Overview . . . 15

Detailed Component Description . . . 16

User Mode vs Kernel Mode . . . 16

The UNIX Kernel and System Call Interface 17

The UNIX Kernel . . . 17

Kernel Subsystems . . . 17

System Calls . . . 17

System Call Categories . . . 18

How a System Call Works . . . 18

Tracing System Calls with `strace` . . . 18

The `/proc` Filesystem — A Window into the Kernel . . . 19

UNIX Directory Structure 19

The Hierarchical File System . . . 20

The Root Directory Tree . . . 20

Critical Directory Descriptions . . . 20

File Types in UNIX . . . 21

Navigating the Directory Structure . . . 22

Unit 1 Summary 23

Unit 1 Practice Questions 24

Unit 2: UNIX File Commands 25
UNIX File Commands — Overview 25
Create Directory — `mkdir` 25
Change Directory — `cd` 26
Move and Copy Directory 27
`cp` — Copy Files and Directories . . . 27
`mv` — Move or Rename Files and Directories . . . 28
Create File 29
Using `touch` . . . 29
Using `cat` with Redirection . . . 29
Using Text Editors . . . 30
Using `printf` or Here-Documents . . . 30
Remove File — `rm` 30
Remove Directory — `rmdir` and `rm -r` 31
`rmdir` — Remove Empty Directories . . . 31
`rm -r` — Remove Non-Empty Directories . . . 31
Listing Directory Information — `ls` 32
Types of Files in UNIX 33
Change File Security — `chmod`, `chown`, `chgrp` 35
Understanding UNIX Permissions . . . 35
`chmod` — Change File Mode . . . 35
`chown` — Change File Owner . . . 36
`chgrp` — Change Group Ownership . . . 36
Creating Empty Files and Changing Timestamps — `touch` 36
Access Time, Modification Time and Change Time 37
`touch` with `-d` and `-t` Options 38
The `-d` Option — Human-Readable Date String . . . 38
The `-t` Option — Numeric Timestamp Format . . . 38
Finding Files and Directories 39
`find` — Search by Any Criterion . . . 39
`locate` — Fast Index-Based Search . . . 40
`which` and `whereis` . . . 40
Soft Link and Hard Link 41
Hard Links . . . 41
Symbolic (Soft) Links . . . 41
Hard Link vs Soft Link — Comparison . . . 42
The i-Node (Index Node) 42
What an i-Node Stores . . . 43
Size of the File System — `df` and `du` 43
`df` — Disk Free (Filesystem Level) . . . 43
`du` — Disk Usage (Directory/File Level) . . . 44
Output Redirection — `>` and `>>` 45
Standard File Descriptors . . . 45
The `>` Operator — Overwrite . . . 45
The `>>` Operator — Append . . . 45

Pipe (|) and tee Command 46
The Pipe Operator | . . . 46
The tee Command . . . 47
Unit 2 Summary 49
Unit 2 Practice Questions 50

Unit 3: File Processing Commands 51
Row-Wise and Column-Wise Selection — cut 51
Syntax and Options . . . 51
Sample Data Files Used in This Unit . . . 51
Column-Wise Selection — Selecting by Character Position . . . 51
Row-Wise and Column-Wise Selection from Files with -f and -d . . . 52
Row-Wise and Column-Wise Selection with awk 54
Merge Lines of Files Horizontally — paste 55
Split Large Files into Smaller Files — split 56
Counting Lines, Words, and Characters — wc 58
Sorting File Content — sort 59
Basic and Alphabetical Sorting . . . 59
Sorting by Specific Column (-k flag) . . . 59
Numerical Sorting (-n flag) . . . 60
Comparing Two Files Line by Line — diff 61
Compare Two Files Character by Character — cmp 63
Compare Two Sorted Files — comm 64
Join Two Files — join 65
The uniq Command 67
The Transformation Command — tr 68
Basic Translation . . . 68
Deleting Characters with -d . . . 69
Squeezing Repeated Characters with -s . . . 69
tr Character Classes . . . 70
Unit 3 Summary 72
Unit 3 Practice Questions 73

Unit 4: The VI Editor 74
The cal Command — Calendar 74
The date Command 75
The pr Command — Print Formatter 77
The who Command — Logged-In Users 78
The bc Command — Arbitrary-Precision Calculator 79
Interactive and Inline Usage . . . 79
The echo Command 81
The zip and unzip Commands 82
zip — Creating ZIP Archives . . . 82
unzip — Extracting ZIP Archives . . . 83
The gzip Command 84
Archiving Files — tar 86

Core Modes and Options . . . 86
Creating Archives . . . 86
Listing and Extracting Archives . . . 87
Compression Format Comparison . . . 87
Unit 4 Summary **89**
Unit 4 Practice Questions **90**

Unit 5: Utility Commands **91**
Overview of the VI Editor **91**
Why Learn VI? . . . 91
Starting VI . . . 91
The Three Modes of VI **92**
Mode Diagram . . . 93
Command Mode . . . 93
Insert Mode . . . 93
Last-Line Mode (Ex Mode) . . . 93
Navigation in VI — Moving the Cursor **93**
Basic Cursor Movement (Command Mode) . . . 94
Moving by Word . . . 94
Moving by Line . . . 94
Moving by Page / Screen . . . 95
Quick Navigation Reference . . . 95
Text Editing in VI **95**
Inserting Text — Entering Insert Mode . . . 95
Deleting Text . . . 96
Copying and Pasting (Yank and Put) . . . 97
Undo and Redo . . . 98
Working with Files — Saving and Exiting **98**
Saving Files . . . 98
Exiting VI . . . 99
File Operations in VI **100**
Opening a New or Different File . . . 100
Shell Commands from VI . . . 100
Search in VI **101**
Forward and Backward Search . . . 101
Regular Expressions in VI Search . . . 101
Search Navigation Summary . . . 102
Search and Replace in VI **103**
Basic Substitute Syntax . . . 103
Replacing on Current Line . . . 103
Global Search and Replace (Entire File) . . . 103
Substitute Flags Reference . . . 104
Range Specifiers . . . 104
Essential VI Cheat Sheet **106**
Unit 5 Summary **107**
Unit 5 Practice Questions **108**

Unit 6: Advanced File Searching — GREP and AWK 109
Searching Files with Pattern Using grep and awk 109
Sample Data Files Used in This Unit . . . 109
Advanced Searching with grep 109
Basic grep Usage . . . 109
grep with Regular Expressions . . . 110
Advanced grep Options . . . 111
awk with print and printf 112
print — Basic Output . . . 112
printf — Formatted Output . . . 113
awk with Comparison Operators 114
awk with Arithmetic Operators 115
awk BEGIN and END Sections 116
awk if-else Statement 118
Built-in Variables: FS and OFS 119
awk String and Arithmetic Functions 120
Built-in String Functions . . . 120
Built-in Arithmetic Functions . . . 121
Loops in awk 122
for Loop . . . 122
while and do-while Loops . . . 123
Search and Substitute Functions in awk 124
sub() and gsub() — Substitution . . . 124
match() — Searching within Fields . . . 125
Complete Practical Pipeline Example . . . 125
Unit 6 Summary 127
Unit 6 Practice Questions 128

Unit 7: Processes in UNIX 129
What is a Process in UNIX? 129
Process vs Program . . . 129
What Every Process Contains . . . 129
The ps Command — Process Status 130
Basic ps Usage . . . 130
ps Column Meanings . . . 131
ps with Different Options . . . 131
The init Process and the UNIX Login Process 132
The init Process (PID 1) . . . 132
The UNIX Login Process . . . 133
Process Creation: fork(), getpid(), getppid(), wait() 133
fork() — Creating a New Process . . . 133
getpid() and getppid() . . . 134
wait() — Waiting for Child Processes . . . 135
Zombie Processes . . . 135
pipe() and Message Passing 136
pipe() — Inter-Process Communication . . . 136

Message Queues . . . 137
UNIX Process States and Diagram **138**
The Seven Process States . . . 139
Process State Transition Diagram . . . 139
Reading Process State in `ps` . . . 139
The UNIX Process Scheduler **140**
Traditional UNIX Scheduler . . . 140
Linux Completely Fair Scheduler (CFS) . . . 140
Swapped Memory **141**
How Swapping Works . . . 142
The `vmstat` Command **143**
Understanding `vmstat` Output . . . 144
The `top` Command **145**
Interactive `top` Commands . . . 145
Non-Interactive `top` Options . . . 145
The `nice` Command **146**
Understanding Priority and Nice Values . . . 146
Practical Use of `nice` . . . 147
Unit 7 Summary **148**
Unit 7 Practice Questions **149**

Unit 8: Shell Programming **150**
Introduction to Shell and Types of Shell **150**
What is a Shell? . . . 150
The Shell's Place in the System . . . 150
Types of Shell . . . 151
Use of Shell — Writing Your First Shell Script **152**
What is a Shell Script? . . . 152
Creating and Running a Script . . . 152
System Variables and User-Defined Variables **153**
Variables in Shell . . . 153
User-Defined Variables . . . 153
System Variables (Environment Variables) . . . 154
Exporting Variables . . . 156
Use of Single Quote, Double Quote, and Backslash **156**
Backslash — Escaping a Single Character . . . 156
Single Quotes — Full Literal Preservation . . . 157
Double Quotes — Controlled Expansion . . . 158
Quoting Rules Summary . . . 158
Command Substitution **159**
Syntax: `$(…)` vs `` `…` `` . . . 159
Practical Uses of Command Substitution . . . 159
`let` — Assigning and Evaluating Expressions **160**
Arithmetic Operators . . . 160
Taking Input from the User **162**
Command Line Parameters **164**

Positional Parameter Variables 164
Validating Arguments and `shift` 165
Use of `if` Statement **166**
Basic `if` Syntax 166
The `test` Command and `[ ]` Brackets 167
The `[[ ]]` Extended Test (Bash-Only) 168
Practical `if` Examples 169
Use of `for`, `while`, and `until` Loops **169**
`for` Loop 169
`while` Loop 170
`until` Loop 171
Loop Control: `break` and `continue` 172
Loop Comparison 173
Observing the Exit Status **173**
Reading and Using `$?` 173
The `exit` Command in Scripts 174
Using Exit Status in Conditionals 174
Custom Exit Codes and `trap` 175
Unit 8 Summary **176**
Unit 8 Practice Questions **177**

References **178**

Quick Reference Cheat Sheet **179**

Glossary **184**
Lab Exercises: Practical Shell Scripts **187**
Index **195**

Unit 1: Introduction to UNIX

Unit 1 — Introduction to UNIX

UNIX History · POSIX Standard · Linux · Distributions · UNIX vs Linux · System Architecture · Kernel & System Calls · Directory Structure

Introduction to UNIX

What is UNIX?

Definition — UNIX

UNIX is a powerful, multiuser, multitasking operating system originally developed in the late 1960s at Bell Labs (AT&T). It is renowned for its *simplicity*, *portability*, *security*, and its philosophy of building small, composable tools.

UNIX is not a single product — it is a **family of operating systems** that share a common design philosophy. The name "UNIX" is now a trademark owned by *The Open Group*, who certifies compliant systems.

A Brief History of UNIX

Year	Milestone
1965	Multics project begins at MIT/Bell Labs/GE
1969	Ken Thompson writes the first UNIX in assembly on PDP-7 at Bell Labs
1971	UNIX First Edition released; written in assembly
1973	UNIX rewritten in C by Dennis Ritchie — portability breakthrough
1975	V6 UNIX released to universities
1977	BSD (Berkeley Software Distribution) fork starts at UC Berkeley
1983	AT&T releases System V — commercial UNIX
1991	Linus Torvalds releases Linux kernel 0.01
2000s	macOS (Darwin kernel) certified as UNIX; Linux dominates servers

The UNIX Philosophy

The UNIX design philosophy, articulated by Doug McIlroy, can be summarized as:

1. **Write programs that do one thing and do it well.**

2. **Write programs that work together.**
3. **Write programs that handle text streams**, because that is a universal interface.

Key Insight

The UNIX philosophy encourages building complex behavior by *composing* simple programs using **pipes** (|), rather than creating monolithic applications.

UNIX Characteristics

- Multiuser environment
- Multitasking / multiprocessing
- Hierarchical file system
- Everything is a file (devices, sockets)
- Portability (written in C)
- Powerful shell scripting
- Built-in networking (TCP/IP)
- Security through user/group permissions
- Open standards (POSIX)
- Robust inter-process communication

Basic UNIX Commands

```
# Display current working directory
$ pwd
/home/student

# List files with details
$ ls -la
total 48
drwxr-xr-x  6 student staff  192 Jan 15 09:30 .
drwxr-xr-x 12 root    root   384 Jan 15 08:00 ..
-rw-r--r--  1 student staff 2048 Jan 15 09:25 notes.txt
drwxr-xr-x  3 student staff   96 Jan 14 14:22 projects/

# Navigate directories
$ cd /etc
$ cd ~            # return to home
$ cd -            # return to previous directory

# Create and remove directories
$ mkdir -p projects/unix/lab1
$ rmdir old_folder

# Copy, Move, Remove files
$ cp source.txt destination.txt
$ mv old_name.txt new_name.txt
$ rm -rf temp_folder/
```

Listing 1: Essential UNIX commands

POSIX Standard

What is POSIX?

Definition — POSIX

POSIX (*Portable Operating System Interface*) is a family of standards specified by the IEEE (Institute of Electrical and Electronics Engineers), formally IEEE 1003. It defines the API, command-line shells, and utility interfaces that ensure compatibility between UNIX-like operating systems.

POSIX was created to address the fragmentation in the UNIX ecosystem during the 1980s, when different vendors (Sun, HP, IBM, AT&T) produced incompatible UNIX variants. The goal: write code once, run it on any POSIX-compliant system.

POSIX Standards Overview

Standard	Formal Name	Scope
POSIX.1	IEEE 1003.1	Core API (C library, processes, files)
POSIX.1b	IEEE 1003.1b	Real-time extensions
POSIX.1c	IEEE 1003.1c	Threads (pthreads)
POSIX.2	IEEE 1003.2	Shell and utilities
POSIX.1-2017	Latest revision	Unified standard (SUSv4)

What POSIX Defines

POSIX mandates standards across six major areas:

1. **C API** — system calls for file I/O, processes, signals, memory
2. **Command Interpreter** — the `sh` (POSIX shell) specification
3. **Utility Programs** — `grep`, `awk`, `sed`, `find`, `make`, etc.
4. **Environment Variables** — `PATH`, `HOME`, `USER`, `TERM`
5. **Regular Expressions** — ERE (Extended Regular Expressions)
6. **Directory Structure** — a minimal hierarchy like `/bin`, `/usr`, `/tmp`

POSIX System Call Example

The following C program uses POSIX-compliant system calls to demonstrate portable file I/O:

```c
#include <fcntl.h>       /* open(), O_RDONLY */
#include <unistd.h>      /* read(), write(), close() */
#include <stdio.h>       /* perror() */
#include <stdlib.h>      /* exit() */

int main(void) {
    int fd;
    char buffer[256];
    ssize_t bytes_read;

    /* Open a file (POSIX system call) */
    fd = open("/etc/hostname", O_RDONLY);
    if (fd == -1) {
        perror("open");
        exit(EXIT_FAILURE);
    }

    /* Read from file descriptor */
    bytes_read = read(fd, buffer, sizeof(buffer) - 1);
    if (bytes_read > 0) {
        buffer[bytes_read] = '\0';   /* null-terminate */
        /* Write to standard output (fd = 1) */
        write(STDOUT_FILENO, buffer, bytes_read);
    }

    /* Close file descriptor */
    close(fd);
```

```
    return 0;
}
```

Listing 2: POSIX file I/O using open/read/write

```
$ gcc -o posix_example posix_example.c
$ ./posix_example
mylinuxbox
```

Listing 3: Compiling and running the POSIX example

POSIX Compliance Today

System	POSIX Status	Notes
macOS	Certified	Certified since 10.5 (Leopard)
Linux (most distros)	Mostly compliant	Not officially certified (cost)
FreeBSD / OpenBSD	Mostly compliant	Very close to POSIX
Windows (WSL2)	Partial	Via Windows Subsystem for Linux
AIX, HP-UX, Solaris	Certified	Commercial UNIX systems

Why POSIX Matters

Writing POSIX-compliant code means your programs can compile and run on Linux, macOS, BSDs, and commercial UNIX systems with little to no modification — a massive advantage in server and embedded development.

Introduction to Linux

What is Linux?

Definition — Linux

Linux is a free and open-source, UNIX-like operating system *kernel* created by Linus Torvalds in 1991. When combined with the GNU user-space tools (developed by Richard Stallman's GNU Project), the result is a complete operating system commonly referred to as a **GNU/Linux** system.

The Birth of Linux

On August 25, 1991, Linus Torvalds posted this famous message to the `comp.os.minix` newsgroup:

```
newsgroup: comp.os.minix --- August 25, 1991

Hello everybody out there using minix ---

I'm doing a (free) operating system (just a hobby, won't be big and
professional like gnu) for 386(486) AT clones.
```

Linux 0.01 was released in September 1991 with about 10,000 lines of code. The Linux kernel 6.x today has over 30 *million* lines.

Key Characteristics of Linux

1. **Open Source** — Licensed under GPLv2; source code is freely available

2. **Monolithic Kernel** — Core OS functionality runs in kernel space
3. **Modular Design** — Loadable Kernel Modules (LKMs) extend functionality
4. **Multiuser & Multitasking** — Supports simultaneous users and processes
5. **POSIX-like** — Near-full POSIX compliance
6. **Network-centric** — Robust TCP/IP stack built into the kernel
7. **Hardware support** — Runs on x86, ARM, RISC-V, MIPS, PowerPC, and more

Checking Linux Kernel Information

```
# Display kernel version
$ uname -r
6.5.0-35-generic

# Display all system information
$ uname -a
Linux mybox 6.5.0-35-generic #35-Ubuntu SMP x86_64 GNU/Linux

# Full OS release information
$ cat /etc/os-release
NAME="Ubuntu"
VERSION="22.04.3 LTS (Jammy Jellyfish)"
ID=ubuntu
ID_LIKE=debian
PRETTY_NAME="Ubuntu 22.04.3 LTS"

# List loaded kernel modules
$ lsmod | head -10
Module                  Size  Used by
nvidia              14561280  68
bluetooth             688128  38 btrtl,btusb
ext4                  983040  2
```

Listing 4: Querying kernel and system information

Most Popular Distributions of Linux

What is a Linux Distribution?

A **Linux distribution** (or *distro*) bundles the Linux kernel together with a package manager, GNU tools, a desktop environment, and pre-installed software to form a complete, usable OS.

Major Linux Distributions

Debian-Based Family

Distribution	Description
Ubuntu	Most popular desktop Linux; 6-month releases; LTS every 2 years
Debian	Community-driven; extremely stable; basis for Ubuntu, Mint, Kali
Linux Mint	Beginner-friendly; ships with media codecs; Cinnamon desktop
Kali Linux	Security & penetration testing; 600+ security tools pre-installed
Raspberry Pi OS	ARM-based; designed for Raspberry Pi single-board computers

```
# Update package index
$ sudo apt update
```

```

# Upgrade all installed packages
$ sudo apt upgrade -y

# Install a new package
$ sudo apt install vim curl wget htop

# Search for packages
$ apt search python3-dev

# Remove a package (keep config files)
$ sudo apt remove packagename

# Purge a package (remove config too)
$ sudo apt purge packagename
```

Listing 5: Package management on Debian/Ubuntu

Red Hat–Based Family

Distribution	Description
RHEL	Red Hat Enterprise Linux; paid; gold standard for enterprise
Fedora	Community-driven; upstream for RHEL; cutting-edge features
CentOS Stream	Rolling preview of RHEL; community-supported
AlmaLinux / Rocky	Free RHEL-compatible; born after CentOS 8 EOL

```
# Update system (dnf --- modern package manager)
$ sudo dnf update -y

# Install a package
$ sudo dnf install nginx mariadb-server

# List installed packages
$ rpm -qa | grep python

# Enable and start a service
$ sudo systemctl enable --now nginx
$ sudo systemctl status nginx
```

Listing 6: Package management on Red Hat/Fedora systems

Other Notable Distributions

Distro	Base	Package Mgr	Best For
Arch Linux	Independent	pacman	Advanced users; rolling release
Gentoo	Independent	portage	Source-based; fully custom
openSUSE	Independent	zypper	Desktop + enterprise; YaST config
Alpine Linux	Independent	apk	Containers; security; minimal size
NixOS	Independent	nix	Reproducible; declarative config
Slackware	Independent	pkgtool	Oldest surviving distro (1993)

Comparison: UNIX vs Linux

Side-by-Side Comparison

Criterion	UNIX	Linux
Origin	Bell Labs, AT&T, 1969	Linus Torvalds, 1991
License	Proprietary (most variants)	GNU GPLv2 (open source)
Cost	Paid / licensed	Free (mostly)
Source	Closed source (usually)	Open source
Hardware	Specific certified hardware	Wide range of hardware
Portability	Limited to certified platforms	x86, ARM, RISC-V, MIPS, etc.
Security	Enterprise-grade security	Strong; SELinux, AppArmor
Filesystems	UFS, ZFS, JFS, VxFS	ext4, XFS, Btrfs, ZFS, F2FS
POSIX	Certified compliant	Mostly compliant (uncertified)
Examples	AIX, HP-UX, Solaris, macOS	Ubuntu, Fedora, Arch, Debian
Shell	sh, ksh, csh	bash, zsh, fish, dash
Kernel	Monolithic or Microkernel	Monolithic + modular (LKMs)
Community	Corporate-driven	Massive open-source community
Updates	Vendor release cycles	Continuous / rolling possible

When to Choose UNIX vs Linux

Choose UNIX When...

- Mission-critical enterprise deployments
- Formal POSIX certification required
- Vendor support contract mandatory
- Running legacy enterprise apps (Oracle, SAP)
- Hardware-specific reliability (IBM AIX, HP-UX)

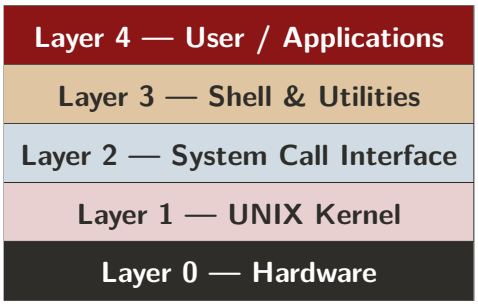
Choose Linux When...

- Cost is a concern
- Cloud / container workloads (Docker, K8s)
- Open-source development preferred
- Need broad hardware support
- DevOps, CI/CD, and automation pipelines

UNIX System Architecture

Layered Architecture Overview

The UNIX system is organized as a set of concentric layers, each providing services to the layer above it:



Detailed Component Description

Layer 0 — Hardware

The physical resources: CPU, RAM, storage devices (HDD/SSD/NVMe), network interfaces, and I/O peripherals. The OS manages these through hardware abstraction layers.

Layer 1 — The Kernel

The heart of UNIX. Runs in **privileged mode** (Ring 0 on x86) and manages:

- Process management & scheduling
- Memory management (virtual memory, paging)
- File system management (VFS)
- Device drivers
- Network stack (TCP/IP)
- Interprocess Communication (IPC)
- Signal handling
- Security enforcement

Layer 2 — System Call Interface

The well-defined boundary between user space and kernel space. Programs invoke system calls via software interrupts (e.g., `int 0x80` on x86 or `syscall` instruction on x86-64).

Layer 3 — Shell and Utilities

The shell (e.g., `bash`, `zsh`) is a command interpreter that reads user commands and translates them into system calls. Standard utilities (`ls`, `grep`, `awk`) provide the UNIX toolbox.

Layer 4 — User Applications

End-user programs: web browsers, editors, compilers, databases. These run in **user mode** (unprivileged) and access hardware only through the kernel via system calls.

User Mode vs Kernel Mode

Aspect	User Mode	Kernel Mode
Privilege Level	Unprivileged (Ring 3)	Privileged (Ring 0)
Memory Access	Virtual address space only	Full physical memory
Hardware Access	No direct access	Direct hardware access
Crash Impact	Kills only that process	Kernel panic (system crash)
Switching Cost	—	Context switch overhead
Examples	Firefox, gcc, Python	Scheduler, drivers, VFS

```
# View all running processes with CPU/MEM usage
$ ps aux | head -15
USER       PID %CPU %MEM    VSZ   RSS TTY STAT START   TIME COMMAND
root         1  0.0  0.1 167028 11936 ?   Ss   10:00   0:02 /sbin/init
root         2  0.0  0.0      0     0 ?   S    10:00   0:00 [kthreadd]
student   1234  2.1  1.5 456789 61440 pts/0 Sl 10:05   0:15 ./myapp

# Live CPU usage with process states
$ top -b -n 1 | head -20

# Show kernel ring buffer (boot messages)
$ dmesg | tail -20

# System calls made by a process (requires strace)
$ strace -c ls /etc 2>&1 | tail -10
```

Listing 7: Monitoring processes and CPU modes

The UNIX Kernel and System Call Interface

The UNIX Kernel

Definition — Kernel

The **kernel** is the core component of an operating system that manages hardware resources and provides fundamental services to user-space programs. In UNIX, the kernel is a single executable loaded at boot and remains resident in memory at all times.

Kernel Subsystems

The UNIX kernel is composed of several tightly integrated subsystems:

1. **Process Scheduler** — Decides which process runs next (CFS in Linux)
2. **Memory Manager** — Virtual memory, demand paging, page replacement
3. **Virtual File System (VFS)** — Abstraction layer over all file systems
4. **Network Stack** — BSD socket interface, TCP/IP protocol suite
5. **Device Drivers** — Hardware abstraction modules
6. **IPC Subsystem** — Pipes, signals, message queues, shared memory, sockets

System Calls

Definition — System Call

A **system call** (syscall) is a programmatic way for a user-space application to request a service from the kernel. It is the controlled entry point into the kernel — the only way a program can access hardware or other privileged operations.

System Call Categories

Category	Examples	Description
Process Control	`fork, exec, exit, wait, getpid`	Create/terminate processes
File Management	`open, read, write, close, lseek`	File I/O operations
Device Management	`ioctl, read, write`	Device control and I/O
Information	`getpid, alarm, time, uname`	Get/set system info
Communication	`pipe, socket, bind, accept`	IPC & networking
Memory	`mmap, munmap, brk, sbrk`	Memory allocation/mapping

How a System Call Works

```c
#include <stdio.h>
#include <unistd.h>     /* fork(), getpid() */
#include <sys/wait.h>   /* wait() */

int main(void) {
    pid_t pid;

    printf("Parent PID: %d\n", getpid());

    pid = fork();   /* <-- System call: creates a child process */

    if (pid < 0) {
        /* fork() failed */
        perror("fork");
        return 1;
    } else if (pid == 0) {
        /* Child process --- pid is 0 */
        printf("Child PID: %d, Parent: %d\n",
               getpid(), getppid());
        /* exec() a new program in child */
        execlp("/bin/ls", "ls", "-l", NULL);
    } else {
        /* Parent process --- pid is child's PID */
        printf("Parent waiting for child %d...\n", pid);
        wait(NULL);     /* Wait for child to finish */
        printf("Child process complete.\n");
    }
    return 0;
}
```

Listing 8: The fork() system call — creating a child process

```
Parent PID: 4521
Parent waiting for child 4522...
Child PID: 4522, Parent: 4521
total 24
-rwxr-xr-x 1 student staff 16384 Jan 15 10:22 fork_example
-rw-r--r-- 1 student staff   512 Jan 15 10:20 fork_example.c
Child process complete.
```

Listing 9: Output of fork() program

Tracing System Calls with `strace`

```
# Trace all system calls made by 'ls'
```

```
$ strace ls /tmp 2>&1 | head -30
execve("/usr/bin/ls", ["ls", "/tmp"], 0x7ffce4bcd9b0) = 0
brk(NULL)                                   = 0x55f8c0000000
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0"..., 832)      = 832
close(3)                                    = 0
openat(AT_FDCWD, "/tmp", O_RDONLY|O_DIRECTORY) = 3
getdents64(3, /* 5 entries */, 32768)   = 152
write(1, "file1.txt  file2.log\n", 21) = 21
close(3)                                    = 0
exit_group(0)                               = ?

# Count system call statistics
$ strace -c ls /tmp 2>&1
% time     seconds  usecs/call     calls    errors syscall
---------- ----------- ----------- --------- --------- --------
 42.31    0.000110          27         4           openat
 28.46    0.000074          18         4           read
 15.38    0.000040          40         1           write
  9.23    0.000024          24         1      execve
```

Listing 10: Using strace to observe system calls

The /proc Filesystem — A Window into the Kernel

Linux exposes kernel data structures through the virtual /proc filesystem:

```
# View info about process with PID 1 (init/systemd)
$ cat /proc/1/status | head -15
Name:   systemd
Umask:  0000
State:  S (sleeping)
Pid:    1
PPid:   0
VmPeak: 167028 kB
VmRSS:  11936 kB

# CPU information from kernel
$ cat /proc/cpuinfo | grep "model name" | head -2
model name : Intel(R) Core(TM) i7-1185G7 @ 3.00GHz
model name : Intel(R) Core(TM) i7-1185G7 @ 3.00GHz

# Memory usage
$ cat /proc/meminfo | head -10
MemTotal:       16248320 kB
MemFree:         4521344 kB
MemAvailable:    9876512 kB
Buffers:          346200 kB
Cached:          5213056 kB

# All system calls available on this kernel
$ ausyscall --dump | head -10
```

Listing 11: Exploring /proc to inspect kernel state

UNIX Directory Structure

The Hierarchical File System

Definition — FHS

The **Filesystem Hierarchy Standard (FHS)** defines the directory structure and directory contents in Linux and UNIX-like operating systems. All files reside under the **root directory** (/), forming an inverted tree.

The Root Directory Tree

```
/            Root of the entire filesystem
|--- bin/    Essential user binaries (ls, cp, mv, bash)
|--- boot/   Boot loader files, kernel images, initramfs
|--- dev/    Device files (block & character devices)
|--- etc/    System-wide configuration files
|--- home/   User home directories (/home/alice, /home/bob)
|--- lib/    Shared libraries (like DLLs on Windows)
|--- media/  Mount points for removable media
|--- mnt/    Temporary mount point for filesystems
|--- opt/    Optional/third-party application packages
|--- proc/   Virtual FS exposing kernel & process info
|--- root/   Root user's home directory
|--- run/    Runtime variable data (PIDs, sockets)
|--- sbin/   System administration binaries (root use)
|--- srv/    Data for services (HTTP, FTP)
|--- sys/    Virtual FS for hardware & device info
|--- tmp/    Temporary files (cleared on reboot)
|--- usr/    Secondary hierarchy (read-only user data)
`--- var/    Variable data (logs, spool, databases)
```

Critical Directory Descriptions

/etc — Configuration

All system-wide configuration files live here. No binaries.

```
$ ls /etc/
hosts           # Static hostname-to-IP mappings
hostname        # System hostname
passwd          # User account information
shadow          # Encrypted passwords (root-readable)
group           # Group definitions
fstab           # Filesystem mount table
crontab         # Scheduled tasks
ssh/            # SSH server and client configuration
```

Listing 12: Important files in /etc

/dev — Device Files

In UNIX, *everything is a file* — including hardware:

```
$ ls -la /dev/ | head -20
crw-rw-rw-  1 root tty   5,  0 Jan 15 10:00 tty
crw-rw-rw-  1 root root  1,  3 Jan 15 10:00 null
crw-rw-rw-  1 root root  1,  5 Jan 15 10:00 zero
brw-rw----  1 root disk  8,  0 Jan 15 10:00 sda    # Hard disk
brw-rw----  1 root disk  8,  1 Jan 15 10:00 sda1   # Partition 1
crw-rw-rw-  1 root root  1,  8 Jan 15 10:00 random

# Write to /dev/null (discard output)
```

```
$ some_noisy_command > /dev/null 2>&1

# Generate random bytes
$ head -c 16 /dev/urandom | xxd
00000000: a3f1 2b8e 4c91 d70a 3b55 f289 c14a 8e67
```

Listing 13: Device files in /dev

/proc and /sys — Virtual Filesystems

```
# Read CPU temperature via sysfs
$ cat /sys/class/thermal/thermal_zone0/temp
45000   # 45.0 degrees Celsius

# Tune kernel parameters via /proc
$ cat /proc/sys/vm/swappiness
60

# Temporarily change swappiness (survives until reboot)
$ echo 10 | sudo tee /proc/sys/vm/swappiness

# Persist kernel parameters across reboots
$ echo "vm.swappiness=10" | sudo tee -a /etc/sysctl.conf
$ sudo sysctl -p
```

Listing 14: Interacting with virtual filesystems

/var — Variable Data

```
$ ls /var/
log/     # System logs (syslog, auth.log, kern.log)
spool/   # Print queues, cron jobs, mail
lib/     # Application state (databases, package info)
tmp/     # Temp files preserved across reboots
cache/   # Application cache files
run/     # Runtime PIDs and sockets

# View system logs
$ tail -f /var/log/syslog
$ journalctl -f                    # systemd journal
$ journalctl -u nginx --since today # Service-specific logs
```

Listing 15: Important /var subdirectories

File Types in UNIX

Symbol	Type	Description
-	Regular file	Text, binary, images, executables
d	Directory	Container for other files/directories
l	Symbolic link	Pointer/alias to another file
c	Character device	Serial access (terminals, keyboard)
b	Block device	Block-level I/O (disks, USB drives)
p	Named pipe (FIFO)	IPC via pipe mechanism
s	Socket	Network or IPC endpoint

```
# The 'file' command identifies file type by content
$ file /bin/ls
/bin/ls: ELF 64-bit LSB pie executable, x86-64

$ file /etc/passwd
/etc/passwd: ASCII text

$ file /dev/sda
/dev/sda: block special (8/0)

# Hard link vs Symbolic link
$ ln   original.txt hard_link.txt       # Hard link
$ ln -s original.txt sym_link.txt       # Symbolic link

$ ls -lai
134217 -rw-r--r-- 2 student staff 1024 original.txt
134217 -rw-r--r-- 2 student staff 1024 hard_link.txt  # same inode!
134218 lrwxrwxrwx 1 student staff   12 sym_link.txt -> original.txt

# UNIX permissions: rwxrwxrwx (owner | group | others)
$ chmod 755 script.sh       # rwxr-xr-x
$ chmod u+x,g-w file.txt
$ chown alice:developers project/
```

Listing 16: Identifying and working with UNIX file types

Navigating the Directory Structure

```
# Absolute vs Relative paths
$ cd /var/log          # Absolute path (from /)
$ cd ../../etc         # Relative path (two levels up, then etc)
$ cd ~/projects        # ~ expands to home directory

# Find files across the directory tree
$ find / -name "*.conf" -type f 2>/dev/null | head -5
/etc/ssh/sshd_config
/etc/apt/apt.conf.d/70debconf
/etc/systemd/journald.conf

# Find recently modified files
$ find /var/log -mtime -1 -type f
/var/log/syslog
/var/log/auth.log

# Disk usage by directory
$ du -sh /var/log/*
4.5M    /var/log/syslog
1.2M    /var/log/auth.log
256K    /var/log/kern.log

# Overall disk usage
$ df -hT
Filesystem      Type    Size  Used Avail Use% Mounted on
/dev/sda1       ext4     50G   18G   30G  38% /
tmpfs           tmpfs   7.8G  0     7.8G   0% /dev/shm
```

Listing 17: Path navigation and file searching

Unit 1 Summary

Key Takeaways — Unit 1: Introduction to UNIX

1. **UNIX** is a multiuser, multitasking OS born at Bell Labs in 1969, foundational to modern computing.
2. **POSIX** standardizes the API across UNIX-like systems, ensuring portability of C programs and shell scripts.
3. **Linux** is a free, open-source UNIX-like kernel by Linus Torvalds (1991) that powers 90%+ of servers and the cloud.
4. **Linux distributions** package the kernel with tools, package managers, and software for specific use cases.
5. **UNIX vs Linux** — UNIX is certified, often proprietary; Linux is open-source, hardware-agnostic, community-driven.
6. **UNIX Architecture** is layered: Hardware → Kernel → System Calls → Shell/Utilities → Applications.
7. **The Kernel** manages processes, memory, file systems, devices, and networking; system calls are the interface to it.
8. **Directory Structure** (FHS) is a single tree rooted at `/`, with each directory serving a specific role.

End of Unit 1 — Introduction to UNIX
Proceed to Unit 2: UNIX File Commands

Unit 1 — Practice Questions

Practice Questions — Unit 1: Introduction to UNIX

A. Short Answer (2 marks each)

1. What does the acronym POSIX stand for? Which organisation maintains it?
2. State **three** key differences between UNIX and Linux.
3. What is a Linux distribution? Name any two Debian-based and two Red Hat-based distros.
4. Explain the role of the `/etc` directory in the FHS hierarchy.
5. What is a system call? Give two examples of system calls used for file I/O.
6. Define "kernel mode" and "user mode". Why does this separation exist?
7. Who created the Linux kernel, and in which year? What licence is it released under?
8. What is the UNIX philosophy? State McIlroy's three rules.

B. Fill in the Blanks (1 mark each)

1. UNIX was originally developed at _____ in the year _____.
2. The command to display the running kernel version is _____.
3. The virtual filesystem that exposes kernel internals is _____.
4. PID 1 on a modern Linux system is typically the _____ process.
5. The UNIX layer directly above the kernel is the _____.

C. Long Answer (5 marks each)

1. Draw and explain the layered architecture of a UNIX system. What happens at each layer?
2. Compare UNIX and Linux across at least **six** criteria using a table.
3. Describe the complete Linux directory structure (FHS). Give the purpose of any **eight** top-level directories.

D. Command Practice

1. Write the command to: (a) display all system info, (b) view the first 10 lines of `/proc/cpuinfo`, (c) list all loaded kernel modules.
2. Using `strace`, how would you count the system calls made by the `ls` command? Write the exact command.

Unit 2: UNIX File Commands

Unit 2 — UNIX File Commands

Directory Management · File Operations · Permissions · Timestamps · Links · i-Nodes · Redirection · Pipes

UNIX File Commands — Overview

The UNIX file system is the backbone of the operating system. Everything in UNIX is treated as a file — regular files, directories, devices, and sockets. UNIX provides a rich set of commands to create, inspect, move, delete, and secure files and directories.

The Golden Rule of UNIX Files

In UNIX, **everything is a file**. Hard disks, terminals, printers, and network sockets all appear as files in the directory tree. This unified model makes the system extremely powerful and consistent.

Category	Commands	Purpose
Directory Mgmt	`mkdir, rmdir, cd, pwd`	Create/navigate directories
File Operations	`touch, cp, mv, rm, cat`	Create/copy/move/delete files
Listing	`ls, file, stat`	Inspect files and directories
Permissions	`chmod, chown, chgrp, umask`	Control access rights
Timestamps	`touch -d, touch -t, stat`	Manage file timestamps
Search	`find, locate, which, whereis`	Locate files/commands
Links	`ln, ln -s`	Hard and symbolic links
I/O Redirection	`>, >>, <, 2>`	Redirect input and output
Pipelines	`|, tee`	Chain commands together

Create Directory — `mkdir`

Definition — Directory

A **directory** (called a folder in GUI systems) is a special file that contains a list of other files and directories. In UNIX, directories form the hierarchical tree structure rooted at `/`.

The `mkdir` (*make directory*) command creates one or more new directories.

Syntax:

```
mkdir [OPTIONS] directory_name(s)
```

Listing 18: mkdir — basic syntax and options

Option	Long Form	Description
-p	--parents	Create parent directories as needed; no error if exists
-m	--mode	Set permission mode (like chmod) at creation time
-v	--verbose	Print a message for each created directory

```
# Create a single directory
$ mkdir projects

# Create multiple directories at once
$ mkdir docs images videos

# Create nested directory tree in one command (-p flag)
$ mkdir -p projects/unix/lab1/solutions
# Without -p, this would fail if 'projects' didn't exist

# Create directory with specific permissions
$ mkdir -m 755 public_html

# Verbose output shows what was created
$ mkdir -pv /tmp/test/a/b/c
mkdir: created directory '/tmp/test'
mkdir: created directory '/tmp/test/a'
mkdir: created directory '/tmp/test/a/b'
mkdir: created directory '/tmp/test/a/b/c'

# Verify the created structure
$ ls -la projects/
total 0
drwxr-xr-x 3 student staff  96 Jan 15 10:00 .
drwxr-xr-x 8 student staff 256 Jan 15 10:00 ..
drwxr-xr-x 3 student staff  96 Jan 15 10:00 unix
```

Listing 19: mkdir usage examples

Good Naming Practice

Avoid spaces in directory names. Use _ (underscore) or - (hyphen) instead. Spaces require quoting or escaping: `mkdir "my folder"` or `mkdir my\folder`.

Change Directory — cd

The cd (*change directory*) command moves your current working location within the directory tree.

Syntax:

```
cd [directory_path]
```

Listing 20: cd — syntax

```
# Print current directory
```

```
$ pwd
/home/student

# Move to an absolute path (starts from /)
$ cd /var/log
$ pwd
/var/log

# Move to a relative path (relative to current location)
$ cd ../..          # Two levels up
$ pwd
/

# Return to home directory (all three are equivalent)
$ cd
$ cd ~
$ cd $HOME

# Jump to the previous directory (toggle between two locations)
$ cd /etc
$ cd /var/log
$ cd -              # Goes back to /etc
/etc

# Move into a subdirectory
$ cd /home/student
$ cd projects/unix  # Relative path
$ pwd
/home/student/projects/unix
```

Listing 21: cd usage — absolute, relative, and special paths

Shortcut	Meaning
`cd`	Go to home directory
`cd $HOME`	Go to home directory (explicit)
`cd -`	Go to previous directory
`cd ..`	Go one level up (parent directory)
`cd ../..`	Go two levels up
`cd /`	Go to root directory

Move and Copy Directory

cp — Copy Files and Directories

Definition — Copy

cp creates a **new, independent copy** of a file or directory. Changes to the copy do not affect the original.

```
# Basic syntax
cp [OPTIONS] source destination

# Copy a single file
$ cp report.txt report_backup.txt

# Copy file to another directory
```

```
$ cp report.txt /tmp/

# Copy and rename at the same time
$ cp report.txt /tmp/report_v2.txt

# Copy multiple files to a directory
$ cp file1.txt file2.txt file3.txt /tmp/

# Copy an entire directory RECURSIVELY (-r flag is mandatory)
$ cp -r projects/ projects_backup/

# Preserve timestamps, ownership, permissions (-p flag)
$ cp -p original.txt preserved_copy.txt

# Interactive mode --- ask before overwriting
$ cp -i file.txt /tmp/file.txt
cp: overwrite '/tmp/file.txt'? y

# Verbose --- show what is being copied
$ cp -rv projects/ /backup/
'projects/' -> '/backup/projects/'
'projects/unix/' -> '/backup/projects/unix/'

# Copy only if source is NEWER than destination
$ cp -u *.txt /backup/
```

Listing 22: cp — copy files and directories

Option	Long Form	Effect
-r	--recursive	Copy directories and all their contents
-p	--preserve	Preserve mode, ownership, timestamps
-i	--interactive	Prompt before overwriting
-u	--update	Copy only when source is newer
-v	--verbose	Show each file being copied
-f	--force	Force overwrite, no prompts

mv — Move or Rename Files and Directories

Definition — Move

mv either **renames** a file (same directory) or **moves** it to another location. Unlike cp, the original is removed.

```
# Basic syntax
mv [OPTIONS] source destination

# Rename a file
$ mv old_name.txt new_name.txt

# Move a file to another directory
$ mv report.txt /tmp/

# Move and rename simultaneously
$ mv report.txt /archive/report_2024.txt

# Move an entire directory
```

```
$ mv projects/ /home/student/workspace/

# Rename a directory
$ mv lab1/ laboratory1/

# Interactive --- prompt before overwriting
$ mv -i source.txt /tmp/source.txt
mv: overwrite '/tmp/source.txt'? y

# Do NOT move if destination exists
$ mv -n source.txt /tmp/source.txt

# Verbose output
$ mv -v *.log /var/archive/
'system.log' -> '/var/archive/system.log'
'error.log'  -> '/var/archive/error.log'
```

Listing 23: mv — move and rename

mv vs cp

`mv` is instant when source and destination are on the **same filesystem** (just updates the directory entry). Across filesystems, `mv` copies the data then deletes the source — similar to `cp` followed by `rm`.

Create File

UNIX offers several ways to create files depending on the use case.

Using `touch`

`touch` creates an empty file if it does not exist, or updates its timestamps if it does.

```
# Create a single empty file
$ touch newfile.txt

# Create multiple files at once
$ touch file1.txt file2.txt file3.py

# Verify file was created (size is 0 bytes)
$ ls -lh newfile.txt
-rw-r--r-- 1 student staff 0 Jan 15 10:30 newfile.txt
```

Listing 24: Creating files with touch

Using `cat` with Redirection

```
# Create a file and type content (Ctrl+D to save)
$ cat > hello.txt
Hello, this is my first UNIX file.
Type as many lines as you want.
Press Ctrl+D when done.
^D

# Verify content
$ cat hello.txt
Hello, this is my first UNIX file.
Type as many lines as you want.
Press Ctrl+D when done.

# Create file with content in one line using echo
$ echo "Hello UNIX" > greeting.txt
```

```
$ cat greeting.txt
Hello UNIX
```

Listing 25: Creating files with cat

Using Text Editors

```
# vi / vim  (universal UNIX editor)
$ vi myfile.txt        # Press 'i' to insert, Esc then :wq to save

# nano  (beginner-friendly)
$ nano myfile.txt      # Ctrl+O to save, Ctrl+X to exit

# gedit (GUI-based, if desktop available)
$ gedit myfile.txt
```

Listing 26: Creating files with editors

Using `printf` or Here-Documents

```
# printf for formatted content
$ printf "Name: Alice\nAge: 22\nCourse: BCA\n" > student.txt
$ cat student.txt
Name: Alice
Age: 22
Course: BCA

# Here-document (heredoc) --- great for multi-line content in scripts
$ cat > config.txt << 'EOF'
hostname=myserver
port=8080
debug=false
EOF

$ cat config.txt
hostname=myserver
port=8080
debug=false
```

Listing 27: Advanced file creation techniques

Remove File — `rm`

Definition — rm

`rm` (*remove*) permanently deletes files. Unlike Windows/macOS, **there is no Recycle Bin** in UNIX by default. Deleted files are gone immediately.

```
# Basic syntax
rm [OPTIONS] file(s)

# Remove a single file
$ rm oldfile.txt

# Remove multiple files
$ rm file1.txt file2.txt file3.log

# Interactive mode --- ALWAYS ask before deleting
$ rm -i important.txt
rm: remove regular file 'important.txt'? y

```

```
# Remove all .tmp files in current directory
$ rm *.tmp

# Force remove --- no error if file doesn't exist, no prompts
$ rm -f ghost.txt       # No error even if ghost.txt doesn't exist

# Verbose --- confirm each deletion
$ rm -v *.log
removed 'access.log'
removed 'error.log'
removed 'system.log'
```

Listing 28: rm — removing files safely and forcefully

Danger — The Most Dangerous UNIX Command

```
# NEVER run this as root --- deletes EVERYTHING recursively
$ rm -rf /          # Destroys the entire filesystem!

# Safer alternative: always specify a real path
$ rm -rf /tmp/my_temp_folder/   # This is fine
```

Modern systems have a `--no-preserve-root` guard, but always double-check your path before using `-rf`.

Remove Directory — rmdir and rm -r

rmdir — Remove Empty Directories

```
# Remove a single empty directory
$ rmdir old_folder

# Remove multiple empty directories
$ rmdir temp1 temp2 temp3

# Remove nested empty directories (-p flag)
$ rmdir -p a/b/c     # Removes c, then b, then a (all must be empty)

# If directory is NOT empty, rmdir will fail
$ rmdir projects
rmdir: failed to remove 'projects': Directory not empty
```

Listing 29: rmdir — only removes empty directories

rm -r — Remove Non-Empty Directories

```
# Remove a directory and ALL its contents
$ rm -r old_project/

# Interactive recursive removal (safer!)
$ rm -ri backup/
rm: descend into directory 'backup/'? y
rm: remove regular file 'backup/file1.txt'? y
rm: remove directory 'backup/'? y

# Force + recursive (no prompts --- use with extreme care)
$ rm -rf build/      # Common in development to clean build artifacts

# Verbose recursive removal
$ rm -rfv /tmp/test_data/
removed '/tmp/test_data/a.txt'
```

```
removed '/tmp/test_data/b.txt'
removed directory '/tmp/test_data/'
```

Listing 30: rm -r — recursive directory removal

Command	Removes	Condition
`rmdir dir`	Empty directory only	Fails if not empty
`rmdir -p a/b/c`	Nested empty dirs	All must be empty
`rm -r dir`	Directory + all contents	Works always
`rm -rf dir`	Directory + all, no prompts	Use with caution

Listing Directory Information — `ls`

`ls` is one of the most frequently used UNIX commands. It lists files and directories along with their attributes.

```
# Basic listing
$ ls
Desktop  Documents  Downloads  Music  Pictures  projects

# Long listing format (-l) --- shows ALL attributes
$ ls -l
total 48
drwxr-xr-x 5 student staff  160 Jan 15 10:00 Desktop
drwxr-xr-x 3 student staff   96 Jan 14 09:22 Documents
-rw-r--r-- 1 student staff 2048 Jan 15 09:30 notes.txt
lrwxrwxrwx 1 student staff   12 Jan 13 11:00 link -> notes.txt

# Show hidden files (files starting with .)
$ ls -a
.  ..  .bashrc  .profile  Desktop  Documents  notes.txt

# Long + hidden files combined
$ ls -la

# Sort by modification time (newest first)
$ ls -lt

# Sort by size (largest first)
$ ls -lS

# Reverse sort order
$ ls -lr

# Human-readable file sizes (KB, MB, GB)
$ ls -lh
total 4.0K
-rw-r--r-- 1 student staff 2.0K Jan 15 notes.txt

# Recursive listing (show all subdirectories)
$ ls -lR

# Show inode number
$ ls -li
134217 -rw-r--r-- 1 student staff 2048 Jan 15 notes.txt

# Classify files with indicator character
$ ls -F
```

```
Desktop/  Documents/  notes.txt  script.sh*  link@
# / = directory, * = executable, @ = symlink
```

Listing 31: ls and its most important options

Understanding the Long Listing Format

```
-rw-r--r-- 1 student staff 2048 Jan 15 10:30 notes.txt
|________| | |_____| |___| |__| |__________| |_______|
     |      |     |      |    |       |            |
   Type &  Links Owner  Group Size  Timestamp    Name
   Perms
```

Listing 32: Anatomy of ls -l output

Field	Example	Meaning
File type & perms	`-rw-r--r--`	Regular file; owner rw, group r, others r
Hard link count	`1`	Number of hard links to this inode
Owner	`student`	User who owns the file
Group	`staff`	Group that owns the file
Size	`2048`	Size in bytes (use `-h` for KB/MB)
Timestamp	`Jan 15`	Last modification time
Name	`notes.txt`	File or directory name

```
$ stat notes.txt
  File: notes.txt
  Size: 2048        Blocks: 8          IO Block: 4096   regular file
Device: 803h/2051d  Inode: 134217      Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/student)  Gid: (1000/staff)
Access: 2024-01-15 10:30:22.000000000 +0530
Modify: 2024-01-15 09:25:11.000000000 +0530
Change: 2024-01-15 09:25:11.000000000 +0530
 Birth: 2024-01-14 08:00:00.000000000 +0530
```

Listing 33: Using stat for full file information

Types of Files in UNIX

UNIX recognizes **seven types of files**. The file type is shown as the first character in the `ls -l` output.

Symbol	Type	Created By	Description
`-`	Regular file	`touch`, editors, programs	Stores data: text, binary, scripts, images
`d`	Directory	`mkdir`	Container holding filenames and inodes
`l`	Symbolic link	`ln -s`	Pointer to another file/directory path
`c`	Character device	Kernel (at boot)	Serial I/O: terminals, keyboards, `/dev/null`
`b`	Block device	Kernel (at boot)	Block I/O: hard disks, USB drives, SSDs
`p`	Named pipe (FIFO)	`mkfifo`	IPC mechanism; one process writes, another reads
`s`	Socket	Programs (e.g. `nginx`)	Bidirectional IPC endpoint (network or local)

```
# Regular file
$ ls -l /etc/passwd
-rw-r--r-- 1 root root 2734 Jan 15 /etc/passwd

# Directory
$ ls -ld /etc
drwxr-xr-x 131 root root 4096 Jan 15 /etc

# Symbolic link
$ ls -l /bin
lrwxrwxrwx 1 root root 7 Jan  1 /bin -> usr/bin

# Character device (terminal, null device)
$ ls -l /dev/null /dev/tty
crw-rw-rw- 1 root root 1, 3 Jan 15 /dev/null
crw-rw-rw- 1 root tty  5, 0 Jan 15 /dev/tty

# Block device (hard disk)
$ ls -l /dev/sda
brw-rw---- 1 root disk 8, 0 Jan 15 /dev/sda

# Named pipe (FIFO)
$ mkfifo mypipe
$ ls -l mypipe
prw-r--r-- 1 student staff 0 Jan 15 mypipe

# Socket (created by running programs)
$ ls -l /run/systemd/private/journal.socket
srw-rw-rw- 1 root root 0 Jan 15 /run/systemd/private/journal.socket

# The 'file' command inspects content to determine type
$ file /etc/passwd /bin/ls /dev/sda
/etc/passwd: ASCII text
/bin/ls:     ELF 64-bit LSB pie executable, x86-64
/dev/sda:    block special (8/0)
```

Listing 34: Identifying all seven file types

Change File Security — chmod, chown, chgrp

Understanding UNIX Permissions

Every file has three sets of permissions, each with three bits:

Owner (u)			Group (g)			Others (o)		
r	w	x	r	w	x	r	w	x
4	2	1	4	2	1	4	2	1

Permission	Symbol	Value	Meaning on File / Directory
Read	r	4	File: view content; Dir: list files
Write	w	2	File: modify content; Dir: create/delete files
Execute	x	1	File: run as program; Dir: enter with cd
None	-	0	No permission

chmod — Change File Mode

```
# ---- SYMBOLIC MODE ----
# Syntax: chmod [who][+/-/=][permissions] file
# who: u=owner, g=group, o=others, a=all

# Add execute permission for owner
$ chmod u+x script.sh

# Remove write permission from group and others
$ chmod go-w sensitive.txt

# Give read+write to owner, read-only to group, nothing to others
$ chmod u=rw,g=r,o= private.txt

# Add execute to all (owner, group, others)
$ chmod a+x deploy.sh

# ---- OCTAL MODE ----
# Each digit = sum of r(4) + w(2) + x(1)

# rwxr-xr-x = 7 5 5  (common for scripts/executables)
$ chmod 755 script.sh

# rw-r--r-- = 6 4 4  (common for data files)
$ chmod 644 notes.txt

# rw------- = 6 0 0  (private to owner only)
$ chmod 600 ~/.ssh/id_rsa

# rwxrwxrwx = 7 7 7  (full access to everyone --- AVOID)
$ chmod 777 shared.txt   # Security risk!

# Apply recursively to directory and all contents
$ chmod -R 755 public_html/

# Verify the change
$ ls -l script.sh
-rwxr-xr-x 1 student staff 512 Jan 15 script.sh
```

Listing 35: chmod — symbolic and octal modes

Octal	Symbolic	Common Use	Meaning
777	rwxrwxrwx	Avoid	Full access for everyone
755	rwxr-xr-x	Scripts/dirs	Owner full; others read+execute
644	rw-r--r--	Text files	Owner rw; others read-only
600	rw-------	Private keys	Owner rw only
700	rwx------	Private dirs	Owner full; no access for others
444	r--r--r--	Read-only	Everyone can read, nobody can write

`chown` — Change File Owner

```
# Syntax: chown [owner][:group] file

# Change owner only
$ sudo chown alice file.txt

# Change owner and group simultaneously
$ sudo chown alice:developers file.txt

# Change group only (note the colon prefix)
$ sudo chown :developers file.txt

# Recursive ownership change
$ sudo chown -R www-data:www-data /var/www/html/

# Verify
$ ls -l file.txt
-rw-r--r-- 1 alice developers 1024 Jan 15 file.txt
```

Listing 36: chown — changing ownership

`chgrp` — Change Group Ownership

```
# Change group of a file
$ chgrp staff report.txt

# Change group recursively
$ chgrp -R developers /opt/project/

# Verify group change
$ ls -l report.txt
-rw-r--r-- 1 student staff 512 Jan 15 report.txt
```

Listing 37: chgrp — changing group

Creating Empty Files and Changing Timestamps — `touch`

Definition — touch

`touch` serves two purposes: (1) **create** an empty file if it does not exist, and (2) **update the timestamps** (access time and modification time) of an existing file to the current time.

```
# Create one or more empty files
$ touch empty1.txt empty2.txt empty3.txt

# If file exists, touch updates its timestamps to NOW
```

```
$ stat notes.txt | grep Modify
Modify: 2024-01-10 08:00:00

$ touch notes.txt

$ stat notes.txt | grep Modify
Modify: 2024-01-15 10:45:00    # Updated to current time

# Update ONLY the access time (-a flag)
$ touch -a notes.txt

# Update ONLY the modification time (-m flag)
$ touch -m notes.txt
```

Listing 38: touch — creating files and updating timestamps

Access Time, Modification Time and Change Time

Every UNIX file maintains **three timestamps**, each recording a different kind of "last event":

Name	Field	Updated When...
Access Time	`atime`	The file's **contents are read** (`cat`, `less`, a program reads it)
Modification Time	`mtime`	The file's **contents are changed** (data written to the file)
Change Time	`ctime`	The file's **metadata changes** (permissions, owner, rename, link count)

ctime ≠ Creation Time

Many beginners confuse `ctime` with "creation time". In UNIX, `ctime` is the **inode change time** — it is updated whenever the file's metadata changes (e.g. you `chmod` or `chown` the file). The actual creation time (`btime`/`crtime`) is only available on some filesystems like ext4, via `stat`.

```
# Create a test file
$ echo "hello" > test.txt
$ stat test.txt
  File: test.txt
  Size: 6
Access: 2024-01-15 10:00:00   # atime
Modify: 2024-01-15 10:00:00   # mtime
Change: 2024-01-15 10:00:00   # ctime

# --- Reading the file updates atime ---
$ cat test.txt
$ stat test.txt | grep Access
Access: 2024-01-15 10:05:00   # atime updated

# --- Writing to the file updates mtime AND ctime ---
$ echo "world" >> test.txt
$ stat test.txt
Access: 2024-01-15 10:05:00
Modify: 2024-01-15 10:08:00   # mtime updated
Change: 2024-01-15 10:08:00   # ctime updated (content = metadata change)
```

```

# --- Changing permissions updates ONLY ctime ---
$ chmod 600 test.txt
$ stat test.txt
Access: 2024-01-15 10:05:00   # unchanged
Modify: 2024-01-15 10:08:00   # unchanged
Change: 2024-01-15 10:12:00   # ctime updated!
```

Listing 39: Observing atime, mtime, and ctime with stat

```
# Find files accessed in the last 1 day
$ find /home -atime -1

# Find files modified more than 7 days ago
$ find /tmp -mtime +7

# Find files whose metadata changed in the last hour (ctime)
$ find /etc -cmin -60

# List files sorted by modification time (newest first)
$ ls -lt
```

Listing 40: Using find with timestamps

touch with -d and -t Options

Instead of setting timestamps to *now*, `touch` can set them to a **specific date and time** using `-d` or `-t`.

The -d Option — Human-Readable Date String

```
# Syntax: touch -d "date string" file

# Set timestamp to a specific date
$ touch -d "2024-06-15" report.txt
$ stat report.txt | grep -E "Access|Modify"
Access: 2024-06-15 00:00:00
Modify: 2024-06-15 00:00:00

# Set specific date AND time
$ touch -d "2024-06-15 14:30:00" report.txt

# Use natural language (if supported by your system's date parser)
$ touch -d "yesterday" notes.txt
$ touch -d "last Monday" archive.txt
$ touch -d "2 days ago" oldfile.txt
$ touch -d "next Friday 09:00" future.txt

# Verify
$ stat report.txt | grep Modify
Modify: 2024-06-15 14:30:00.000000000 +0530
```

Listing 41: touch -d — set timestamp using natural date strings

The -t Option — Numeric Timestamp Format

```
# Syntax: touch -t [[CC]YY]MMDDhhmm[.SS] file
# CC = century, YY = year, MM = month, DD = day
# hh = hour, mm = minute, SS = seconds

# Set to January 20, 2024 at 09:30:00
$ touch -t 202401200930.00 document.txt
```

```

# Set to December 31, 2023 at 23:59
$ touch -t 202312312359 oldyear.txt

# Set to March 15 at 08:00 (current year assumed)
$ touch -t 03150800 quarterly.txt

# Verify
$ stat document.txt | grep Modify
Modify: 2024-01-20 09:30:00.000000000 +0530
```

Listing 42: touch -t — set timestamp using numeric format

Option	Format	Example
-d	Human-readable string	touch -d "2024-06-15 14:30" f.txt
-t	YYYYMMDDhhmm.SS	touch -t 202406151430.00 f.txt
-a	(no date needed)	Update access time only to now
-m	(no date needed)	Update modification time only to now

Finding Files and Directories

find — Search by Any Criterion

Definition — find

find is the most powerful UNIX search tool. It traverses the directory tree and matches files based on name, type, size, timestamp, permissions, owner, and more — then optionally executes actions on matches.

```
# Basic syntax
find [path] [expression] [action]

# --- SEARCH BY NAME ---
# Find all .txt files anywhere under /home
$ find /home -name "*.txt"

# Case-insensitive name search
$ find /home -iname "readme*"

# Find files named exactly 'passwd'
$ find / -name "passwd" 2>/dev/null

# --- SEARCH BY TYPE ---
# Find only directories
$ find /var -type d

# Find only regular files
$ find /etc -type f

# Find symbolic links
$ find /usr -type l

# --- SEARCH BY SIZE ---
# Files larger than 100MB
$ find /var -size +100M

# Files smaller than 1KB
```

```
$ find /tmp -size -1k

# Files exactly 512 bytes
$ find . -size 512c

# --- SEARCH BY TIME ---
# Modified in the last 7 days
$ find /home -mtime -7

# Accessed more than 30 days ago
$ find /tmp -atime +30

# Changed in the last 60 minutes
$ find /etc -cmin -60

# --- SEARCH BY PERMISSIONS ---
# Files with exact permissions 777 (dangerous!)
$ find / -perm 777

# Files with SUID bit set
$ find / -perm -4000 2>/dev/null

# --- SEARCH BY OWNER ---
$ find /home -user alice
$ find /var -group www-data

# --- ACTIONS ON RESULTS ---
# Delete all .tmp files found
$ find /tmp -name "*.tmp" -delete

# Execute a command on each result
$ find . -name "*.log" -exec ls -lh {} \;

# Print with null separator (safe for filenames with spaces)
$ find . -name "*.txt" -print0 | xargs -0 wc -l
```

Listing 43: find — comprehensive examples

`locate` — Fast Index-Based Search

```
# Search for a file by name (very fast, uses pre-built index)
$ locate passwd
/etc/passwd
/etc/pam.d/passwd
/usr/share/man/man5/passwd.5.gz

# Case-insensitive search
$ locate -i readme

# Update the locate database (run as root, needed after new files)
$ sudo updatedb

# Note: locate may miss very recently created files
# Use 'find' for real-time accuracy
```

Listing 44: locate — instant search via database

`which` and `whereis`

```
# which: finds the FIRST executable in your PATH
$ which python3
/usr/bin/python3

```

```
$ which ls
/usr/bin/ls

# whereis: finds binary, source code, AND man page
$ whereis ls
ls: /usr/bin/ls /usr/share/man/man1/ls.1.gz

$ whereis python3
python3: /usr/bin/python3 /usr/lib/python3 /usr/share/man/man1/python3.1.gz
```

Listing 45: which and whereis — finding commands

Soft Link and Hard Link

UNIX supports two kinds of file links, each with very different behaviour.

Hard Links

Definition — Hard Link

A **hard link** is a second (or third, or fourth…) **directory entry pointing to the same inode**. There is no original and no copy — all hard links are equal references to the same data on disk.

```
# Syntax: ln source link_name
$ ln original.txt hardlink.txt

# Both names point to the SAME inode (same data, same permissions)
$ ls -li original.txt hardlink.txt
134217 -rw-r--r-- 2 student staff 1024 Jan 15 hardlink.txt
134217 -rw-r--r-- 2 student staff 1024 Jan 15 original.txt
#      ^                ^ link count = 2 (two names for this inode)
#      same inode!

# Editing via either name modifies the same data
$ echo "added line" >> hardlink.txt
$ cat original.txt        # Shows the addition!

# Deleting ONE name does NOT remove the data
$ rm original.txt
$ cat hardlink.txt        # Still works! Data survives.

# Data is only deleted when link count reaches 0
```

Listing 46: Hard links — creation and behaviour

Hard Link Restrictions

- Hard links **cannot span filesystems** (both must be on the same partition)
- Hard links **cannot link to directories** (prevents circular loops)
- Hard links work only for **regular files**

Symbolic (Soft) Links

Definition — Symbolic Link

A **symbolic link** (symlink or soft link) is a special file that contains a **text path** pointing to another file or directory. It is similar to a Windows shortcut.

```
# Syntax: ln -s target link_name
$ ln -s original.txt symlink.txt

# Symlink has its OWN inode (different from original)
$ ls -li original.txt symlink.txt
134217 -rw-r--r-- 1 student staff 1024 Jan 15 original.txt
134220 lrwxrwxrwx 1 student staff   12 Jan 15 symlink.txt -> original.txt
#      ^              ^ starts with 'l' (symlink), different inode

# Can link to directories (hard links cannot!)
$ ln -s /var/log logs          # 'logs' now points to /var/log
$ ls logs/                     # Works like cd /var/log

# Can span filesystems
$ ln -s /mnt/usb/data.txt local_link.txt

# Dangling symlink --- original deleted, link becomes broken
$ rm original.txt
$ cat symlink.txt
cat: symlink.txt: No such file or directory   # Dangling!

# Find all broken (dangling) symlinks
$ find . -xtype l

# Remove a symlink (use rm, NOT rmdir)
$ rm symlink.txt
```

Listing 47: Symbolic links — creation and behaviour

Hard Link vs Soft Link — Comparison

Feature	Hard Link	Soft (Symbolic) Link
Inode	Same inode as original	Own separate inode
Cross-filesystem	No	Yes
Link to dirs	No (usually)	Yes
Dangling link	Impossible	Possible (if original deleted)
If original deleted	Data persists	Link becomes broken
File type char	`-` (same as file)	`l` (symlink)
Size shown	Actual data size	Length of path string
Permissions	Inherited from inode	Always `lrwxrwxrwx`
Use case	Backup aliases, `/bin`	Shortcuts, version aliases

The i-Node (Index Node)

Definition — i-Node

An **i-node** (index node) is a data structure on disk that stores all **metadata** about a file except its name. Every file and directory has exactly one inode. The directory entry simply maps a *filename* to an *inode number*.

What an i-Node Stores

Field	Contents
File type	Regular, directory, symlink, device, etc.
Permissions	`rwxrwxrwx` bits + SUID/SGID/Sticky
Owner UID	Numeric user ID of file owner
Owner GID	Numeric group ID
File size	Size in bytes
Timestamps	`atime`, `mtime`, `ctime`
Hard link count	Number of directory entries pointing here
Data block pointers	Addresses of disk blocks holding file content

What an i-Node Does NOT Store

The **filename** is NOT stored in the inode — it lives in the *directory entry* that points to the inode. This is exactly why you can have multiple filenames (hard links) for a single inode.

```
# Show inode number with ls -i
$ ls -li /etc/passwd
131073 -rw-r--r-- 1 root root 2734 Jan 15 /etc/passwd

# Show inode with stat
$ stat /etc/passwd | grep Inode
Device: 803h/2051d  Inode: 131073  Links: 1

# See inode usage on the filesystem
$ df -i
Filesystem      Inodes  IUsed   IFree IUse% Mounted on
/dev/sda1       655360  89412  565948   14% /
tmpfs           498765    952  497813    1% /dev/shm

# Find a file by its inode number
$ find / -inum 131073 2>/dev/null
/etc/passwd
```

Listing 48: Inspecting inodes

How a Filename Resolves to Data

The path `/home/student/notes.txt` is resolved in three steps:

1. Look up `notes.txt` in the **directory entry** of `/home/student/` → get inode number, e.g. 134217
2. Read **inode 134217** → get block pointers, permissions, size
3. Follow block pointers to the **disk blocks** that hold the actual file content

Size of the File System — `df` and `du`

`df` — Disk Free (Filesystem Level)

```
# Show disk space for all mounted filesystems
$ df
```

```
Filesystem     1K-blocks    Used Available Use% Mounted on
/dev/sda1       51475068 18432000  30368608  38% /
tmpfs            8021988       0   8021988   0% /dev/shm

# Human-readable sizes (KB, MB, GB)
$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        50G   18G   29G  38% /
tmpfs           7.7G     0  7.7G   0% /dev/shm

# Show filesystem type as well
$ df -hT
Filesystem     Type   Size  Used Avail Use% Mounted on
/dev/sda1      ext4    50G   18G   29G  38% /

# Check inode usage (not block usage)
$ df -i
Filesystem     Inodes  IUsed   IFree IUse% Mounted on
/dev/sda1      655360  89412  565948   14% /

# Show info for a specific directory
$ df -h /var
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        50G   18G   29G  38% /
```

Listing 49: df — check overall filesystem space

du — Disk Usage (Directory/File Level)

```
# Disk usage of current directory (recursive)
$ du
1024    ./projects/unix
2048    ./projects
512     ./docs
4096    .

# Human-readable
$ du -h
1.0M    ./projects/unix
2.0M    ./projects
512K    ./docs
4.0M    .

# Summary only (total for a directory, no subdirs)
$ du -sh /var/log
145M    /var/log

# Show each file AND directory separately
$ du -ah /var/log | sort -rh | head -10
145M    /var/log
 98M    /var/log/syslog
 23M    /var/log/journal
  8M    /var/log/auth.log

# Limit depth of display
$ du -h --max-depth=1 /usr
4.0G    /usr/lib
2.1G    /usr/share
389M    /usr/bin
6.7G    /usr
```

Listing 50: du — check size of files and directories

Command	Scope	Best For
`df -h`	Entire filesystem	See if disk is nearly full
`df -i`	Inode usage	Diagnose "no space" despite free blocks
`du -sh dir`	Single directory	How big is this folder?
`du -ah | sort -rh`	All files ranked	Find the largest files

Output Redirection — > and >>

Definition — Redirection

Redirection changes where a command's input or output goes. By default, commands read from **stdin** (keyboard) and write to **stdout** (terminal). Redirection lets you connect these to files instead.

Standard File Descriptors

Name	FD	Default	Redirect	Example
stdin	0	Keyboard	`< file`	`sort < unsorted.txt`
stdout	1	Terminal	`> file`	`ls > listing.txt`
stderr	2	Terminal	`2> file`	`cmd 2> errors.txt`

The > Operator — Overwrite

```
# Redirect ls output to a file (creates/overwrites)
$ ls -l > directory_list.txt
$ cat directory_list.txt
total 48
drwxr-xr-x 5 student staff 160 Jan 15 Desktop
...

# Redirect echo output
$ echo "Hello UNIX" > greeting.txt

# OVERWRITE: running again REPLACES the file contents
$ echo "First line" > file.txt
$ echo "Second line" > file.txt   # Replaces first line!
$ cat file.txt
Second line

# Redirect stderr to a file
$ find / -name "passwd" 2> errors.txt

# Redirect BOTH stdout and stderr to the same file
$ find / -name "passwd" > results.txt 2>&1

# Discard all output (send to /dev/null)
$ noisy_command > /dev/null 2>&1
```

Listing 51: > operator — redirects stdout, overwrites file

The >> Operator — Append

```
# Append to a file (does NOT overwrite)
$ echo "Monday log entry" > log.txt
$ echo "Tuesday log entry" >> log.txt
```

```
$ echo "Wednesday log entry" >> log.txt

$ cat log.txt
Monday log entry
Tuesday log entry
Wednesday log entry

# Append command output to a log file
$ date >> activity.log
$ ls /tmp >> activity.log
$ who >> activity.log

# Append stderr to error log
$ bad_command 2>> error.log

# Common pattern: logging in a shell script
$ echo "[$(date)] Backup started" >> /var/log/backup.log
$ tar -czf /backup/home.tar.gz /home >> /var/log/backup.log 2>&1
$ echo "[$(date)] Backup done" >> /var/log/backup.log
```

Listing 52: » operator — appends to existing file

Operator	Action	If File Exists
>	Redirect stdout	Overwrites completely
>>	Redirect stdout (append)	Adds to end of file
2>	Redirect stderr	Overwrites
2>>	Redirect stderr (append)	Adds to end
&>	Redirect stdout + stderr	Overwrites
2>&1	Merge stderr into stdout	Depends on stdout
<	Redirect stdin from file	N/A

Pipe (|) and tee Command

The Pipe Operator |

Definition — Pipe

A **pipe** (|) connects the **stdout** of one command directly to the **stdin** of the next. It lets you chain commands together to form powerful data-processing pipelines without creating temporary files.

```
# Basic syntax: cmd1 | cmd2 | cmd3 ...

# Count files in a directory
$ ls /etc | wc -l
131

# Find the 5 largest files in /var/log
$ du -ah /var/log | sort -rh | head -5
 98M /var/log/syslog
 23M /var/log/journal
  8M /var/log/auth.log
  2M /var/log/kern.log
512K /var/log/dpkg.log

```

```
# Search for a running process
$ ps aux | grep nginx
www-data  1234  0.0  0.1  8192 4096 ?  S  10:00  0:00 nginx: worker

# Count lines matching a pattern
$ cat /var/log/syslog | grep "error" | wc -l
42

# View long output page-by-page
$ ls -la /usr/bin | less

# Show unique users who logged in
$ last | awk '{print $1}' | sort | uniq

# Pipeline: find .py files, count lines, sort by size
$ find . -name "*.py" -exec wc -l {} \; | sort -n

# Remove duplicate lines from a file
$ sort names.txt | uniq

# Show disk usage, sorted, top 10
$ du -sh /* 2>/dev/null | sort -rh | head -10
```

Listing 53: Pipe — chaining commands

The tee Command

Definition — tee

tee reads from **stdin** and writes simultaneously to both **stdout** (terminal) AND one or more **files**. The name comes from a T-shaped pipe fitting that splits flow in two directions.

```
# Basic syntax: command | tee [options] file(s)

# Show output on screen AND save to file simultaneously
$ ls -l | tee directory.txt
total 48
drwxr-xr-x 5 student staff 160 Jan 15 Desktop
...
# Both printed to terminal AND saved to directory.txt

# Append mode (-a flag) --- don't overwrite the file
$ echo "New entry" | tee -a log.txt

# Write to MULTIPLE files at once
$ date | tee file1.txt file2.txt file3.txt

# Common pattern: build output + log
$ make 2>&1 | tee build.log
# Compilation output appears on screen AND is saved

# tee with sudo --- write to privileged files
$ echo "vm.swappiness=10" | sudo tee /etc/sysctl.conf

# Chaining tee in a pipeline (tap the pipe mid-stream)
$ cat /var/log/syslog | grep "error" | tee errors_found.txt | wc -l
# Shows count on screen, saves matching lines to file
```

Listing 54: tee — split output to screen AND file

Pipe vs Redirection vs `tee`

Tool	What it Does	Use When...
> / >>	Sends output to a file, terminal sees nothing	You only need the file, not screen output
|	Sends output of one command into another	Chaining processing steps
`tee`	Sends output to screen AND file simultaneously	You need both live output and a saved log

```
# Monitor system errors in real-time AND save to a dated log
$ tail -f /var/log/syslog \
    | grep --line-buffered "error\|warn\|fail" \
    | tee -a /tmp/alerts_$(date +%Y%m%d).log

# Breakdown:
# tail -f     -- follow the log in real time
# grep        -- filter only error/warning lines
# tee -a      -- show on screen AND append to dated file
```

Listing 55: Real-world combined pipeline example

Unit 2 Summary

Key Takeaways — Unit 2: UNIX File Commands

1. **mkdir / rmdir** create and remove directories; use `-p` for nested trees and `rm -r` for non-empty directories.
2. **cd** navigates the directory tree; `~`, `..`, and `-` are powerful shortcuts.
3. **cp** copies files/directories (`-r` for dirs); **mv** moves or renames, removing the original.
4. Files can be created via `touch`, `cat >`, `echo >`, editors, or heredocs.
5. UNIX has **7 file types**: regular, directory, symlink, character device, block device, named pipe, socket.
6. **chmod** controls read/write/execute permissions in symbolic (`u+x`) or octal (`755`) notation; **chown/chgrp** change ownership.
7. Every file has three timestamps: **atime** (last read), **mtime** (last content write), **ctime** (last metadata change). `touch -d` and `touch -t` set them manually.
8. **find** is the most powerful search tool, matching by name, type, size, time, owner, and permissions.
9. **Hard links** share an inode; **soft links** store a path. Hard links survive original deletion; soft links become dangling.
10. An **i-node** stores all file metadata except the name; **df** measures filesystem-level disk space; **du** measures directory/file-level usage.
11. `>` overwrites, `>>` appends; both redirect stdout to files.
12. **Pipes** (`|`) chain commands; **tee** splits output to both the terminal and a file simultaneously.

End of Unit 2 — UNIX File Commands
Proceed to Unit 3: File Processing Commands

Unit 2 — Practice Questions

Practice Questions — Unit 2: UNIX File Commands

A. Short Answer (2 marks each)

1. What is the difference between `rmdir` and `rm -r`? When would you use each?
2. Explain `atime`, `mtime`, and `ctime`. Which one is updated when you run `chmod`?
3. What is an i-node? List **four** pieces of information stored in an i-node.
4. Distinguish between a hard link and a symbolic link with one example each.
5. What does the command `chmod 755 script.sh` do? Explain using the octal table.
6. Explain the `tee` command. How does it differ from simple `>` redirection?
7. What is the significance of `$?` in shell? Give a practical use case.
8. What does `find /tmp -mtime +7 -delete` do? Explain each flag.

B. True or False (1 mark each) — Justify your answer

1. A hard link can span two different filesystems.
2. Deleting a file with `rm` sends it to a Recycle Bin.
3. `ctime` records the file creation time.
4. The `ls -la` command shows hidden files.
5. `chmod 777` is a recommended permission for shared scripts.

C. Long Answer (5 marks each)

1. Explain all **seven file types** in UNIX with their `ls -l` symbol, how they are created, and one real example of each.
2. Describe the i-node structure in detail. How does the OS resolve the path `/home/student/notes.txt` to actual disk data?
3. Explain I/O redirection in UNIX. What are stdin, stdout, and stderr? Give examples of `>`, `>>`, `2>`, and `2>&1`.

D. Command Practice

1. Write commands to: (a) create a nested directory `a/b/c` in one step, (b) copy a directory recursively preserving permissions, (c) find all `.log` files modified in the last 24 hours.
2. Write a pipeline that lists all files in `/etc`, filters those containing "conf", counts them, and saves the count to `result.txt` while also displaying it on screen.

Unit 3: File Processing Commands

Unit 3 — File Processing Commands

cut · awk · paste · split · wc · sort · diff · cmp · comm · join · uniq · tr

UNIX provides a powerful suite of text-processing commands. Each command does one job well and can be combined with others through pipes to build sophisticated data-processing pipelines — this is the UNIX philosophy at work.

Row-Wise and Column-Wise Selection — cut

Definition — cut

cut extracts specific columns or fields from each line of a file or stdin. It operates on three modes: by *byte position*, by *character position*, or by *field delimiter*.

Syntax and Options

```
cut [OPTIONS] [FILE]

-b LIST   Cut by BYTE positions
-c LIST   Cut by CHARACTER positions
-f LIST   Cut by FIELD number (requires -d delimiter)
-d DELIM  Set field delimiter (default: TAB)
--complement   Select everything EXCEPT the specified fields
```

Listing 56: cut — syntax and key options

Sample Data Files Used in This Unit

Throughout Unit 3 we will use these sample files:

```
# students.txt  (colon-delimited)
Alice:20:BCA:Hooghly
Bob:22:BCA:Kolkata
Charlie:21:MCA:Hooghly
Diana:23:MCA:Kolkata
Eve:20:BCA:Hooghly

# scores.txt  (space-delimited)
Alice   85  92  78
Bob     70  65  88
Charlie 91  87  95
Diana   60  72  68
Eve     88  90  82
```

Listing 57: students.txt and scores.txt — sample data

Column-Wise Selection — Selecting by Character Position

```
# Create the sample file
```

```
$ cat > students.txt << 'EOF'
Alice:20:BCA:Hooghly
Bob:22:BCA:Kolkata
Charlie:21:MCA:Hooghly
Diana:23:MCA:Kolkata
Eve:20:BCA:Hooghly
EOF

# Extract characters 1 to 5 from each line (first 5 chars = name start)
$ cut -c1-5 students.txt
Alice
Bob:2
Charl
Diana
Eve:2

# Extract a single character (column 1 only)
$ cut -c1 students.txt
A
B
C
D
E

# Extract characters 1-3 AND 7-8 (comma separates ranges)
$ cut -c1-3,7-8 students.txt
Ali0
Bob
Cha1
Dia3
Eve0

# Extract from character 7 to end of line
$ cut -c7- students.txt
0:BCA:Hooghly
:BCA:Kolkata
ie:21:MCA:Hooghly
:23:MCA:Kolkata
0:BCA:Hooghly
```

Listing 58: cut -c — select by character position

Row-Wise and Column-Wise Selection from Files with -f and -d

```
# Extract field 1 (Name) using colon delimiter
$ cut -d: -f1 students.txt
Alice
Bob
Charlie
Diana
Eve

# Extract field 3 (Course) --- column-wise selection
$ cut -d: -f3 students.txt
BCA
BCA
MCA
MCA
BCA

# Extract fields 1 AND 3 (Name + Course) --- multiple columns
$ cut -d: -f1,3 students.txt
Alice:BCA
```

```
Bob:BCA
Charlie:MCA
Diana:MCA
Eve:BCA

# Extract a RANGE of fields (fields 2 through 4)
$ cut -d: -f2-4 students.txt
20:BCA:Hooghly
22:BCA:Kolkata
21:MCA:Hooghly
23:MCA:Kolkata
20:BCA:Hooghly

# Select everything EXCEPT field 2 (--complement)
$ cut -d: -f2 --complement students.txt
Alice:BCA:Hooghly
Bob:BCA:Kolkata
Charlie:MCA:Hooghly
Diana:MCA:Kolkata
Eve:BCA:Hooghly

# Tab-delimited file (default delimiter for -f is TAB)
$ cut -f2 scores.txt     # Extract column 2 (Math score)
85
70
91
60
88

# Cut from /etc/passwd --- extract usernames (field 1) and shells (field 7)
$ cut -d: -f1,7 /etc/passwd | head -5
root:/bin/bash
daemon:/usr/sbin/nologin
bin:/usr/sbin/nologin
student:/bin/bash
www-data:/usr/sbin/nologin
```

Listing 59: cut -f -d — field-based (column) extraction

Option	Example	Selects
`-c 3`	`cut -c3`	Single char at position 3
`-c 1-5`	`cut -c1-5`	Characters 1 through 5
`-c 1,3,5`	`cut -c1,3,5`	Chars at positions 1, 3, 5
`-c -5`	`cut -c-5`	Characters 1 through 5
`-c 4-`	`cut -c4-`	Character 4 to end of line
`-f 2`	`cut -d: -f2`	Field 2, colon-delimited
`-f 1,3`	`cut -d: -f1,3`	Fields 1 and 3
`-f 2-4`	`cut -d: -f2-4`	Fields 2 through 4

When cut is Not Enough — Use awk

`cut` cannot handle variable-length whitespace as a delimiter. For files where columns are separated by multiple spaces or tabs, use `awk` (§) which handles this automatically.

Row-Wise and Column-Wise Selection with awk

Definition — awk

`awk` is a full programming language designed for **pattern scanning and processing**. It reads a file line by line, splits each line into fields, and applies user-defined actions. It can do everything `cut` does — and much more.

In `awk`, each field is accessed as `$1`, `$2`, … `$NF` (last field). The entire line is `$0`.

```
# Basic syntax
awk 'pattern { action }' file

# --- COLUMN-WISE SELECTION ---

# Print column 1 (Name) from space-delimited scores.txt
$ awk '{print $1}' scores.txt
Alice
Bob
Charlie
Diana
Eve

# Print columns 1 and 3
$ awk '{print $1, $3}' scores.txt
Alice 92
Bob 65
Charlie 87
Diana 72
Eve 90

# Use colon as delimiter (-F flag) --- like cut -d
$ awk -F: '{print $1, $3}' students.txt
Alice BCA
Bob BCA
Charlie MCA
Diana MCA
Eve BCA

# Print with custom output separator
$ awk -F: '{print $1 " studies " $3 " at " $4}' students.txt
Alice studies BCA at Hooghly
Bob studies BCA at Kolkata
Charlie studies MCA at Hooghly

# --- ROW-WISE SELECTION (filtering rows) ---

# Print only rows where field 2 (age) is greater than 21
$ awk -F: '$2 > 21 {print $0}' students.txt
Bob:22:BCA:Kolkata
Diana:23:MCA:Kolkata

# Print only rows where course (field 3) is MCA
$ awk -F: '$3 == "MCA" {print $0}' students.txt
Charlie:21:MCA:Hooghly
Diana:23:MCA:Kolkata

# Print rows where the name starts with 'A' (regex match)
$ awk -F: '$1 ~ /^A/ {print $0}' students.txt
Alice:20:BCA:Hooghly

# --- PRINT SPECIFIC LINE NUMBERS (row selection) ---
```

```
# Print line 3 only
$ awk 'NR==3' students.txt
Charlie:21:MCA:Hooghly

# Print lines 2 through 4
$ awk 'NR>=2 && NR<=4' students.txt
Bob:22:BCA:Kolkata
Charlie:21:MCA:Hooghly
Diana:23:MCA:Kolkata

# Print total number of records (rows)
$ awk 'END {print "Total rows:", NR}' students.txt
Total rows: 5

# --- USEFUL awk BUILT-IN VARIABLES ---
# NR = current record number (line number)
# NF = number of fields in current record
# FS = field separator (input)
# OFS = output field separator
# $0 = entire line; $1..$NF = individual fields

# Add line numbers to output
$ awk '{print NR".", $0}' students.txt
1. Alice:20:BCA:Hooghly
2. Bob:22:BCA:Kolkata
3. Charlie:21:MCA:Hooghly

# Calculate average of column 2 (Math scores)
$ awk '{sum += $2} END {print "Math Average:", sum/NR}' scores.txt
Math Average: 78.8
```

Listing 60: awk — row and column selection

Merge Lines of Files Horizontally — paste

Definition — paste

paste merges lines from multiple files side-by-side (horizontally), joining corresponding lines with a delimiter (TAB by default). This is the opposite of `cut` — it adds columns rather than removing them.

```
# Create sample files
$ printf "Alice\nBob\nCharlie\n" > names.txt
$ printf "20\n22\n21\n" > ages.txt
$ printf "BCA\nBCA\nMCA\n" > courses.txt

# Basic paste --- merge two files side by side (TAB-separated)
$ paste names.txt ages.txt
Alice    20
Bob      22
Charlie  21

# Merge THREE files
$ paste names.txt ages.txt courses.txt
Alice    20   BCA
Bob      22   BCA
Charlie  21   MCA

# Use a custom delimiter with -d
$ paste -d: names.txt ages.txt
Alice:20
```

```
Bob:22
Charlie:21

# Use comma as delimiter
$ paste -d, names.txt ages.txt courses.txt
Alice,20,BCA
Bob,22,BCA
Charlie,21,MCA

# Use MULTIPLE delimiters (cycled through fields)
# -d',;' alternates: comma between f1-f2, semicolon between f2-f3
$ paste -d',;' names.txt ages.txt courses.txt
Alice,20;BCA
Bob,22;BCA
Charlie,21;MCA

# Serial mode (-s): paste all lines of ONE file into one row
$ paste -s names.txt
Alice	Bob	Charlie

# Serial with delimiter
$ paste -s -d, names.txt
Alice,Bob,Charlie

# Practical: combine two CSV columns
$ paste -d, names.txt courses.txt > name_course.csv
$ cat name_course.csv
Alice,BCA
Bob,BCA
Charlie,MCA

# Merge from stdin using - (dash)
$ echo -e "X\nY\nZ" | paste names.txt -
Alice	X
Bob	Y
Charlie	Z
```

Listing 61: paste — merging files and lines side by side

Option	Example	Effect
(default)	`paste f1 f2`	TAB-separated side-by-side merge
`-d,`	`paste -d, f1 f2`	Comma as delimiter
`-d':;'`	`paste -d':;' f1 f2 f3`	Cycle through multiple delimiters
`-s`	`paste -s f1`	All lines of a file into one row
`-s -d,`	`paste -s -d, f1`	Single row, comma-separated

Split Large Files into Smaller Files — split

Definition — split

`split` divides a large file into smaller pieces. It can split by *number of lines*, *file size*, or *number of chunks*. The output files are named with a prefix and a sequential suffix.

```
# Basic syntax
split [OPTIONS] [INPUT [PREFIX]]

```

```
# --- CREATE A LARGE TEST FILE ---
$ seq 1 100 > numbers.txt     # File with numbers 1 to 100 (100 lines)

# --- SPLIT BY LINE COUNT (default: 1000 lines per file) ---
# Split into files of 25 lines each
$ split -l 25 numbers.txt part_
# Creates: part_aa  part_ab  part_ac  part_ad

$ ls part_*
part_aa  part_ab  part_ac  part_ad

$ wc -l part_*
 25 part_aa
 25 part_ab
 25 part_ac
 25 part_ad
100 total

# --- SPLIT BY FILE SIZE ---
# Split a large file into 1MB chunks
$ split -b 1M largefile.log chunk_

# Split into 500KB pieces
$ split -b 500k bigdata.txt piece_

# Split into 100-byte pieces (for demo)
$ split -b 100 numbers.txt byte_part_

# --- SPLIT INTO N EQUAL CHUNKS ---
# Split into exactly 4 equal pieces
$ split -n 4 numbers.txt quarter_

# --- USE NUMERIC SUFFIXES (instead of aa, ab ...) ---
$ split -l 25 -d numbers.txt num_part_
$ ls num_part_*
num_part_00  num_part_01  num_part_02  num_part_03

# --- CONTROL SUFFIX LENGTH ---
# Use 3-character suffix (num_part_000 etc.)
$ split -l 25 -d -a 3 numbers.txt section_
$ ls section_*
section_000  section_001  section_002  section_003

# --- VERBOSE: print output filenames ---
$ split -l 25 --verbose numbers.txt v_
creating file 'v_aa'
creating file 'v_ab'
creating file 'v_ac'
creating file 'v_ad'

# --- REASSEMBLE split files ---
$ cat part_* > reassembled.txt
$ diff numbers.txt reassembled.txt   # No output = identical
```

Listing 62: split — splitting large files

Option	Example	Splits by
`-l N`	`split -l 100`	N lines per output file
`-b SIZE`	`split -b 1M`	Size per file (k=KB, M=MB, G=GB)
`-n N`	`split -n 4`	Into exactly N equal files
`-d`	`split -d`	Numeric suffixes (00, 01 …)
`-a N`	`split -a 3`	Suffix length (default: 2)

Counting Lines, Words, and Characters — WC

Definition — wc

`wc` (*word count*) counts **lines**, **words**, **characters**, and **bytes** in a file or stdin. It is one of the most-used commands in shell pipelines.

```
# Basic syntax
wc [OPTIONS] [FILE(s)]

# Default output: lines  words  characters  filename
$ wc students.txt
 5  5 100 students.txt
# ^ 5 lines, 5 words (one per line), 100 characters

# Count LINES only (-l)
$ wc -l students.txt
5 students.txt

# Count WORDS only (-w)
$ wc -w students.txt
5 students.txt

# Count CHARACTERS only (-c)
$ wc -c students.txt
100 students.txt

# Count BYTES (-c is bytes, -m is characters/multibyte-safe)
$ wc -m students.txt

# Count LONGEST LINE length (-L)
$ wc -L students.txt
20 students.txt   # longest line has 20 characters

# Count multiple files at once (total line added)
$ wc -l names.txt ages.txt courses.txt
 5 names.txt
 5 ages.txt
 5 courses.txt
15 total

# Count ALL at once (lines, words, chars, bytes)
$ wc students.txt
  5   5 100 students.txt

# Common pipeline uses
# Count number of files in /etc
$ ls /etc | wc -l
131
```

```

# Count number of running processes
$ ps aux | wc -l
87

# Count how many lines contain "error" in a log
$ grep -i "error" /var/log/syslog | wc -l
42

# Count total words in all .txt files
$ cat *.txt | wc -w
```

Listing 63: wc — counting lines, words, and characters

Sorting File Content — sort

Definition — sort

`sort` sorts lines of text files. It can sort *alphabetically*, *numerically*, *by specific column*, in *reverse*, and much more.

Basic and Alphabetical Sorting

```
# Basic syntax
sort [OPTIONS] [FILE]

# Default: alphabetical (lexicographic) sort
$ sort names.txt
Alice
Bob
Charlie
Diana
Eve

# Reverse sort (-r)
$ sort -r names.txt
Eve
Diana
Charlie
Bob
Alice

# Remove duplicate lines while sorting (-u = unique)
$ printf "banana\napple\nbanana\ncherry\napple\n" | sort -u
apple
banana
cherry

# Case-insensitive sort (-f)
$ printf "Zebra\napple\nMango\nbanana\n" | sort -f
apple
banana
Mango
Zebra
```

Listing 64: sort — alphabetical and basic options

Sorting by Specific Column (-k flag)

```
# Sort students.txt by field 2 (age), colon-delimited
$ sort -t: -k2 students.txt
Alice:20:BCA:Hooghly
```

```
Eve:20:BCA:Hooghly
Charlie:21:MCA:Hooghly
Bob:22:BCA:Kolkata
Diana:23:MCA:Kolkata

# Sort by field 3 (course), then by field 1 (name) as tiebreaker
$ sort -t: -k3,3 -k1,1 students.txt
Alice:20:BCA:Hooghly
Bob:22:BCA:Kolkata
Eve:20:BCA:Hooghly
Charlie:21:MCA:Hooghly
Diana:23:MCA:Kolkata

# Sort scores.txt by column 2 (Math score), space-delimited
$ sort -k2 scores.txt
Diana   60  72  68
Bob     70  65  88
Alice   85  92  78
Eve     88  90  82
Charlie 91  87  95

# Sort by column 2, REVERSED (highest score first)
$ sort -k2 -r scores.txt
Charlie 91  87  95
Eve     88  90  82
Alice   85  92  78
Bob     70  65  88
Diana   60  72  68
```

Listing 65: sort -k — sort by a specific field/column

Numerical Sorting (-n flag)

Why -n Matters

Without **-n**, **sort** compares numbers as *strings*: "10" comes before "9" because "1" < "9" lexicographically. The **-n** flag tells sort to compare as actual *numbers*.

```
$ printf "10\n2\n30\n5\n100\n1\n" > nums.txt

# WITHOUT -n (wrong! string sort)
$ sort nums.txt
1
10
100
2
30
5

# WITH -n (correct numeric sort)
$ sort -n nums.txt
1
2
5
10
30
100

# Numeric sort DESCENDING (largest first)
$ sort -n -r nums.txt
100
30
```

```
10
5
2
1

# Sort by column 2 NUMERICALLY (age column)
$ sort -t: -k2 -n students.txt
Alice:20:BCA:Hooghly
Eve:20:BCA:Hooghly
Charlie:21:MCA:Hooghly
Bob:22:BCA:Kolkata
Diana:23:MCA:Kolkata

# Human-readable size sort (-h flag)
$ du -sh /var/log/* 2>/dev/null | sort -h
4.0K  /var/log/alternatives.log
 12K  /var/log/dpkg.log
512K  /var/log/kern.log
 23M  /var/log/journal
 98M  /var/log/syslog

# Sort by Math score (col 2) numerically, save to file
$ sort -k2 -n scores.txt -o scores_ranked.txt
$ cat scores_ranked.txt
Diana   60  72  68
Bob     70  65  88
Alice   85  92  78
Eve     88  90  82
Charlie 91  87  95
```

Listing 66: sort -n — numerical sorting

Option	Example	Effect
(default)	`sort file`	Alphabetical (lexicographic)
`-n`	`sort -n`	Numeric sort
`-r`	`sort -r`	Reverse order
`-u`	`sort -u`	Sort and remove duplicates
`-f`	`sort -f`	Case-insensitive
`-k N`	`sort -k2`	Sort by field N
`-t D`	`sort -t:`	Use D as field delimiter
`-h`	`sort -h`	Human-readable sizes (1K, 2M)
`-o`	`sort -o out.txt`	Write output to file

Comparing Two Files Line by Line — diff

Definition — diff

`diff` compares two files **line by line** and reports what changes would need to be made to transform the first file into the second. It is the foundation of version control systems like `git`.

```
# Create two slightly different files
$ cat > file_v1.txt << 'EOF'
```

```
Line 1: Introduction
Line 2: UNIX basics
Line 3: File commands
Line 4: End of document
EOF

$ cat > file_v2.txt << 'EOF'
Line 1: Introduction
Line 2: UNIX and Linux basics
Line 3: File commands
Line 5: Summary added
Line 4: End of document
EOF

# Basic diff output
$ diff file_v1.txt file_v2.txt
2c2
< Line 2: UNIX basics
---
> Line 2: UNIX and Linux basics
3a4
> Line 5: Summary added

# Understanding diff output:
# 2c2   = line 2 in file1 CHANGED to line 2 in file2
# 3a4   = after line 3 in file1, ADD line 4 from file2
# <     = line from the FIRST file
# >     = line from the SECOND file
# ---   = separator between old and new versions
```

Listing 67: diff — comparing files line by line

Diff Change Codes

Code	Meaning
NcN	**Change** — line N in file1 changed to line N in file2
NaN	**Add** — after line N in file1, add line N from file2
NdN	**Delete** — line N in file1 was deleted
<	Line from the first (old) file
>	Line from the second (new) file
---	Separator between old and new

```
# Unified format (-u) --- like git diff, much easier to read
$ diff -u file_v1.txt file_v2.txt
--- file_v1.txt  2024-01-15 10:00
+++ file_v2.txt  2024-01-15 11:00
@@ -1,4 +1,5 @@
 Line 1: Introduction
-Line 2: UNIX basics
+Line 2: UNIX and Linux basics
 Line 3: File commands
+Line 5: Summary added
 Line 4: End of document

# Context format (-c) --- shows surrounding lines
$ diff -c file_v1.txt file_v2.txt
```

```

# Side-by-side format (-y) --- visual comparison
$ diff -y file_v1.txt file_v2.txt
Line 1: Introduction            Line 1: Introduction
Line 2: UNIX basics           | Line 2: UNIX and Linux basics
Line 3: File commands           Line 3: File commands
                              > Line 5: Summary added
Line 4: End of document         Line 4: End of document

# Ignore whitespace differences (-w)
$ diff -w file_v1.txt file_v2.txt

# Ignore blank lines (-B)
$ diff -B file_v1.txt file_v2.txt

# Ignore case differences (-i)
$ diff -i file_v1.txt file_v2.txt

# Recursive directory comparison (-r)
$ diff -r dir1/ dir2/

# Quiet mode --- just report IF files differ (-q)
$ diff -q file_v1.txt file_v2.txt
Files file_v1.txt and file_v2.txt differ
# (no output if files are identical --- useful in scripts)
```

Listing 68: diff with useful options

Compare Two Files Character by Character — cmp

Definition — cmp

cmp compares two files **byte by byte** (character by character). It reports the *first* position where the files differ. Unlike diff, it works on both text and binary files.

```
# Basic syntax
cmp [OPTIONS] file1 file2

# If files are IDENTICAL: no output, exit code 0
$ cmp names.txt names.txt
$ echo $?
0

# If files DIFFER: reports first difference location
$ cmp file_v1.txt file_v2.txt
file_v1.txt file_v2.txt differ: char 31, line 2
# Files differ at the 31st character, which is on line 2

# Verbose mode (-l): list ALL differing bytes
$ cmp -l file_v1.txt file_v2.txt
 31  x  U   # byte 31: 'x'(120) in file1, 'U'(85) in file2
 32  n  N
 33  i  I
...

# Silent mode (-s): no output, only exit code (for scripts)
$ cmp -s file1.txt file2.txt
$ if cmp -s original.txt backup.txt; then
>   echo "Files are identical"
> else
>   echo "Files are different"
```

```
> fi
Files are different

# Compare binary files (cmp works on any file type)
$ cmp /bin/ls /bin/cp
/bin/ls /bin/cp differ: char 25, line 1
```

Listing 69: cmp — byte-level file comparison

Tool	Granularity	Best For
diff	Line by line	Text files, finding what changed
cmp	Byte by byte	Any file; quick check if identical

Compare Two Sorted Files — comm

Definition — comm

comm compares **two sorted files** line by line and produces output in three columns: lines only in file1, lines only in file2, and lines common to both. Both input files **must be sorted** first.

```
# Create two sorted sample files
$ printf "apple\nbanana\ncherry\ndate\nfig\n" > fruits1.txt
$ printf "banana\ncherry\nelderberry\nfig\ngrape\n" > fruits2.txt

# Default output: 3 columns
$ comm fruits1.txt fruits2.txt
apple                   # <- column 1: only in fruits1.txt
        banana          # <- column 3: in BOTH files
        cherry
date                    # <- column 1: only in fruits1.txt
    elderberry          # <- column 2: only in fruits2.txt
        fig
    grape               # <- column 2: only in fruits2.txt

# Understanding the 3 columns:
# Column 1 (no indent)    = lines ONLY in file1
# Column 2 (one tab)      = lines ONLY in file2
# Column 3 (two tabs)     = lines in BOTH files

# Suppress column 1 (-1): show only file2 lines and common
$ comm -1 fruits1.txt fruits2.txt
        banana
        cherry
elderberry
        fig
grape

# Suppress columns 1 and 2 (-12): show ONLY COMMON lines
$ comm -12 fruits1.txt fruits2.txt
banana
cherry
fig

# Suppress columns 1 and 3 (-13): show ONLY in file2
$ comm -13 fruits1.txt fruits2.txt
elderberry
grape
```

```

# Suppress columns 2 and 3 (-23): show ONLY in file1
$ comm -23 fruits1.txt fruits2.txt
apple
date

# IMPORTANT: files must be sorted first!
$ sort file1.txt > sorted1.txt
$ sort file2.txt > sorted2.txt
$ comm sorted1.txt sorted2.txt

# One-liner: sort on the fly
$ comm <(sort fruits1.txt) <(sort unsorted_fruits2.txt)
```

Listing 70: comm — comparing sorted files

Option	Output
(none)	All 3 columns
-1	Suppress column 1 (remove lines only in file1)
-2	Suppress column 2 (remove lines only in file2)
-3	Suppress column 3 (remove lines common to both)
-12	Show ONLY lines common to both files
-13	Show ONLY lines unique to file2
-23	Show ONLY lines unique to file1

Join Two Files — join

Definition — join

`join` performs a **relational join** on two sorted files, like an SQL `JOIN`. It merges lines from two files that share a common *join field* (key column). Both files must be sorted on the join field.

```
# Create two related files (sorted by the key column)
$ cat > student_ids.txt << 'EOF'
101 Alice
102 Bob
103 Charlie
104 Diana
EOF

$ cat > student_grades.txt << 'EOF'
101 A
102 B+
103 A+
104 B
EOF

# Basic join on field 1 (default key field)
$ join student_ids.txt student_grades.txt
101 Alice A
102 Bob B+
103 Charlie A+
104 Diana B
```

```

# Specify join field with -1 and -2
# -1 N = use field N from file1 as key
# -2 N = use field N from file2 as key
$ join -1 1 -2 1 student_ids.txt student_grades.txt
101 Alice A
102 Bob B+
103 Charlie A+
104 Diana B

# Custom delimiter (-t flag)
$ cat > ids_colon.txt << 'EOF'
101:Alice:BCA
102:Bob:BCA
103:Charlie:MCA
EOF

$ cat > grades_colon.txt << 'EOF'
101:A:85
102:B+:70
103:A+:91
EOF

$ join -t: ids_colon.txt grades_colon.txt
101:Alice:BCA:A:85
102:Bob:BCA:B+:70
103:Charlie:MCA:A+:91

# Include unmatched lines from file1 (-a 1)
$ cat > extra_ids.txt << 'EOF'
101 Alice
102 Bob
103 Charlie
105 Eve
EOF
$ join -a 1 extra_ids.txt student_grades.txt
101 Alice A
102 Bob B+
103 Charlie A+
105 Eve        # Eve has no grade, but still shown

# Include unmatched from both files (-a 1 -a 2)
$ join -a 1 -a 2 extra_ids.txt student_grades.txt

# Select output fields with -o
# -o 1.2,2.2 means: field 2 from file1, field 2 from file2
$ join -o 1.2,2.2 student_ids.txt student_grades.txt
Alice A
Bob B+
Charlie A+
Diana B
```

Listing 71: join — merging files on a common field

join vs paste

paste merges files by line position (1st line with 1st line, regardless of content). **join** merges only lines with a *matching key field* — far more powerful for relational data.

The uniq Command

Definition — uniq

`uniq` **filters out adjacent duplicate lines** from a sorted input. It can report, count, or suppress repeated lines. Because it only compares *adjacent* lines, input is usually piped through `sort` first.

```
# Create a file with duplicates
$ cat > fruits.txt << 'EOF'
apple
apple
banana
cherry
cherry
cherry
date
banana
EOF

# Basic uniq --- remove ADJACENT duplicates
$ uniq fruits.txt
apple
banana
cherry
date
banana      # NOT removed! banana reappeared after a gap

# Correct approach: sort first, then uniq
$ sort fruits.txt | uniq
apple
banana
cherry
date

# Count occurrences of each line (-c flag)
$ sort fruits.txt | uniq -c
      2 apple
      2 banana
      3 cherry
      1 date

# Sort by count (most frequent first)
$ sort fruits.txt | uniq -c | sort -rn
      3 cherry
      2 apple
      2 banana
      1 date

# Show ONLY duplicate lines (-d flag)
$ sort fruits.txt | uniq -d
apple
banana
cherry

# Show ONLY unique (non-duplicated) lines (-u flag)
$ sort fruits.txt | uniq -u
date

# Case-insensitive comparison (-i flag)
$ printf "Apple\napple\nBANANA\nbanana\n" | sort | uniq -i
Apple
BANANA
```

```

# Compare only first N characters (-w N)
$ sort fruits.txt | uniq -w 3
apple
ban       # Only compares first 3 chars (ban=banana)
che       # cherry
dat       # date

# Skip first N fields before comparing (-f N)
$ uniq -f 1 scores.txt   # Skip field 1 (name) when comparing

# Real-world use: find most common commands in bash history
$ history | awk '{print $2}' | sort | uniq -c | sort -rn | head -10
     42 ls
     28 cd
     15 git
     12 vim
      8 cat
```

Listing 72: uniq — removing and counting duplicate lines

Option	Example	Effect
(default)	`uniq`	Remove adjacent duplicates
`-c`	`uniq -c`	Prefix lines with occurrence count
`-d`	`uniq -d`	Print only lines that are duplicated
`-u`	`uniq -u`	Print only lines that are NOT duplicated
`-i`	`uniq -i`	Case-insensitive comparison
`-w N`	`uniq -w3`	Compare only first N characters
`-f N`	`uniq -f1`	Skip first N fields before comparing

The Transformation Command — tr

Definition — tr

`tr` (*translate*) **transforms, deletes, or squeezes characters** from stdin. It works character by character and is commonly used for case conversion, stripping characters, and replacing characters. `tr` reads only from **stdin** (not files directly).

Basic Translation

```
# Basic syntax (always reads from stdin)
tr [OPTIONS] SET1 [SET2]

# Translate every character in SET1 to corresponding char in SET2

# Convert lowercase to UPPERCASE
$ echo "hello unix world" | tr 'a-z' 'A-Z'
HELLO UNIX WORLD

# Convert UPPERCASE to lowercase
$ echo "HELLO UNIX WORLD" | tr 'A-Z' 'a-z'
hello unix world

# Replace spaces with underscores
$ echo "hello unix world" | tr ' ' '_'
```

```
hello_unix_world

# Replace colons with tabs (useful for CSV conversion)
$ echo "Alice:20:BCA" | tr ':' '\t'
Alice	20	BCA

# Replace newlines with spaces (put file on one line)
$ cat names.txt | tr '\n' ' '
Alice Bob Charlie Diana Eve

# Translate digits to X (mask numbers)
$ echo "Phone: 9876543210" | tr '0-9' 'X'
Phone: XXXXXXXXXX
```

Listing 73: tr — basic character translation

Deleting Characters with -d

```
# Delete ALL occurrences of characters in SET1

# Remove all vowels
$ echo "hello unix world" | tr -d 'aeiou'
hll nx wrld

# Remove all digits
$ echo "Phone: 9876543210, Code: A1B2C3" | tr -d '0-9'
Phone: , Code: ABC

# Remove all spaces
$ echo "hello unix world" | tr -d ' '
hellounixworld

# Remove newlines (join all lines into one)
$ cat names.txt | tr -d '\n'
AliceBobCharlieDianaEve

# Remove carriage returns (fix Windows files with \r\n)
$ tr -d '\r' < windows_file.txt > unix_file.txt

# Remove non-printable characters
$ tr -d '[:cntrl:]' < messy.txt > clean.txt

# Remove all punctuation
$ echo "Hello, World! How are you?" | tr -d '[:punct:]'
Hello World How are you
```

Listing 74: tr -d — deleting characters

Squeezing Repeated Characters with -s

```
# Squeeze repeated characters into one

# Replace multiple spaces with single space
$ echo "Alice    20    BCA" | tr -s ' '
Alice 20 BCA

# Squeeze multiple newlines into one (remove blank lines)
$ cat -A file_with_blanks.txt | tr -s '\n'

# Squeeze repeated vowels
$ echo "gooooood moooorning" | tr -s 'aeiou'
god morning

```

```
# Practical: normalize whitespace in a file
$ cat messy_data.txt | tr -s '[:space:]' ' '
```

Listing 75: tr -s — squeeze repeated characters

tr Character Classes

```
# POSIX classes work inside [:class:] notation

# Convert all lowercase to uppercase using class
$ echo "hello unix" | tr '[:lower:]' '[:upper:]'
HELLO UNIX

# Delete all digits
$ echo "abc123def456" | tr -d '[:digit:]'
abcdef

# Delete all alphabetic characters
$ echo "abc123def456" | tr -d '[:alpha:]'
123456

# Keep only alphanumeric characters (delete everything else)
$ echo "Hello, World! 2024." | tr -dc '[:alnum:][:space:]'
Hello World 2024

# Delete all non-printable (control) characters
$ tr -d '[:cntrl:]' < input.txt

# Available POSIX classes:
# [:alpha:]  = a-z and A-Z
# [:lower:]  = a-z
# [:upper:]  = A-Z
# [:digit:]  = 0-9
# [:alnum:]  = a-z, A-Z, 0-9
# [:space:]  = spaces, tabs, newlines
# [:punct:]  = punctuation characters
# [:cntrl:]  = control characters (non-printable)
# [:print:]  = printable characters
```

Listing 76: tr — using POSIX character classes

Option	Syntax	Effect
(default)	`tr SET1 SET2`	Translate chars in SET1 to SET2
`-d`	`tr -d SET1`	Delete all chars in SET1
`-s`	`tr -s SET1`	Squeeze repeated chars in SET1 to one
`-c`	`tr -c SET1 SET2`	Complement: translate chars NOT in SET1
`-dc`	`tr -dc SET1`	Delete chars NOT in SET1 (keep only SET1)

```
# Extract only numbers from a string
$ echo "Order #1024 Total: Rs.4500 Date: 15/01/24" \
  | tr -dc '0-9 \n'
1024 4500 150124

# Word frequency counter (classic pipeline)
$ cat students.txt \
  | tr '[:upper:]' '[:lower:]' \      # normalize case
  | tr -s '[:space:]' '\n' \           # one word per line
  | tr -d '[:punct:]' \                # remove punctuation
```

```
    | sort \                          # sort alphabetically
    | uniq -c \                       # count occurrences
    | sort -rn \                      # sort by frequency
    | head -10                        # show top 10
```

Listing 77: Real-world tr pipelines

Unit 3 Summary

Key Takeaways — Unit 3: File Processing Commands

1. **cut** extracts columns by character position (`-c`) or field number (`-f` with `-d` delimiter); use `--complement` to invert selection.
2. **awk** is a full programming language for row/column selection; `$1..$NF` access fields, `NR` is the line number, `-F` sets the delimiter, and patterns filter rows.
3. **paste** merges files *horizontally* (side by side); `-d` sets delimiter; `-s` folds a file into a single row.
4. **split** breaks large files into parts by `-l` (lines), `-b` (bytes), or `-n` (N equal chunks); reassemble with `cat`.
5. **wc** counts lines (`-l`), words (`-w`), characters (`-c`), and longest line length (`-L`).
6. **sort** sorts alphabetically by default; use `-n` for numbers, `-k` for a specific column, `-t` for delimiter, `-r` to reverse, and `-u` to deduplicate.
7. **diff** compares two files line by line; `-u` gives unified (git-style) output; `-y` shows side by side; `-q` reports only whether files differ.
8. **cmp** compares byte by byte; reports the first differing position; works on binary files; `-s` is silent (for scripts).
9. **comm** compares two *sorted* files in three columns (file1 only, file2 only, both); `-12` prints only common lines.
10. **join** performs a relational SQL-style join on two sorted files using a common key field; `-t` sets delimiter; `-a` includes unmatched lines.
11. **uniq** removes adjacent duplicates; always `sort` first; `-c` counts, `-d` shows only duplicates, `-u` shows only unique lines.
12. **tr** translates, deletes (`-d`), or squeezes (`-s`) characters; supports ranges (`a-z`) and POSIX classes (`[:alpha:]`).

End of Unit 3 — File Processing Commands
Proceed to Unit 4: VI Editor

Unit 3 — Practice Questions

Practice Questions — Unit 3: File Processing Commands

A. Short Answer (2 marks each)

1. What is the difference between `diff` and `cmp`? When would you prefer each?
2. Explain the `-k` and `-t` flags of the `sort` command with an example.
3. What does `sort | uniq -c | sort -rn` accomplish as a pipeline?
4. What is the difference between `cut -c` and `cut -f`? Give one example of each.
5. Explain `comm -12 file1 file2`. What prerequisite must both files satisfy?
6. What does `tr -d '\r'` do and when is it useful?
7. How does `join` differ from `paste`? Give a scenario for each.
8. What does `split -b 1M largefile.dat chunk_` produce?

B. Predict the Output (2 marks each)

Given `data.txt`: `Alice:20:BCA`, `Bob:22:MCA`, `Charlie:21:BCA`

1. `cut -d: -f1,3 data.txt`
2. `awk -F: '$3=="BCA"{print $1}' data.txt`
3. `sort -t: -k2 -n data.txt`
4. `awk -F: '{sum+=$2} END{print sum/NR}' data.txt`

C. Long Answer (5 marks each)

1. Explain the `awk` command in detail. Describe its built-in variables (`NR`, `NF`, `FS`, `OFS`, `$0`) with examples.
2. Describe the `diff` output format. Explain the change codes `c`, `a`, `d`. How does unified format (`-u`) differ from default?

D. Command Practice

1. Using `/etc/passwd`, write a pipeline to extract all usernames (field 1) and login shells (field 7), sort alphabetically, and display only unique shells with a count.
2. Write a command to find the 10 most frequently occurring words in a file `essay.txt` using `tr`, `sort`, and `uniq`.

Unit 4: The VI Editor

Unit 4 — The VI Editor

cal · date · pr · who · bc · echo · zip · unzip · gzip · File Archiving with tar

UNIX ships with a rich collection of utility commands for everyday tasks — displaying calendars and dates, formatting output for printing, querying who is logged in, performing arithmetic, manipulating text, and compressing files. These tools are small, focused, and compose beautifully with pipes.

The `cal` Command — Calendar

Definition — cal

`cal` displays a simple **calendar** in the terminal. It can show the current month, a specific month and year, an entire year, or span multiple months. The current day is highlighted.

```
# Display CURRENT month (today highlighted)
$ cal
     January 2024
Su Mo Tu We Th Fr Sa
     1  2  3  4  5  6
 7  8  9 10 11 12 13
14 15 16 17 18 19 20
21 22 23 24 25 26 27
28 29 30 31

# Display a specific month and year
$ cal 6 2024
      June 2024
Su Mo Tu We Th Fr Sa
                    1
 2  3  4  5  6  7  8
 9 10 11 12 13 14 15
16 17 18 19 20 21 22
23 24 25 26 27 28 29
30

# Display an entire year (all 12 months)
$ cal 2024

# Three months: previous, current, next (-3 flag)
$ cal -3

# Week starts on Monday (-m flag)
$ cal -m
     January 2024
Mo Tu We Th Fr Sa Su
 1  2  3  4  5  6  7
```

```
 8  9 10 11 12 13 14
15 16 17 18 19 20 21
22 23 24 25 26 27 28
29 30 31

# Julian day numbers (day of year 1-366) (-j flag)
$ cal -j

# N months ahead from today (-A flag)
$ cal -A 3

# N months before today (-B flag)
$ cal -B 2

# Historic or future year
$ cal 1947
```

Listing 78: cal — displaying calendars

Syntax	Example	Output
`cal`	`cal`	Current month
`cal MM YYYY`	`cal 8 2024`	August 2024
`cal YYYY`	`cal 2024`	Full year
`cal -3`	`cal -3`	Prev, current, next month
`cal -m`	`cal -m`	Week starts Monday
`cal -j`	`cal -j`	Julian day numbers
`cal -A N`	`cal -A 3`	Current + N months ahead
`cal -B N`	`cal -B 2`	Current + N months before

The date Command

Definition — date

`date` displays or sets the **system date and time**. It supports a rich set of format directives (prefixed with `+%`) to produce output in any desired format — timestamps, log entries, filenames, and more.

```
# Default output
$ date
Mon Jan 15 10:30:45 IST 2024

# UTC time
$ date -u
Mon Jan 15 05:00:45 UTC 2024

# Unix timestamp (seconds since 1 Jan 1970)
$ date +%s
1705296645

# Convert a Unix timestamp to readable form
$ date -d @1705296645
Mon Jan 15 10:30:45 IST 2024

# --- CUSTOM FORMAT DIRECTIVES ---
```

```
$ date +"%d/%m/%Y"              # 15/01/2024
$ date +"%Y-%m-%d"              # 2024-01-15  (ISO 8601)
$ date +"%B %d, %Y"             # January 15, 2024
$ date +"%A, %d %B %Y"          # Monday, 15 January 2024
$ date +"%H:%M:%S"              # 10:30:45  (24-hour)
$ date +"%I:%M %p"              # 10:30 AM  (12-hour)
$ date +"%Y-%m-%d %H:%M:%S"     # 2024-01-15 10:30:45

# Day / week info
$ date +"%A"     # Monday
$ date +"%a"     # Mon
$ date +"%B"     # January
$ date +"%b"     # Jan
$ date +"%j"     # 015  (day of year)
$ date +"%W"     # 03   (week number)
$ date +"%Z"     # IST  (timezone)

# --- RELATIVE DATES (-d flag) ---
$ date -d "30 days"
$ date -d "next Monday"
$ date -d "last year"
$ date -d "+7 days" +"%A, %d %B %Y"
Monday, 22 January 2024

# --- PRACTICAL USES ---
# Timestamped backup filename
$ BACKUP="backup_$(date +%Y%m%d_%H%M%S).tar.gz"
$ tar -czf "$BACKUP" /home/student/

# Add timestamp to a log
$ echo "[$(date +%Y-%m-%d\ %H:%M:%S)] Started" >> run.log

# Check leap year
$ cal 2 2024 | grep -q 29 && echo "Leap year" || echo "Not leap"
Leap year
```

Listing 79: date — displaying and formatting date/time

Directive	Meaning	Example
%Y	4-digit year	2024
%m	Month (01–12)	01
%B	Full month name	January
%d	Day of month	15
%A	Full weekday	Monday
%H	Hour (00–23)	10
%M	Minute (00–59)	30
%S	Second (00–59)	45
%T	HH:MM:SS	10:30:45
%s	Unix timestamp	1705296645
%j	Day of year	015
%Z	Timezone	IST

The pr Command — Print Formatter

Definition — pr

pr formats text files for printing. It adds headers (filename, date, page number), paginates output, arranges text into multiple columns, and merges files side by side. Output is typically piped to a printer or saved to a file.

```
# Default: adds header (filename, date, page no.) and paginates
$ pr students.txt

2024-01-15 10:30          students.txt          Page 1

Alice:20:BCA:Hooghly
Bob:22:BCA:Kolkata
Charlie:21:MCA:Hooghly
Diana:23:MCA:Kolkata
Eve:20:BCA:Hooghly

# Remove header entirely (-t flag)
$ pr -t students.txt

# Custom header text (-h flag)
$ pr -h "Student Database --- BCA 6th Sem" students.txt

# Multi-column layout (-N where N = columns)
$ pr -3 names.txt
2024-01-15              names.txt               Page 1

Alice                Bob                  Charlie
Diana                Eve

# Merge files SIDE BY SIDE (-m flag)
$ pr -m names.txt ages.txt courses.txt
2024-01-15                                       Page 1

Alice                20                   BCA
Bob                  22                   BCA
Charlie              21                   MCA

# Add line numbers (-n flag)
$ pr -n students.txt
     1  Alice:20:BCA:Hooghly
     2  Bob:22:BCA:Kolkata
     3  Charlie:21:MCA:Hooghly

# Double-space the output (-d flag)
$ pr -d students.txt

# Set page width (-w, default 72 chars)
$ pr -w 100 students.txt

# Set page length (-l, default 66 lines)
$ pr -l 50 students.txt

# Set column separator (-s flag)
$ pr -3 -s"|" names.txt
Alice|Bob|Charlie

# Pipe to printer
$ pr -h "BCA Report" students.txt | lpr
```

Listing 80: pr — formatting files for printing

Option	Example	Effect
(default)	`pr file`	Header + pagination
`-t`	`pr -t file`	Remove header/footer
`-h`	`pr -h "Title"`	Custom header text
`-N`	`pr -3 file`	N-column layout
`-m`	`pr -m f1 f2`	Merge files as columns
`-n`	`pr -n file`	Add line numbers
`-d`	`pr -d file`	Double-space output
`-w N`	`pr -w 100`	Set page width
`-l N`	`pr -l 50`	Set page length
`-s C`	`pr -3 -s"|"`	Column separator character

The who Command — Logged-In Users

Definition — who

`who` shows **which users are currently logged into the system**, along with their terminal, login time, and remote host. It reads from `/var/run/utmp`, which the kernel maintains continuously.

```
# Basic usage
$ who
student  pts/0   2024-01-15 09:00 (:0)
root     tty1    2024-01-15 08:45
alice    pts/1   2024-01-15 10:15 (192.168.1.5)
# Fields: username  terminal  login-time  (remote host)

# All information (-a flag)
$ who -a
           system boot  2024-01-15 08:00
           run-level 5  2024-01-15 08:01
student  + pts/0   2024-01-15 09:00  01:30  1234 (:0)

# Current user only
$ who am i
student  pts/0   2024-01-15 09:00 (:0)

# Just the username
$ whoami
student

# Login time + idle time (-u flag)
$ who -u
student  pts/0  2024-01-15 09:00  01:30  1234 (:0)
#                                 ^ idle time (hh:mm)

# Count of logged-in users (-q flag)
$ who -q
student root alice
```

```
# users=3

# Last system boot time (-b flag)
$ who -b
          system boot  2024-01-15 08:00

# Current run level (-r flag)
$ who -r
          run-level 5  2024-01-15 08:01

# With column headers (-H flag)
$ who -H
NAME     LINE    TIME             COMMENT
student  pts/0   2024-01-15 09:00 (:0)
root     tty1    2024-01-15 08:45

# Related commands
$ w               # Users + what they are running right now
$ last            # Full login/logout history
$ last student    # History for a specific user
$ users           # Just usernames, space-separated
```

Listing 81: who — querying logged-in users

Command	Option	Shows
`who`	(default)	Users, terminal, login time, host
`who -a`	`-a`	All info including boot and run-level
`who -u`	`-u`	Idle time for each user
`who -q`	`-q`	Usernames and count only
`who -H`	`-H`	Column headers
`who -b`	`-b`	Last system boot time
`who am i`		Current user's session only
`whoami`		Just the current username
`w`		Users + currently running commands
`last`		Full login/logout history

The bc Command — Arbitrary-Precision Calculator

Definition — bc

bc (*basic calculator*) is an **arbitrary-precision calculator language**. It handles integer and floating-point arithmetic, base conversion, and even has variables, loops, and functions. It reads expressions from stdin or a file.

Interactive and Inline Usage

```
# Start an interactive session
$ bc
2 + 3
5
100 - 47
53
12 * 8
```

```
96
100 / 7
14        # Integer division by default!
quit

# --- SCALE controls decimal places (default: 0) ---
$ bc
scale = 4
100 / 7
14.2857
22 / 7
3.1428
quit

# --- INLINE use with echo (most common in scripting) ---
$ echo "2 + 3" | bc
5

$ echo "scale=2; 22/7" | bc
3.14

$ echo "scale=6; sqrt(2)" | bc -l
1.414213

$ echo "2^10" | bc
1024

$ echo "3^5" | bc
243

# --- BASE CONVERSION ---
# Decimal to binary
$ echo "obase=2; 255" | bc
11111111

$ echo "obase=2; 10" | bc
1010

# Binary to decimal
$ echo "ibase=2; 11111111" | bc
255

# Decimal to hexadecimal
$ echo "obase=16; 255" | bc
FF

# Hex to decimal
$ echo "ibase=16; FF" | bc
255

# --- MATH LIBRARY (-l flag) ---
$ bc -l
scale = 10
s(1)              # sin(1 radian)
.8414709848
c(0)              # cos(0)
1.0000000000
l(2.718281828)    # natural log
.9999999998
4 * a(1)          # pi = 4 * atan(1)
3.1415926532
quit
```

```

# --- IN SHELL SCRIPTS ---
#!/bin/bash
radius=7
area=$(echo "scale=4; 3.14159 * $radius * $radius" | bc)
echo "Area = $area"
# Output: Area = 153.9380
```

Listing 82: bc — interactive and inline calculations

Feature	Syntax	Example
Decimal places	`scale=N`	`echo "scale=2; 10/3" | bc`
Math library	`bc -l`	`echo "sqrt(2)" | bc -l`
Power	`x^y`	`echo "2^8" | bc`
Output base	`obase=N`	`echo "obase=2; 10" | bc`
Input base	`ibase=N`	`echo "ibase=16; FF" | bc`
Sine	`s(x)`	`echo "s(1)" | bc -l`
Cosine	`c(x)`	`echo "c(0)" | bc -l`
Natural log	`l(x)`	`echo "l(2.718)" | bc -l`
Pi	`4*a(1)`	`echo "4*a(1)" | bc -l`

The echo Command

Definition — echo

echo prints its arguments to stdout followed by a newline. It is the most-used command in shell scripting — displaying messages, variable values, generating text, and writing to files via redirection.

```
# Simple message
$ echo "Hello, World!"
Hello, World!

# Print a variable
$ NAME="Amitendu"
$ echo "Hello, $NAME"
Hello, Amitendu

# --- OPTIONS ---

# Suppress trailing newline (-n)
$ echo -n "Enter your name: "
Enter your name: $    # cursor stays on the same line

# Enable escape sequences (-e)
$ echo -e "Line 1\nLine 2\nLine 3"
Line 1
Line 2
Line 3

$ echo -e "Name:\tAlice\nAge:\t20"
Name:   Alice
Age:    20
```

```

# --- ESCAPE SEQUENCES (used with -e) ---
# \n   newline
# \t   horizontal tab
# \b   backspace
# \r   carriage return
# \\   literal backslash
# \a   terminal bell (alert)

# Colour output using ANSI escape codes
$ echo -e "\e[31mRED text\e[0m"
$ echo -e "\e[32mGREEN text\e[0m"
$ echo -e "\e[1mBOLD text\e[0m"
$ echo -e "\e[4mUNDERLINED text\e[0m"
$ echo -e "\e[1;34mBold BLUE text\e[0m"

# --- QUOTING RULES ---
# Single quotes: NO expansion, everything is literal
$ echo 'Hello $NAME today is $(date)'
Hello $NAME today is $(date)

# Double quotes: variables and commands ARE expanded
$ echo "Hello $NAME today is $(date +%A)"
Hello Amitendu today is Monday

# --- WRITING TO FILES ---
$ echo "Hello, UNIX" > hello.txt       # overwrite
$ echo "Second line" >> hello.txt      # append

# --- COMMAND SUBSTITUTION ---
$ echo "You are: $(whoami)"
You are: student

$ echo "Today: $(date +%d-%b-%Y)"
Today: 15-Jan-2024

$ echo "Files here: $(ls | wc -l)"
Files here: 12
```

Listing 83: echo — basic usage, options, and escape sequences

Option	Effect	Example
(default)	Print text with newline	`echo "hello"`
`-n`	No trailing newline	`echo -n "Enter: "`
`-e`	Enable escape sequences	`echo -e "a\tb"`
`-E`	Disable escapes (default)	`echo -E "a\tb"`

The `zip` and `unzip` Commands

Definition — zip / unzip

`zip` creates a **compressed archive** in ZIP format (cross-platform: Windows, macOS, Linux). `unzip` extracts files from ZIP archives. ZIP both archives and compresses in a single step.

`zip` — Creating ZIP Archives

```
# Archive one file
$ zip myarchive.zip notes.txt
  adding: notes.txt (deflated 42%)

# Archive multiple files
$ zip documents.zip file1.txt file2.txt file3.txt

# Archive an entire directory recursively (-r)
$ zip -r projects.zip projects/
  adding: projects/ (stored 0%)
  adding: projects/unix/lab1.sh (deflated 51%)

# Compression level: 0 (none) to 9 (maximum)
$ zip -9 -r best.zip large_folder/
$ zip -0 -r fast.zip large_folder/

# Add files to an existing archive
$ zip myarchive.zip newfile.txt

# Update only changed files (-u flag)
$ zip -u myarchive.zip *.txt

# Delete a file from the archive (-d flag)
$ zip -d myarchive.zip oldfile.txt

# Password-protect the archive (-e flag)
$ zip -e secret.zip private.txt
Enter password: ****

# Exclude files matching a pattern (-x flag)
$ zip -r backup.zip projects/ -x "*.log" -x "*.tmp"

# List contents without extracting
$ zip -sf myarchive.zip
Archive contains:
  notes.txt
  file1.txt
```

Listing 84: zip — creating and managing archives

unzip — Extracting ZIP Archives

```
# Extract all to current directory
$ unzip myarchive.zip

# Extract to a specific directory (-d flag)
$ unzip myarchive.zip -d /tmp/extracted/

# List contents WITHOUT extracting (-l flag)
$ unzip -l myarchive.zip
Archive:  myarchive.zip
  Length      Date    Time    Name
---------  ---------- -----   ----
     2048  01-15-2024 10:30   notes.txt
     1024  01-15-2024 09:00   file1.txt

# Test archive integrity (-t flag)
$ unzip -t myarchive.zip
    testing: notes.txt     OK
    testing: file1.txt     OK
No errors detected.

```

```
# Extract only specific file(s)
$ unzip myarchive.zip notes.txt

# Overwrite without prompting (-o flag)
$ unzip -o myarchive.zip

# Quiet mode (-q flag)
$ unzip -q myarchive.zip

# Detailed info about archive (-v flag)
$ unzip -v myarchive.zip
```

Listing 85: unzip — extracting and inspecting

Task	zip	unzip
Create archive	`zip out.zip files`	—
Recursive	`zip -r out.zip dir/`	—
Max compression	`zip -9 out.zip files`	—
List contents	`zip -sf out.zip`	`unzip -l out.zip`
Extract all	—	`unzip out.zip`
Extract to dir	—	`unzip out.zip -d dir/`
Test integrity	—	`unzip -t out.zip`
Password protect	`zip -e out.zip files`	(auto-prompts)

The gzip Command

Definition — gzip

`gzip` (*GNU zip*) **compresses a single file** using the DEFLATE algorithm and replaces it with a `.gz` file. Unlike `zip`, it cannot bundle multiple files — for that, combine it with `tar`.

```
# Compress a file (REPLACES original with .gz)
$ gzip notes.txt
$ ls
notes.txt.gz

# Decompress (removes .gz, restores original)
$ gzip -d notes.txt.gz
# OR equivalently:
$ gunzip notes.txt.gz

# Keep the original while compressing (-k flag)
$ gzip -k notes.txt
$ ls
notes.txt  notes.txt.gz   # Both exist

# Compression levels: 1 (fastest) to 9 (best)
$ gzip -1 bigfile.log      # Fastest
$ gzip -9 bigfile.log      # Best compression (slower)

# Show compression stats (-v flag)
$ gzip -v notes.txt
notes.txt:   42.3% -- replaced with notes.txt.gz
```

```

# List compression info without decompressing (-l flag)
$ gzip -l notes.txt.gz
     compressed  uncompressed  ratio  name
            585          1024  42.9%  notes.txt

# Test integrity (-t flag)
$ gzip -t notes.txt.gz && echo "File OK"
File OK

# Decompress to stdout (keep .gz file intact)
$ gzip -dc notes.txt.gz     # same as: zcat notes.txt.gz

# Compress multiple files (each gets its own .gz)
$ gzip file1.txt file2.txt file3.txt
# Creates: file1.txt.gz  file2.txt.gz  file3.txt.gz

# Recursively compress all files in a directory (-r)
$ gzip -r /var/log/old_logs/

# Work inside .gz files WITHOUT extracting
$ zcat notes.txt.gz          # print contents to terminal
$ zless notes.txt.gz         # page through contents
$ zgrep "error" log.gz       # grep inside .gz file
```

Listing 86: gzip — compressing and decompressing files

Option	Example	Effect
(default)	`gzip file`	Compress; replace original
`-d`	`gzip -d file.gz`	Decompress (= `gunzip`)
`-k`	`gzip -k file`	Compress and keep original
`-v`	`gzip -v file`	Show compression ratio
`-l`	`gzip -l file.gz`	List sizes and ratio
`-t`	`gzip -t file.gz`	Test integrity
`-c`	`gzip -c file`	Compress to stdout
`-1`	`gzip -1 file`	Fastest compression
`-9`	`gzip -9 file`	Best (slowest) compression
`-r`	`gzip -r dir/`	Recursively compress directory

gzip vs zip

gzip compresses *one file* only. **zip** both archives and compresses multiple files. For UNIX multi-file archives, the standard is `tar + gzip` (.tar.gz) — see the next section.

Archiving Files — `tar`

Definition — tar

`tar` (*tape archive*) **bundles multiple files and directories into a single archive file** while preserving directory structure, permissions, and timestamps. Combined with `gzip` or `bzip2`, it creates compressed archives (`.tar.gz`, `.tar.bz2`).

Core Modes and Options

Flag	Mode	Action
`-c`	Create	Create a new archive
`-x`	Extract	Extract files from archive
`-t`	List	List contents of archive
`-u`	Update	Append files newer than archive copy
`-r`	Append	Add files to end of existing archive
`-f`	File	Specify archive filename (always needed)
`-v`	Verbose	Show files being processed
`-z`	gzip	Compress/decompress with gzip
`-j`	bzip2	Compress/decompress with bzip2
`-J`	xz	Compress/decompress with xz
`-C`	Change	Extract to specified directory

Creating Archives

```
# Create a plain .tar (archive only, no compression)
$ tar -cvf backup.tar projects/
projects/
projects/unix/
projects/unix/lab1.sh

# Create a gzip-compressed .tar.gz
$ tar -czvf backup.tar.gz projects/

# Create a bzip2-compressed .tar.bz2 (better compression)
$ tar -cjvf backup.tar.bz2 projects/

# Create an xz-compressed .tar.xz (best compression)
$ tar -cJvf backup.tar.xz projects/

# Archive multiple directories
$ tar -czvf full_backup.tar.gz /home/student/ /etc/

# Timestamped archive name
$ tar -czvf "backup_$(date +%Y%m%d).tar.gz" projects/

# Exclude files matching a pattern
$ tar -czvf backup.tar.gz projects/ \
    --exclude="*.log" --exclude="*.tmp"

# Preserve permissions (useful with sudo for system backups)
$ sudo tar -czvpf system.tar.gz /etc/
```

Listing 87: tar -c — creating archives

Listing and Extracting Archives

```
# --- LISTING (inspect without extracting) ---
$ tar -tzvf backup.tar.gz
drwxr-xr-x student/staff    0 2024-01-15 10:00 projects/
-rw-r--r-- student/staff 2048 2024-01-15 09:30 projects/unix/notes.txt
-rwxr-xr-x student/staff  512 2024-01-14 14:00 projects/unix/lab1.sh

# --- EXTRACTING ---
# Extract .tar.gz to current directory
$ tar -xzvf backup.tar.gz

# Extract to a SPECIFIC directory (-C flag)
$ tar -xzvf backup.tar.gz -C /tmp/restore/

# Extract only specific files
$ tar -xzvf backup.tar.gz projects/unix/notes.txt

# Extract files matching a wildcard
$ tar -xzvf backup.tar.gz --wildcards "*.sh"

# Extract without verbose (quiet)
$ tar -xzf backup.tar.gz

# Auto-detect compression (modern tar)
$ tar -xvf backup.tar.gz    # -z not needed, tar figures it out
```

Listing 88: tar -t and -x — listing and extracting

Compression Format Comparison

Format	Extension	Flag	Characteristic	Speed
None	.tar	—	Archive only, no compression	Fastest
gzip	.tar.gz	-z	Good compression, universal support	Fast
bzip2	.tar.bz2	-j	Better than gzip, smaller files	Medium
xz	.tar.xz	-J	Best compression, smallest files	Slow

```
# CREATE
tar -czvf archive.tar.gz  directory/    # Create .tar.gz
tar -cjvf archive.tar.bz2 directory/    # Create .tar.bz2

# LIST (peek without extracting)
tar -tzvf archive.tar.gz

# EXTRACT
tar -xzvf archive.tar.gz                # Extract here
tar -xzvf archive.tar.gz -C /dest/      # Extract to /dest/

# Mnemonic for flags:
# c=Create  x=eXtract  t=lisT  z=gZip  j=bzip2  J=xz
# v=Verbose  f=File  C=Change-dir
```

Listing 89: tar — the commands you will use every day

Auto-detect Compression

Modern `tar` can detect the format from the extension. You can usually write `tar -xvf archive.tar.gz` without `-z` and tar will handle it automatically.

Unit 4 Summary

Key Takeaways — Unit 4: The VI Editor

1. **cal** displays calendars; `cal MM YYYY` for a specific month, `cal YYYY` for a full year, `-3` for three-month view, `-j` for Julian day numbers.
2. **date** formats the system time with `+%` directives; `-d` computes relative dates; essential for timestamped log entries and backup filenames in scripts.
3. **pr** formats files for printing — adds headers, paginates, creates multi-column layout (`-N`), merges files side by side (`-m`), and adds line numbers (`-n`).
4. **who** shows logged-in users with terminal and login time; `who -q` gives count; `whoami` shows current user; `w` shows what each user is running.
5. **bc** is an arbitrary-precision calculator; use `scale=N` for decimal places; `bc -l` loads math functions (`s, c, l, sqrt`); pipe `echo "expr" | bc` for inline use in scripts.
6. **echo** prints text and variables; `-n` suppresses newline; `-e` enables escape sequences (`\n, \t`); single quotes prevent expansion, double quotes allow it.
7. **zip** creates cross-platform ZIP archives (`-r` for directories, `-e` for passwords, `-9` for max compression); **unzip -l** lists and **unzip -t** tests integrity.
8. **gzip** compresses a single file replacing it with `.gz`; `-k` keeps the original; `-9` best compression; `gunzip/gzip -d` decompresses; `zcat/zgrep` work on `.gz` without extracting.
9. **tar** bundles files into an archive (`-c` create, `-x` extract, `-t` list, `-f` filename); combine with `-z/-j/-J` for compression; `-C` extracts to a specific directory.
10. The standard UNIX archive format is **.tar.gz**: `tar -czvf` to create, `tar -xzvf` to extract — fast, portable, and universally supported.

End of Unit 4 — The VI Editor
Proceed to Unit 5: Utility Commands

Unit 4 — Practice Questions

Practice Questions — Unit 4: The VI Editor

A. Short Answer (2 marks each)

1. What are the three modes of the vi editor? How do you switch between them?
2. What is the difference between `:w`, `:q`, `:wq`, and `:q!`?
3. How do you delete an entire line in vi? How do you undo the last change?
4. Explain the difference between the `yy` and `p` commands in vi.
5. What does `:set nu` do in vi? Name two other useful `:set` options.
6. How do you search for a word in vi? How do you find and replace all occurrences?
7. What is the difference between `i`, `I`, `a`, `A`, `o`, and `O` for entering insert mode?
8. How would you go to line 42 directly in vi?

B. Match the VI Command (1 mark each)

Command	Action
`dw`	Delete to end of file
`G`	Delete word
`gg`	Go to last line
`dG`	Copy current line
`yy`	Go to first line

C. Long Answer (5 marks each)

1. Explain the vi editor's command mode navigation keys. Cover movement by character, word, line, and screen.
2. Describe the search and substitution features of vi. Explain `:s/old/new/g` and `:%s/old/new/gc` with examples.

D. Practical Scenario

1. You open `report.txt` in vi. Write the sequence of commands to: go to line 15, delete 3 lines, insert new text, replace all occurrences of "error" with "warning", and save without exiting.

Unit 5: Utility Commands

Unit 5 — Utility Commands

cal · date · pr · who · bc · echo · zip · unzip · gzip · File Archiving with tar

Overview of the VI Editor

Definition — VI

VI (*Visual Editor*) is a **screen-oriented text editor** originally written by Bill Joy in 1976 for BSD UNIX. It is the **universal editor** of UNIX — guaranteed to be present on every UNIX and Linux system, no matter how minimal. **Vim** (*VI Improved*), written by Bram Moolenaar in 1991, is the modern extended version and is what you get when you type `vi` on most Linux systems today.

Why Learn VI?

- **Always available** — VI is on every UNIX/Linux system, even embedded systems and minimal server installs where no other editor exists.
- **Works without a GUI** — essential when you are logged in via SSH, in recovery mode, or working on a headless server.
- **Extremely fast** — once you know the keyboard shortcuts, editing is faster than any mouse-driven editor.
- **Handles very large files** — VI loads files efficiently and does not struggle with multi-megabyte logs or source files.
- **Industry standard** — system administrators, DevOps engineers, and developers are expected to know VI basics.

vi vs vim

On virtually all modern Linux systems, `vi` is actually `vim` under the hood. You can check with `vi --version` or `which vi`. Throughout this unit, **VI** and **Vim** are used interchangeably — everything here works in both.

Starting VI

```
# Open a file (create it if it does not exist)
$ vi filename.txt

# Open an existing file
$ vi /etc/hosts

# Open a file and jump to line 25
$ vi +25 filename.txt

# Open a file and jump to the last line
```

```
$ vi + filename.txt

# Open a file and go to first occurrence of a pattern
$ vi +/pattern filename.txt
$ vi +/error logfile.txt

# Open multiple files (navigate with :n and :N)
$ vi file1.txt file2.txt file3.txt

# Open VI with an empty buffer (no file)
$ vi

# Check version
$ vi --version | head -3
VIM - Vi IMproved 9.0
```

Listing 90: Starting VI — opening files

When VI opens, the screen shows:

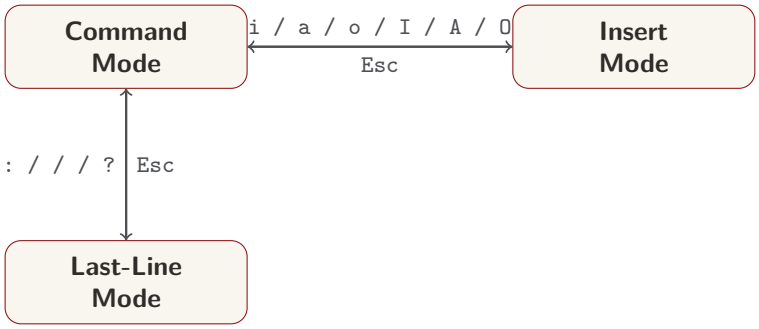
- The file content (or tildes ~ for lines below the file end)
- The filename and line/column count at the bottom status bar
- The cursor positioned at the top-left

The Three Modes of VI

The most important concept in VI is that it operates in **three distinct modes**. Every key you press does something *different* depending on the current mode. Beginners often get confused because they type text and unexpected things happen — almost always because they are in the wrong mode.

Mode	Purpose	How to Enter
Command Mode	Navigate, delete, copy, paste, undo. Keypresses are *commands*, not text.	Press `Esc` from any mode
Insert Mode	Type and edit text. Keys produce characters in the file.	`i`, `a`, `o`, `I`, `A`, `O` from Command Mode
Last-Line Mode	Save, quit, search, replace, run shell commands. Commands appear on the bottom line.	`:`, `/`, or `?` from Command Mode

Mode Diagram



The Golden Rule of VI

When in doubt, press `Esc`. Pressing `Esc` from any mode always returns you to Command Mode safely. Press it twice to be sure.

Command Mode

Command Mode is the **default mode** when VI starts. Every keystroke is interpreted as a command — never as text input. This is where you navigate, delete, copy, paste, and undo.

```
Hello, this is my file.
This is line two.
~
~
~
"myfile.txt" 2L, 44C                    2,1   All
```

Listing 91: What you see in Command Mode

Insert Mode

In Insert Mode, every key you press inserts a character into the file at the cursor position. The word `-- INSERT --` appears at the bottom of the screen.

```
Hello, this is my file.|
This is line two.
~
~
-- INSERT --                             1,26 All
```

Listing 92: What you see in Insert Mode

Last-Line Mode (Ex Mode)

Last-Line Mode is entered by pressing `:` in Command Mode. A colon appears at the bottom of the screen and you type commands there — saving, quitting, searching, replacing.

```
Hello, this is my file.
This is line two.
~
~
:w myfile.txt_
```

Listing 93: What you see in Last-Line Mode

Navigation in VI — Moving the Cursor

Basic Cursor Movement (Command Mode)

VI predates arrow keys on keyboards, so it uses `h j k l` for movement. Arrow keys also work in Vim, but knowing `hjkl` keeps your fingers on the home row — much faster.

`k` (up)

`h` (left) `l` (right)

`j` (down)

```
" ---- BASIC CHARACTER MOVEMENT ----
h       " Move cursor LEFT  one character
l       " Move cursor RIGHT one character
j       " Move cursor DOWN  one line
k       " Move cursor UP    one line

" Also works: arrow keys (Left / Right / Up / Down)

" ---- MULTIPLIER PREFIX ----
" Prefix any motion with a NUMBER to repeat it
5j      " Move DOWN  5 lines
10l     " Move RIGHT 10 characters
3k      " Move UP    3 lines
```

Listing 94: VI navigation — cursor movement in Command Mode

Moving by Word

```
" ---- WORD MOVEMENT ----
w       " Jump to START of NEXT word
W       " Jump to start of next WORD (ignores punctuation)
b       " Jump BACK to start of previous word
B       " Jump back ignoring punctuation
e       " Jump to END of current word
E       " Jump to end of word (ignores punctuation)

" Examples (cursor marked with |):
" |Hello, world! today
" w  -> Hello, |world! today
" w  -> Hello, world! |today
" b  -> Hello, |world! today
" e  -> Hell|o, world! today
```

Listing 95: VI navigation — word-level movement

Moving by Line

```
" ---- LINE MOVEMENT ----
0       " Jump to BEGINNING of current line (column 1)
^       " Jump to FIRST NON-BLANK character of line
$       " Jump to END of current line
+       " Move to first non-blank of NEXT line
-       " Move to first non-blank of PREVIOUS line

" ---- SPECIFIC LINE NUMBERS ----
1G      " Go to line 1  (top of file)
G       " Go to LAST line of file
25G     " Go to line 25
:25     " Go to line 25 (Last-Line mode)
```

Listing 96: VI navigation — line-level movement

Moving by Page / Screen

```
" ---- PAGE / SCREEN SCROLLING ----
Ctrl+f  " Scroll FORWARD one full page (Page Down)
Ctrl+b  " Scroll BACKWARD one full page (Page Up)
Ctrl+d  " Scroll DOWN half a page
Ctrl+u  " Scroll UP half a page
Ctrl+e  " Scroll screen down one line (cursor stays)
Ctrl+y  " Scroll screen up one line (cursor stays)

" ---- SCREEN POSITIONING ----
H       " Move cursor to TOP of screen    (High)
M       " Move cursor to MIDDLE of screen (Middle)
L       " Move cursor to BOTTOM of screen (Low)

" ---- FILE POSITION INFO ----
Ctrl+g  " Show filename, current line, total lines, position %
```

Listing 97: VI navigation — page-level movement

Quick Navigation Reference

Key(s)	Movement	Category
`h / l`	Left / Right one character	Character
`j / k`	Down / Up one line	Line
`w / b`	Next / prev word start	Word
`e`	End of current word	Word
`0 / $`	Line start / end	Line
`^`	First non-blank of line	Line
`G`	Last line of file	File
`1G / gg`	First line of file	File
`NNG`	Go to line NN	File
`Ctrl+f/b`	Page down / up	Page
`Ctrl+d/u`	Half page down / up	Page
`H / M / L`	Top / Middle / Bottom of screen	Screen

Text Editing in VI

Inserting Text — Entering Insert Mode

There are **six ways** to enter Insert Mode, each placing the cursor at a different position:

Key	Name	Where Insertion Begins
`i`	insert	**Before** the cursor
`I`	Insert (line)	At the **beginning** of the current line
`a`	append	**After** the cursor
`A`	Append (line)	At the **end** of the current line
`o`	open below	Opens a **new line below** and enters Insert Mode
`O`	Open above	Opens a **new line above** and enters Insert Mode

```
" Assume cursor is here:  Hell|o world
" (| marks cursor position)

i   " insert BEFORE cursor -> Hell|o world  (type here)
a   " append AFTER cursor  -> Hello| world  (type after 'l')
I   " Insert at LINE start -> |Hello world  (type at col 1)
A   " Append at LINE end   -> Hello world|  (type at end)
o   " Open new line BELOW  -> new blank line appears below
O   " Open new line ABOVE  -> new blank line appears above

" After entering insert mode:
" - Type your text normally
" - Press Esc when done to return to Command Mode
```

Listing 98: Insert mode — all six entry points

Replacing a Single Character Without Insert Mode

`r` in Command Mode replaces the character *under* the cursor with the next key you press — without entering Insert Mode. `R` enters Replace Mode (overwrites characters as you type).

Deleting Text

All delete commands operate in **Command Mode** and also **copy the deleted text** to VI's internal buffer (so you can paste it afterwards with `p`).

```
" ---- DELETE CHARACTERS ----
x       " Delete character UNDER cursor
X       " Delete character BEFORE cursor (like Backspace)
5x      " Delete 5 characters starting from cursor

" ---- DELETE WORDS ----
dw      " Delete from cursor to start of NEXT word
dW      " Delete to next WORD (ignores punctuation)
db      " Delete from cursor BACK to previous word
de      " Delete from cursor to END of current word
d$      " Delete from cursor to END of line
d0      " Delete from cursor to BEGINNING of line

" ---- DELETE LINES ----
dd      " Delete the ENTIRE current line
5dd     " Delete 5 lines starting from current
D       " Delete from cursor to end of line (same as d$)
dG      " Delete from current line to END of file
d1G     " Delete from current line to START of file

```

```
" ---- DELETE TO A PATTERN ----
d/word  " Delete from cursor to next occurrence of 'word'

" ---- EXAMPLES ----
" Line:  The quick brown fox
" dd     -> (entire line deleted)
" dw     -> The |quick brown fox  (cursor now on 'q')
"        -> The | brown fox       (after dw: 'quick ' deleted)
" D      -> The|                  (rest of line deleted)
" x      -> Th|e quick brown fox  ('e' deleted)
```

Listing 99: Deleting text in Command Mode

Command	Deletes	Notes
`x`	Character under cursor	Deleted text goes to buffer
`X`	Character before cursor	Like Backspace
`dw`	Word (cursor to next word)	Includes trailing space
`dd`	Entire current line	Most-used delete command
`5dd`	5 lines	Number prefix repeats
`D`	Cursor to end of line	Same as `d$`
`dG`	Cursor to end of file	Useful for truncating
`d0`	Cursor to start of line	

Copying and Pasting (Yank and Put)

In VI, "copy" is called **yank** and "paste" is called **put**. Deleted text is also automatically yanked, so `dd` followed by `p` moves a line.

```
" ---- YANK (COPY) ----
yy      " Yank (copy) the ENTIRE current line
Y       " Same as yy --- yank current line
5yy     " Yank 5 lines starting from current
yw      " Yank from cursor to end of word
y$      " Yank from cursor to end of line
y0      " Yank from cursor to start of line
yG      " Yank from current line to end of file

" ---- PUT (PASTE) ----
p       " Put AFTER cursor (or below current line for lines)
P       " Put BEFORE cursor (or above current line for lines)

" ---- COMMON WORKFLOWS ----
" Duplicate a line:
yy      " yank current line
p       " paste it below -> line is duplicated

" Move a line down 3 lines:
dd      " delete (cuts) the current line
3j      " move down 3 lines
p       " paste below current position

" Copy a block of 5 lines and paste elsewhere:
5yy     " yank 5 lines
G       " jump to end of file
p       " paste 5 lines at bottom
```

```

" Copy a word and paste it:
yw       " yank word
w        " move to next word position
P        " paste before cursor (inserts the word here)
```

Listing 100: Yank (copy) and Put (paste) in Command Mode

Command	Action	Notes
`yy` / `Y`	Yank current line	Most-used copy command
`5yy`	Yank 5 lines	Number prefix repeats
`yw`	Yank word	
`y$`	Yank to end of line	
`p`	Paste after / below cursor	
`P`	Paste before / above cursor	

Undo and Redo

```
" ---- UNDO ----
u          " Undo the LAST change
u          " Press again to undo the change before that
5u         " Undo the last 5 changes
U          " Undo ALL changes on the CURRENT line only
           " (restores line to what it was when you moved to it)

" ---- REDO (Vim only, not classic vi) ----
Ctrl+r     " Redo --- re-apply the last undone change
5 Ctrl+r   " Redo the last 5 undone changes

" ---- PRACTICAL EXAMPLE ----
" Made 3 typing mistakes:
u          " undo 1st mistake
u          " undo 2nd mistake
u          " undo 3rd mistake
Ctrl+r     " oops, went too far --- redo 1 step
```

Listing 101: Undo and Redo in Command Mode

Vim vs Classic VI — Undo Levels

Classic `vi` supports only **one level** of undo. **Vim** supports unlimited undo history for the session. On any modern Linux system `vi` is Vim, so you have unlimited undo. `Ctrl+r` for redo is **Vim-only** — it does not exist in classic vi.

Working with Files — Saving and Exiting

File operations in VI are done from **Last-Line Mode** (entered by pressing `:` in Command Mode).

Saving Files

```
" Enter Last-Line Mode by pressing : in Command Mode

:w               " Write (save) the file --- keep editing
:w filename      " Save AS a new filename (save a copy)
:w!              " Force write --- override read-only flag (if permitted)
```

```

" Examples:
:w                      " Save changes to current file
:w backup.txt           " Save a copy to backup.txt (current file unchanged)
:w /tmp/draft.txt       " Save copy to a full path

" Write a range of lines to a file
:1,10w output.txt       " Save lines 1 to 10 to output.txt
:.,$ w rest.txt         " Save from current line to end of file
```

Listing 102: Saving files in Last-Line Mode

Exiting VI

```
:q              " Quit (only works if NO unsaved changes)
:q!             " Force quit --- EXIT WITHOUT SAVING (discard changes)
:wq             " Write and Quit --- save then exit
:wq!            " Force write then quit
:x              " Write if modified, then quit (like :wq but smarter)
ZZ              " Save and quit (Command Mode shortcut, no : needed)
ZQ              " Quit WITHOUT saving (Command Mode shortcut)

" ---- COMMON SCENARIOS ----
" You edited a file and want to save and exit:
:wq     " or press ZZ in Command Mode

" You made a mistake and want to exit WITHOUT saving:
:q!     " discard all changes and exit

" You want to save but keep editing:
:w      " saves file, cursor stays, keep working

" File is read-only but you have sudo rights:
:w !sudo tee %  " write using sudo (Vim trick)
```

Listing 103: Exiting VI

Command	Mode	Effect
`:w`	Last-Line	Save file, continue editing
`:w fn`	Last-Line	Save as new filename `fn`
`:q`	Last-Line	Quit (fails if unsaved changes exist)
`:q!`	Last-Line	Force quit — discard ALL changes
`:wq`	Last-Line	Save and quit
`:wq!`	Last-Line	Force save and quit
`:x`	Last-Line	Save only if modified, then quit
`ZZ`	Command	Shortcut for `:wq`
`ZQ`	Command	Shortcut for `:q!`

Common Beginner Trap — "I can't exit VI!"

The most famous joke in computing: *"How do I exit VI?"* The answer:

1. Press `Esc` (make sure you are in Command Mode)
2. Type `:q!` and press Enter (exit without saving)
3. OR type `:wq` and press Enter (save and exit)

If nothing seems to work, keep pressing `Esc` until the mode indicator at the bottom disappears, then try again.

File Operations in VI

Opening a New or Different File

```
" ---- OPEN A FILE (while already in VI) ----
:e filename          " Open (edit) filename in current window
:e /etc/hosts        " Open system hosts file
:e!                  " Re-open CURRENT file --- discard unsaved changes
:e!                  " Useful to reload a file edited externally

" ---- OPEN MULTIPLE FILES / SWITCHING ----
" When VI was started with multiple files: vi f1.txt f2.txt
:n                   " Go to NEXT file in argument list
:N                   " Go to PREVIOUS file
:args                " Show all files in argument list
:first               " Go to first file
:last                " Go to last file

" ---- SPLIT WINDOWS (Vim) ----
:split filename      " Open filename in a HORIZONTAL split
:vsplit filename     " Open filename in a VERTICAL split
Ctrl+w w             " Switch between split windows
Ctrl+w q             " Close current split window

" ---- INCLUDING ANOTHER FILE ----
:r filename          " Read filename and INSERT it below cursor
:r /etc/hosts        " Insert /etc/hosts content at cursor position
:r !date             " Insert output of shell command 'date' at cursor
```

Listing 104: File operations from within VI

Shell Commands from VI

```
" Run a shell command and see its output
:!ls -la             " Run 'ls -la' and show output temporarily
:!pwd                " Show current directory
:!date               " Show current date

" Suspend VI and go to shell (Ctrl+Z)
" Then type 'fg' to return to VI

" Run shell and return
:sh                  " Start a shell; type 'exit' to return to VI

" Filter selected text through a command
:!sort               " Sort entire file in place
:1,5!sort            " Sort lines 1 to 5 in place
```

Listing 105: Running shell commands without leaving VI

Search in VI

Definition — Searching in VI

VI's search function is entered from **Command Mode**. Searching forward uses `/`, searching backward uses `?`. VI supports **POSIX regular expressions** for pattern matching.

Forward and Backward Search

```
" ---- FORWARD SEARCH (from cursor toward end of file) ----
/pattern        " Search FORWARD for 'pattern'
/error          " Find next occurrence of the word 'error'
/[0-9]\+        " Search for one or more digits (regex)

" After typing /pattern and pressing Enter:
n               " Jump to NEXT occurrence (forward)
N               " Jump to PREVIOUS occurrence (backward)

" ---- BACKWARD SEARCH (from cursor toward start of file) ----
?pattern        " Search BACKWARD for 'pattern'
?function       " Find previous occurrence of 'function'

" After ?pattern:
n               " Continue searching BACKWARD (same direction)
N               " Reverse direction (search forward now)

" ---- SEARCH FOR WORD UNDER CURSOR ----
*               " Search FORWARD for exact word under cursor
#               " Search BACKWARD for exact word under cursor

" ---- SEARCH WRAPPING ----
" By default search wraps around end of file
" You will see: search hit BOTTOM, continuing at TOP

" ---- CLEAR SEARCH HIGHLIGHT ----
:noh            " No highlight --- turn off search highlighting
:nohlsearch     " Same as :noh (full form)
```

Listing 106: Searching forward and backward

Regular Expressions in VI Search

VI supports POSIX Basic Regular Expressions (BRE) in search patterns:

Pattern	Meaning	Example
`.`	Any single character	`/c.t` matches cat, cot, cut
`*`	Zero or more of preceding	`/ab*` matches a, ab, abb …
`^`	Start of line	`/^Hello` matches lines starting with Hello
`$`	End of line	`/world$` matches lines ending with world
`[abc]`	Character class	`/[aeiou]` matches any vowel
`[a-z]`	Range	`/[a-z]` matches any lowercase letter
`[^abc]`	Negated class	`/[^0-9]` matches non-digit
`\w`	Word character (`[a-zA-Z0-9_]`)	`/\w\+` matches a word
`\+`	One or more (Vim BRE)	`/[0-9]\+` matches number
`\( \)`	Grouping	`/\(ab\)*`

```
" Find lines starting with 'Alice'
/^Alice

" Find lines ending with a semicolon
/;$

" Find any word containing only digits
/^[0-9]\+$

" Find email-like patterns
/@[a-zA-Z]\+\.[a-zA-Z]\+/

" Find blank lines (lines with only whitespace)
/^\s*$

" Find lines longer than 80 characters
/.\{81,\}

" Case-insensitive search (prepend \c)
/\cerror        " matches error, Error, ERROR, eRrOr

" Case-sensitive search (prepend \C)
/\CError        " matches ONLY 'Error' (capital E, lowercase rror)
```

Listing 107: Regex search examples in VI

Search Navigation Summary

Key	Action	Notes
`/pattern`	Search forward for pattern	Supports regex
`?pattern`	Search backward for pattern	Supports regex
`n`	Repeat search (same direction)	
`N`	Repeat search (reverse direction)	
`*`	Search forward for word under cursor	Exact word match
`#`	Search backward for word under cursor	Exact word match
`:noh`	Turn off search highlight	

Search and Replace in VI

Definition — Substitute Command

The **substitute command** (`:s`) performs search and replace. It is entered in **Last-Line Mode** and follows the pattern `:s/old/new/flags`. It is one of the most powerful features of VI.

Basic Substitute Syntax

```
:[range]s/pattern/replacement/[flags]
    |          |         |          |
  scope     what to    replace    how
            find       with
```

Listing 108: Substitute command structure

Replacing on Current Line

```
" ---- REPLACE FIRST OCCURRENCE ON CURRENT LINE ----
:s/old/new          " Replace FIRST 'old' with 'new' on current line
:s/unix/UNIX        " Replace first 'unix' with 'UNIX' on current line

" Example:
" Line:  hello unix, unix is great
" :s/unix/UNIX
" Result: hello UNIX, unix is great   (only first one changed)

" ---- REPLACE ALL OCCURRENCES ON CURRENT LINE (/g flag) ----
:s/old/new/g        " Replace ALL occurrences on current line
:s/unix/UNIX/g      " Replace every 'unix' on this line

" Example:
" Line:  hello unix, unix is great
" :s/unix/UNIX/g
" Result: hello UNIX, UNIX is great   (both changed)
```

Listing 109: :s — replacing on the current line

Global Search and Replace (Entire File)

```
" ---- REPLACE IN ENTIRE FILE (% range = all lines) ----
:%s/old/new/g       " Replace ALL occurrences EVERYWHERE in file
:%s/unix/UNIX/g     " Replace every 'unix' with 'UNIX' in whole file

" ---- REPLACE WITH CONFIRMATION (/c flag) ----
:%s/old/new/gc      " Ask before EACH replacement
" Prompts for each match:
"   y = yes (replace this one)
"   n = no  (skip this one)
"   a = all (replace all remaining)
"   q = quit (stop replacing)
"   l = last (replace this one then stop)

" ---- CASE-INSENSITIVE REPLACE (/i flag) ----
:%s/unix/UNIX/gi    " Replace unix, Unix, UNIX, uNiX etc.

" ---- REPLACE WITH REGEX PATTERNS ----
" Replace multiple spaces with a single space
:%s/  */ /g

" Remove trailing whitespace from all lines
:%s/\s\+$//g
```

```

" Delete blank lines
:g/^\s*$/d

" Add a comment character to beginning of every line
:%s/^/# /

" Remove leading spaces from all lines
:%s/^\s\+//

" Wrap every word in double quotes
:%s/\(\w\+\)/"\1"/g

" Replace whole word 'cat' only (not 'concatenate')
:%s/\bcat\b/dog/g

" ---- REPLACE IN A SPECIFIC LINE RANGE ----
:1,10s/old/new/g    " Replace in lines 1 to 10 only
:5,15s/foo/bar/g    " Replace in lines 5 to 15
:.,$ s/old/new/g    " Replace from current line to end of file
:.,+5s/old/new/g    " Replace in current line + next 5 lines
```

Listing 110: :%s — global search and replace

Substitute Flags Reference

Flag	Name	Effect
`g`	global	Replace all occurrences on each line (not just first)
`c`	confirm	Prompt for confirmation before each replacement
`i`	ignore	Case-insensitive matching
`I`	Ignore	Force case-sensitive (overrides `ignorecase` setting)
`e`	error	Suppress error if pattern not found
`p`	print	Print the last changed line

Range Specifiers

Range	Meaning	Example
(none)	Current line only	`:s/a/b/g`
`%`	Entire file	`:%s/a/b/g`
`1,10`	Lines 1 to 10	`:1,10s/a/b/g`
`.`	Current line	`:.s/a/b/g`
`$`	Last line	`:$s/a/b/g`
`.,$`	Current to last line	`:.,$s/a/b/g`
`1,$`	First to last (= %)	`:1,$s/a/b/g`
`.,+5`	Current + next 5 lines	`:.,+5s/a/b/g`

```
" ---- REAL-WORLD EXAMPLES ----

" Fix a typo ('teh' -> 'the') throughout a document
:%s/teh/the/g
```

```

" Replace a variable name in a source file (whole word)
:%s/\bold_variable\b/new_variable/g

" Convert Windows line endings (\r\n) to UNIX (\n)
:%s/\r//g

" Make all text lowercase (using \L)
:%s/.*/\L&/

" Make first character of each line uppercase
:%s/^\(.\)/\u\1/

" Number all lines in the file
:%s/^/\=line('.') . '. '/

" Delete all lines containing the word 'debug'
:g/debug/d

" Delete all lines NOT containing 'important'
:g!/important/d

" ---- VERIFY BEFORE COMMITTING ---
" Always use /c (confirm) flag when making large replacements
:%s/username/newname/gc
" This lets you see and approve each change individually
```

Listing 111: Complete search and replace — practical examples

Essential VI Cheat Sheet

Mode Switching		
`Esc`	Any → Command	Return to Command Mode (use freely!)
`i / a`	Command → Insert	Before / after cursor
`I / A`	Command → Insert	Start / end of line
`o / O`	Command → Insert	New line below / above
`:`	Command → Last-Line	Enter ex command
Navigation		
`h j k l`		Left / Down / Up / Right
`w / b`		Next / prev word
`0 / $`		Line start / end
`gg / G`		File start / end
`:N`		Go to line N
`Ctrl+f / b`		Page down / up
Editing		
`x`	Delete char	Under cursor
`dd`	Delete line	Also cuts to buffer
`yy`	Yank (copy) line	
`p / P`	Paste after / before	
`u`	Undo	Unlimited in Vim
`Ctrl+r`	Redo	Vim only
`r`	Replace char	No Insert Mode needed
`.`	Repeat last change	Very powerful!
Save and Quit		
`:w`	Save	Keep editing
`:wq / ZZ`	Save and quit	Both work
`:q!`	Quit no save	Discard changes
Search and Replace		
`/pattern`	Search forward	`n`/`N` next/prev
`?pattern`	Search backward	
`*`	Search word at cursor	Forward
`:%s/old/new/g`	Replace all	Entire file
`:%s/old/new/gc`	Replace w/ confirm	Safer for bulk changes

Unit 5 Summary

Key Takeaways — Unit 5: Utility Commands

1. **VI is universal** — it is guaranteed to be present on every UNIX/Linux system and is essential for server/remote work where no GUI is available.
2. **Three modes**: **Command Mode** (default, navigation and editing commands), **Insert Mode** (text entry), and **Last-Line Mode** (save, quit, search, replace). Press `Esc` at any time to return to Command Mode.
3. **Start VI** with `vi filename`; use `vi +N file` to open at line N; `vi +/pattern` to open at a search match.
4. **Navigation**: `h j k l` move left/down/up/right; `w/b` by word; `0/$` to line start/end; `gg/G` to file start/end; `Ctrl+f/b` page down/up.
5. **Inserting text**: `i` before cursor, `a` after, `I` at line start, `A` at line end, `o` new line below, `O` new line above.
6. **Deleting**: `x` deletes a character, `dd` a line, `dw` a word, `D` to end of line; all deletions are also cut to the buffer.
7. **Yank and Put**: `yy` copies a line, `5yy` copies 5 lines, `p` pastes below/after, `P` pastes above/before.
8. **Undo/Redo**: `u` undoes (unlimited in Vim), `Ctrl+r` redoes (Vim only). Press `u` repeatedly to step back through changes.
9. **Saving and exiting**: `:w` save, `:q` quit (if no changes), `:q!` force quit without saving, `:wq` or `ZZ` save and quit.
10. **File operations**: `:e filename` opens a file, `:r filename` inserts a file at cursor, `:!cmd` runs a shell command.
11. **Searching**: `/pattern` searches forward, `?pattern` backward; `n/N` repeat; `*` searches the word under the cursor; full POSIX regex supported.
12. **Search and Replace**: `:s/old/new/g` replaces on current line; `:%s/old/new/g` replaces throughout the file; `/c` flag adds confirmation; `/i` makes it case-insensitive.

End of Unit 5 — Utility Commands

Proceed to Unit 6: Advanced File Searching with GREP and AWK

Unit 5 — Practice Questions

Practice Questions — Unit 5: Utility Commands

A. Short Answer (2 marks each)

1. What is the difference between `gzip` and `zip`? When would you use `tar -czvf`?
2. Explain the `date +"%Y-%m-%d %H:%M:%S"` command. What does each directive mean?
3. What does `bc -l` do differently from plain `bc`?
4. How does `cal -3` differ from `cal 2024`?
5. What is the difference between `who` and `w`? What extra info does `w` show?
6. Explain `tar -xzvf backup.tar.gz -C /restore/`. What does each flag do?
7. What does `echo -e "Name:\tAlice\nAge:\t22"` produce?
8. What is the difference between `gzip -k` and plain `gzip`?

B. Fill in the Blanks (1 mark each)

1. To calculate `22/7` to 4 decimal places using `bc`: _____.
2. The command to list contents of a `.tar.gz` without extracting: _____.
3. To display the current month's calendar with week starting Monday: _____.
4. The `pr -3` flag arranges output in _____ columns.
5. `gunzip file.gz` is equivalent to _____.

C. Long Answer (5 marks each)

1. Explain the `tar` command in detail. Describe all major flags and give examples of creating, listing, and extracting archives in `.tar.gz`, `.tar.bz2`, and `.tar.xz` formats.
2. Write a shell script using `date`, `tar`, and `gzip` that creates a timestamped backup of `/home/student/` every time it is run, logging start and end times to `backup.log`.

Unit 6: Advanced File Searching — GREP and AWK

Unit 6 — Advanced File Searching: GREP and AWK

grep patterns · Advanced grep · awk print/printf · Comparison & Arithmetic · BEGIN/END · if-else · FS/OFS · String & Math Functions · Loops · Search & Substitute

`grep` and `awk` are the two most powerful text-searching and text-processing tools in UNIX. Together they form the backbone of shell-based data processing — finding patterns, extracting fields, computing values, and transforming text, all without writing a full program.

Searching Files with Pattern Using `grep` and `awk`

Definition — grep

grep (*Global Regular Expression Print*) searches files line by line and prints every line that matches a given pattern. It supports Basic Regular Expressions (BRE), Extended Regular Expressions (ERE with `-E`), and Perl-compatible expressions (with `-P`).

Definition — awk

awk is a full pattern-action programming language. For each line of input it checks a *pattern*; if the pattern matches, it runs an *action*. It excels at field-based data extraction, arithmetic, and formatted output.

Sample Data Files Used in This Unit

```
# employees.txt
Alice   Engineering  85000  F  Hooghly
Bob     Marketing    60000  M  Kolkata
Charlie Engineering  92000  M  Hooghly
Diana   HR           55000  F  Kolkata
Eve     Engineering  78000  F  Hooghly
Frank   Marketing    67000  M  Kolkata
Grace   HR           58000  F  Hooghly

# sales.txt (Month:Product:Units:Price)
Jan:Laptop:10:45000
Feb:Phone:25:12000
Mar:Laptop:8:45000
Apr:Phone:30:12000
May:Tablet:15:18000
Jun:Laptop:12:45000
```

Listing 112: employees.txt and sales.txt — sample data

Advanced Searching with `grep`

Basic grep Usage

```
# Basic syntax
grep [OPTIONS] PATTERN [FILE(s)]

# Search for a word in a file
$ grep "Engineering" employees.txt
Alice   Engineering  85000  F  Hooghly
Charlie Engineering  92000  M  Hooghly
Eve     Engineering  78000  F  Hooghly

# Case-insensitive search (-i)
$ grep -i "engineering" employees.txt

# Show line numbers (-n)
$ grep -n "Kolkata" employees.txt
2:Bob     Marketing    60000  M  Kolkata
4:Diana   HR           55000  F  Kolkata
6:Frank   Marketing    67000  M  Kolkata

# Count matching lines (-c)
$ grep -c "Engineering" employees.txt
3

# Invert match --- show lines that do NOT match (-v)
$ grep -v "Engineering" employees.txt
Bob     Marketing    60000  M  Kolkata
Diana   HR           55000  F  Kolkata
Frank   Marketing    67000  M  Kolkata
Grace   HR           58000  F  Hooghly

# Show only the matching PART of the line (-o)
$ grep -o "Engineering\|Marketing\|HR" employees.txt
Engineering
Marketing
Engineering
HR
Engineering
Marketing
HR
```

Listing 113: grep — basic pattern searching

grep with Regular Expressions

```
# Lines starting with 'A' (^ = start of line)
$ grep "^A" employees.txt
Alice   Engineering  85000  F  Hooghly

# Lines ending with 'ly' ($ = end of line)
$ grep "ly$" employees.txt
Alice   Engineering  85000  F  Hooghly
Charlie Engineering  92000  M  Hooghly
Eve     Engineering  78000  F  Hooghly
Grace   HR           58000  F  Hooghly

# Match any single character with .
$ grep "^.li" employees.txt
Alice   Engineering  85000  F  Hooghly

# Match EXACT word (--word-regexp or -w)
$ grep -w "HR" employees.txt
Diana   HR           55000  F  Kolkata
Grace   HR           58000  F  Hooghly
```

```

# Extended regex (-E) --- use + ? | () without backslash
$ grep -E "Alice|Eve|Grace" employees.txt
Alice   Engineering  85000  F  Hooghly
Eve     Engineering  78000  F  Hooghly
Grace   HR           58000  F  Hooghly

# Match salary range: 5 digits starting with 8 or 9
$ grep -E "[89][0-9]{4}" employees.txt
Alice   Engineering  85000  F  Hooghly
Charlie Engineering  92000  M  Hooghly
Eve     Engineering  78000  F  Hooghly

# Lines containing both 'F' and 'Hooghly' (chain greps)
$ grep "F" employees.txt | grep "Hooghly"
Alice   Engineering  85000  F  Hooghly
Eve     Engineering  78000  F  Hooghly
Grace   HR           58000  F  Hooghly
```

Listing 114: grep — regular expression patterns

Advanced grep Options

```
# Recursive search through directories (-r)
$ grep -r "TODO" /home/student/projects/
/home/student/projects/main.c:42:    // TODO: fix buffer overflow
/home/student/projects/utils.sh:15:# TODO: add error handling

# Show N lines of context AFTER match (-A N)
$ grep -A 2 "Charlie" employees.txt
Charlie Engineering  92000  M  Hooghly
Diana   HR           55000  F  Kolkata
Eve     Engineering  78000  F  Hooghly

# Show N lines of context BEFORE match (-B N)
$ grep -B 1 "Diana" employees.txt
Bob     Marketing    60000  M  Kolkata
Diana   HR           55000  F  Kolkata

# Show N lines BEFORE and AFTER match (-C N)
$ grep -C 1 "Diana" employees.txt
Bob     Marketing    60000  M  Kolkata
Diana   HR           55000  F  Kolkata
Eve     Engineering  78000  F  Hooghly

# Show only filenames that contain a match (-l)
$ grep -l "error" /var/log/*.log
/var/log/syslog
/var/log/auth.log

# Show only filenames that do NOT match (-L)
$ grep -L "error" /var/log/*.log

# Suppress error messages (-s flag)
$ grep -s "pattern" /nonexistent/file.txt

# Fixed string search (no regex, faster) (-F)
$ grep -F "85000" employees.txt
Alice   Engineering  85000  F  Hooghly

# Count lines in multiple files
$ grep -c "error" /var/log/syslog /var/log/auth.log
/var/log/syslog:42
```

```
/var/log/auth.log:7

# Perl regex: match salary > 80000 (lookahead etc.)
$ grep -P "\b[89]\d{4}\b" employees.txt
Alice   Engineering  85000  F  Hooghly
Charlie Engineering  92000  M  Hooghly
Eve     Engineering  78000  F  Hooghly
```

Listing 115: grep — advanced flags and practical uses

Option	Long form	Effect
-i	--ignore-case	Case-insensitive matching
-v	--invert-match	Show lines that do NOT match
-n	--line-number	Prefix each match with line number
-c	--count	Count matching lines only
-l	--files-with-matches	Show filenames that match
-w	--word-regexp	Match whole words only
-o	--only-matching	Print only the matched portion
-r	--recursive	Recurse into subdirectories
-E	--extended-regexp	Use extended regular expressions
-F	--fixed-strings	Treat pattern as literal string
-P	--perl-regexp	Use Perl-compatible regex
-A N	--after-context	Show N lines after each match
-B N	--before-context	Show N lines before each match
-C N	--context	Show N lines before and after

awk with print and printf

Definition — awk Program Structure

Every `awk` program follows the pattern: `awk 'pattern { action }' file`. If the pattern is omitted, the action runs on every line. If the action is omitted, the matching line is printed. **Fields** are split by whitespace by default: `$1` is the first field, `$2` the second, `$0` is the entire line, `NF` is the total field count, `NR` is the current line number.

print — Basic Output

```
# Print entire line (same as cat)
$ awk '{print}' employees.txt
$ awk '{print $0}' employees.txt    # identical

# Print specific fields
$ awk '{print $1}' employees.txt    # names only
Alice
Bob
Charlie
Diana
Eve
Frank
```

```
Grace

# Print multiple fields with custom separator
$ awk '{print $1, $2}' employees.txt
Alice Engineering
Bob Marketing
Charlie Engineering

# Print with custom output string (concatenation)
$ awk '{print $1 " works in " $2 " and earns " $3}' employees.txt
Alice works in Engineering and earns 85000
Bob works in Marketing and earns 60000
Charlie works in Engineering and earns 92000

# Print fields with custom delimiter (using OFS)
$ awk 'BEGIN{OFS=","} {print $1,$2,$3}' employees.txt
Alice,Engineering,85000
Bob,Marketing,60000
Charlie,Engineering,92000

# Print with line numbers
$ awk '{print NR". "$1, $2}' employees.txt
1. Alice Engineering
2. Bob Marketing
3. Charlie Engineering

# Print number of fields in each line
$ awk '{print NR, NF, $0}' employees.txt
1 5 Alice   Engineering  85000  F  Hooghly
2 5 Bob     Marketing    60000  M  Kolkata
```

Listing 116: awk print — extracting and formatting fields

printf — Formatted Output

printf in **awk** works exactly like C's **printf** — giving precise control over field widths, alignment, and number formatting.

```
# printf syntax: printf "format", arg1, arg2, ...
# Format codes: %s = string, %d = integer, %f = float
#               %10s = right-align in 10 chars
#               %-10s = left-align in 10 chars
#               %.2f = float with 2 decimal places

# Right-aligned columns (neat table)
$ awk '{printf "%-10s %-15s %8d\n", $1, $2, $3}' employees.txt
Alice      Engineering       85000
Bob        Marketing         60000
Charlie    Engineering       92000
Diana      HR                55000
Eve        Engineering       78000
Frank      Marketing         67000
Grace      HR                58000

# Add a header before the data (using BEGIN)
$ awk 'BEGIN {printf "%-10s %-15s %8s\n", "Name", "Dept", "Salary"}
      {printf "%-10s %-15s %8d\n", $1, $2, $3}' employees.txt
Name       Dept               Salary
Alice      Engineering         85000
Bob        Marketing           60000

# Format salary as currency
$ awk '{printf "%s earns Rs.%d per year\n", $1, $3}' employees.txt
```

```
Alice earns Rs.85000 per year
Bob earns Rs.60000 per year

# Print float with 2 decimal places (salary in lakhs)
$ awk '{printf "%-10s %.2f lakhs\n", $1, $3/100000}' employees.txt
Alice      0.85 lakhs
Bob        0.60 lakhs
Charlie    0.92 lakhs
```

Listing 117: awk printf — formatted and aligned output

Format	Meaning	Example Output
`%s`	String	`Alice`
`%d`	Integer	`85000`
`%f`	Float	`85000.000000`
`%.2f`	Float, 2 decimals	`85000.00`
`%10s`	Right-align, width 10	`     Alice`
`%-10s`	Left-align, width 10	`Alice`
`%010d`	Zero-pad to width 10	`0000085000`
`\n`	Newline	(line break)
`\t`	Tab	(tab space)

awk with Comparison Operators

Pattern in awk

In `awk`, a **comparison expression** used as the pattern filters which lines the action runs on. Only lines where the comparison is **true** are processed.

```
# Comparison operators: == != < <= > >= ~ !~

# Print employees with salary > 70000
$ awk '$3 > 70000' employees.txt
Alice   Engineering  85000  F  Hooghly
Charlie Engineering  92000  M  Hooghly
Eve     Engineering  78000  F  Hooghly

# Print employees with salary == 55000
$ awk '$3 == 55000 {print $1, $3}' employees.txt
Diana 55000

# Print employees with salary between 60000 and 80000
$ awk '$3 >= 60000 && $3 <= 80000 {print $1, $3}' employees.txt
Bob 60000
Eve 78000
Frank 67000

# Print Female employees (field 4 == "F")
$ awk '$4 == "F" {print $1, $2}' employees.txt
Alice Engineering
Diana HR
Eve Engineering
Grace HR
```

```

# Regex match with ~ (field 2 matches "Engineering")
$ awk '$2 ~ /Engineering/ {print $1, $3}' employees.txt
Alice 85000
Charlie 92000
Eve 78000

# Regex NOT match with !~ (department is NOT HR)
$ awk '$2 !~ /HR/ {print $1, $2}' employees.txt
Alice Engineering
Bob Marketing
Charlie Engineering
Eve Engineering
Frank Marketing

# String comparison
$ awk '$5 == "Hooghly" {print $1}' employees.txt
Alice
Charlie
Eve
Grace

# Multiple conditions with OR (||)
$ awk '$3 > 90000 || $2 == "HR"' employees.txt
Charlie Engineering  92000  M  Hooghly
Diana   HR           55000  F  Kolkata
Grace   HR           58000  F  Hooghly

# NOT operator (!)
$ awk '!($2 == "Marketing")' employees.txt
Alice   Engineering  85000  F  Hooghly
Charlie Engineering  92000  M  Hooghly
Diana   HR           55000  F  Kolkata
Eve     Engineering  78000  F  Hooghly
Grace   HR           58000  F  Hooghly
```

Listing 118: awk comparison operators — filtering rows

Operator	Meaning	Example
`==`	Equal to	`$4 == "F"`
`!=`	Not equal to	`$2 != "HR"`
`>`	Greater than	`$3 > 70000`
`>=`	Greater than or equal	`$3 >= 60000`
`<`	Less than	`$3 < 60000`
`<=`	Less than or equal	`$3 <= 80000`
`~`	Matches regex	`$2 ~/Eng/`
`!~`	Does not match regex	`$2 !~/HR/`
`&&`	Logical AND	`$3>60000 && $4=="F"`
`||`	Logical OR	`$3>90000 || $2=="HR"`
`!`	Logical NOT	`!($4=="M")`

awk with Arithmetic Operators

```
# Basic arithmetic on fields
$ awk '{print $1, $3 + 10000}' employees.txt   # salary + bonus
Alice 95000
Bob 70000

# Arithmetic operators: + - * / % ^ (power)

# Calculate annual salary (assume $3 is monthly)
$ awk '{print $1, $3 * 12 " per year"}' employees.txt
Alice 1020000 per year

# Calculate 20% tax
$ awk '{tax = $3 * 0.20; print $1, "Tax:", tax}' employees.txt
Alice Tax: 17000
Bob Tax: 12000

# Calculate take-home after 20% tax
$ awk '{takehome = $3 - ($3 * 0.20)
        printf "%-10s Gross: %d  Tax: %d  Take-Home: %d\n",
               $1, $3, $3*0.20, takehome}' employees.txt
Alice      Gross: 85000  Tax: 17000  Take-Home: 68000
Bob        Gross: 60000  Tax: 12000  Take-Home: 48000
Charlie    Gross: 92000  Tax: 18400  Take-Home: 73600

# Modulo operator (remainder)
$ awk 'NR % 2 == 0 {print NR, $0}' employees.txt  # even lines only
2 Bob     Marketing    60000  M  Kolkata
4 Diana   HR           55000  F  Kolkata
6 Frank   Marketing    67000  M  Kolkata

# Increment / Decrement operators
$ awk '{count++; print count, $1}' employees.txt
1 Alice
2 Bob
3 Charlie

# Power operator
$ awk 'BEGIN {print 2^8}'
256

$ awk 'BEGIN {print 3^3}'
27

# Compound assignment: += -= *= /=
$ awk '{total += $3} END {print "Total payroll:", total}' employees.txt
Total payroll: 493000
```

Listing 119: awk arithmetic operators

awk BEGIN and END Sections

BEGIN and END

BEGIN is a special pattern whose action runs **once before** any input is read — used for initialization, printing headers, and setting variables. **END** runs **once after** all input has been processed — used for printing totals, summaries, and final calculations.

```
# Basic structure
awk 'BEGIN { initialization }
     pattern { action }
```

```
     END { finalization }' file

# Print header and footer
$ awk 'BEGIN { print "=== Employee Report ===" }
      { print $1, $2, $3 }
      END   { print "=== End of Report ===" }' employees.txt
=== Employee Report ===
Alice Engineering 85000
Bob Marketing 60000
Charlie Engineering 92000
Diana HR 55000
Eve Engineering 78000
Frank Marketing 67000
Grace HR 58000
=== End of Report ===

# Count and sum (most common use of BEGIN/END)
$ awk 'BEGIN { count=0; total=0 }
      { count++; total += $3 }
      END { print "Employees:", count
            print "Total Payroll:", total
            print "Average Salary:", total/count }' employees.txt
Employees: 7
Total Payroll: 493000
Average Salary: 70428.6

# Find max and min salary
$ awk 'BEGIN { max=0; min=999999 }
      { if ($3 > max) { max=$3; maxname=$1 }
        if ($3 < min) { min=$3; minname=$1 } }
      END { print "Highest:", maxname, max
            print "Lowest: ", minname, min }' employees.txt
Highest: Charlie 92000
Lowest:  Diana 55000

# Formatted summary table
$ awk 'BEGIN {
         printf "%-12s %-15s %10s\n", "Name", "Department", "Salary"
         printf "%s\n", "---------------------------------------"
       }
      { printf "%-12s %-15s %10d\n", $1, $2, $3
        total += $3
      }
      END {
        printf "%s\n", "---------------------------------------"
        printf "%-12s %-15s %10d\n", "TOTAL", "", total
        printf "%-12s %-15s %10.0f\n", "AVERAGE", "", total/NR
      }' employees.txt
Name         Department          Salary
---------------------------------------
Alice        Engineering          85000
Bob          Marketing            60000
Charlie      Engineering          92000
Diana        HR                   55000
Eve          Engineering          78000
Frank        Marketing            67000
Grace        HR                   58000
---------------------------------------
TOTAL                            493000
AVERAGE                           70429

# Set field separator in BEGIN (instead of -F flag)
```

```
$ awk 'BEGIN { FS=":" } { print $1, $3 }' sales.txt
Jan 10
Feb 25
Mar 8
```

Listing 120: awk BEGIN and END — headers, totals, and summaries

awk if-else Statement

if-else in awk

`awk` supports full `if / else if / else` conditional logic inside action blocks. This allows you to categorize data, apply different transformations, and produce conditional output.

```
# Basic if-else
$ awk '{ if ($3 > 75000)
             print $1, "HIGH salary:", $3
         else
             print $1, "standard salary:", $3
       }' employees.txt
Alice HIGH salary: 85000
Bob standard salary: 60000
Charlie HIGH salary: 92000
Diana standard salary: 55000
Eve HIGH salary: 78000
Frank standard salary: 67000
Grace standard salary: 58000

# if - else if - else (multiple categories)
$ awk '{ if ($3 >= 90000)
             grade = "A"
         else if ($3 >= 75000)
             grade = "B"
         else if ($3 >= 60000)
             grade = "C"
         else
             grade = "D"
         printf "%-10s %6d  Grade: %s\n", $1, $3, grade
       }' employees.txt
Alice       85000  Grade: B
Bob         60000  Grade: C
Charlie     92000  Grade: A
Diana       55000  Grade: D
Eve         78000  Grade: B
Frank       67000  Grade: C
Grace       58000  Grade: D

# Nested if-else: check department AND salary
$ awk '{ if ($2 == "Engineering") {
             if ($3 > 80000)
                 print $1, "Senior Engineer"
             else
                 print $1, "Junior Engineer"
         } else {
             print $1, $2, "(non-engineering)"
         }
       }' employees.txt
Alice Senior Engineer
Bob Marketing (non-engineering)
Charlie Senior Engineer
```

```
Diana HR (non-engineering)
Eve Junior Engineer
Frank Marketing (non-engineering)
Grace HR (non-engineering)

# Ternary operator (condition ? true : false)
$ awk '{ status = ($3 > 70000) ? "above avg" : "below avg"
         print $1, status }' employees.txt
Alice above avg
Bob below avg
Charlie above avg
Diana below avg
Eve above avg
Frank below avg
Grace below avg
```

Listing 121: awk if-else — conditional logic

Built-in Variables: FS and OFS

FS and OFS

FS (*Field Separator*) controls how `awk` splits each input line into fields. **OFS** (*Output Field Separator*) is the character placed between fields when printing with `print $1, $2, ...`. Both can be set via `-v`, in BEGIN, or via `-F` for FS.

```
# FS --- Input Field Separator

# Default FS is whitespace (spaces and tabs)
$ awk '{print $1, $2}' employees.txt
Alice Engineering

# Set FS to colon (for sales.txt)
$ awk -F: '{print $1, $2, $3}' sales.txt
Jan Laptop 10

# Same using BEGIN
$ awk 'BEGIN{FS=":"} {print $1, $2, $3}' sales.txt
Jan Laptop 10

# FS can be a regex (multiple separators at once)
$ awk 'BEGIN{FS="[,;:]"} {print $1, $2}' mixed_data.txt

# FS as multiple characters (literal string)
$ awk 'BEGIN{FS="::"} {print $1}' double_colon.txt

# OFS --- Output Field Separator

# Default OFS is a single space
$ awk '{print $1, $2, $3}' employees.txt
Alice Engineering 85000

# Change OFS to comma (create CSV)
$ awk 'BEGIN{OFS=","} {print $1,$2,$3}' employees.txt
Alice,Engineering,85000
Bob,Marketing,60000

# Change OFS to pipe
$ awk 'BEGIN{OFS="|"} {print $1,$2,$3,$4,$5}' employees.txt
Alice|Engineering|85000|F|Hooghly

```

```
# OFS used with field reassignment (triggers rebuild of $0)
$ awk 'BEGIN{OFS="-"} {$1=$1; print}' employees.txt
Alice-Engineering-85000-F-Hooghly
Bob-Marketing-60000-M-Kolkata

# Convert whitespace-delimited to CSV
$ awk 'BEGIN{OFS=","} {$1=$1; print}' employees.txt > employees.csv

# NF (Number of Fields) and NR (Number of Records)
$ awk '{print NR, NF, $1}' employees.txt
1 5 Alice
2 5 Bob
3 5 Charlie

# RS --- Record Separator (default: newline)
# Change RS to read paragraph blocks
$ awk 'BEGIN{RS=""; FS="\n"} {print NR, $1}' paragraphs.txt
```

Listing 122: FS and OFS — field separators

Variable	Meaning	Default
FS	Input field separator	Whitespace
OFS	Output field separator	Single space
RS	Input record separator	Newline
ORS	Output record separator	Newline
NF	Number of fields in current record	(varies)
NR	Current record number (global)	(varies)
FNR	Record number within current file	(varies)
FILENAME	Name of current input file	(varies)

awk String and Arithmetic Functions

Built-in String Functions

```
# length(s) --- length of string (or $0 if no arg)
$ awk '{print $1, length($1)}' employees.txt
Alice 5
Bob 3
Charlie 7

# substr(s, start, len) --- extract substring
# Positions start from 1 in awk
$ awk '{print substr($1, 1, 3)}' employees.txt   # first 3 chars
Ali
Bob
Cha

# toupper(s) and tolower(s)
$ awk '{print toupper($1), tolower($2)}' employees.txt
ALICE engineering
BOB marketing
CHARLIE engineering

# index(s, target) --- position of target in s (0 if not found)
$ awk '{pos = index($1, "li")
```

```
        if (pos > 0) print $1, "contains li at pos", pos
       }' employees.txt
Alice contains li at pos 3

# split(s, array, sep) --- split string into array
$ awk 'BEGIN {
         n = split("Jan:Feb:Mar:Apr", months, ":")
         for (i=1; i<=n; i++) print i, months[i]
       }'
1 Jan
2 Feb
3 Mar
4 Apr

# sub(regex, replacement, target) --- replace FIRST match
$ awk '{sub(/Engineering/, "Eng"); print}' employees.txt
Alice   Eng  85000  F  Hooghly
Bob     Marketing  60000  M  Kolkata

# gsub(regex, replacement, target) --- replace ALL matches
$ awk '{gsub(/o/, "0"); print $1}' employees.txt
Alice
B0b
Charlie
Diana
Eve
Frank
Grace

# match(s, regex) --- find regex in string; returns position
$ awk '{if (match($1, /^[AEI]/)) print $1, "starts with vowel"}' employees.txt
Alice starts with vowel
Eve starts with vowel

# sprintf --- format string (like printf but returns a string)
$ awk '{padded = sprintf("%-10s", $1); print padded "|" $2}' employees.txt
Alice      |Engineering
Bob        |Marketing
```

Listing 123: awk string functions

Built-in Arithmetic Functions

```
# sin(x), cos(x), atan2(y,x) --- trigonometry (in radians)
$ awk 'BEGIN { print sin(3.14159/2) }'    # sin(90 deg)
1

$ awk 'BEGIN { pi = atan2(0,-1); print pi }'
3.14159

# exp(x) --- e to the power x
$ awk 'BEGIN { print exp(1) }'
2.71828

# log(x) --- natural logarithm
$ awk 'BEGIN { print log(2.71828) }'
1

# sqrt(x) --- square root
$ awk 'BEGIN { print sqrt(144) }'
12

$ awk '{print $1, sqrt($3)}' employees.txt
```

```
Alice 291.548
Bob 244.949

# int(x) --- truncate to integer (remove decimal)
$ awk 'BEGIN { print int(3.99) }'
3
$ awk 'BEGIN { print int(-3.7) }'
-3

# rand() --- random number between 0 and 1
$ awk 'BEGIN { srand(); for(i=1;i<=3;i++) printf "%.4f\n", rand() }'
0.3842
0.7156
0.1293

# srand(seed) --- seed the random number generator
$ awk 'BEGIN { srand(42); print rand() }'

# Practical: calculate statistics
$ awk 'BEGIN { sum=0; sumsq=0 }
      { sum += $3; sumsq += $3^2 }
      END {
        mean = sum / NR
        variance = sumsq/NR - mean^2
        stddev = sqrt(variance)
        printf "Mean:   %.2f\n", mean
        printf "StdDev: %.2f\n", stddev
      }' employees.txt
Mean:   70428.57
StdDev: 13289.55
```

Listing 124: awk arithmetic functions

Loops in awk

awk supports three types of loops: for, while, and do-while.

for Loop

```
# C-style for loop
# Print each field on a separate line
$ awk '{ for (i=1; i<=NF; i++) print i, $i }' employees.txt
1 Alice
2 Engineering
3 85000
4 F
5 Hooghly
1 Bob
2 Marketing
...

# Use for loop to build a multiplication table
$ awk 'BEGIN {
         for (i=1; i<=5; i++) {
             for (j=1; j<=5; j++)
                 printf "%4d", i*j
             print ""
         }
       }'
   1   2   3   4   5
   2   4   6   8  10
   3   6   9  12  15
   4   8  12  16  20
```

```
   5  10  15  20  25

# for-in loop --- iterate over associative array keys
$ awk '{ dept_count[$2]++ }
      END {
          for (dept in dept_count)
              printf "%-15s %d employees\n", dept, dept_count[dept]
      }' employees.txt
Engineering     3 employees
Marketing       2 employees
HR              2 employees
```

Listing 125: awk for loop — C-style and array iteration

while and do-while Loops

```
# while loop
$ awk 'BEGIN {
         n = 1
         while (n <= 5) {
             print n, n*n
             n++
         }
       }'
1 1
2 4
3 9
4 16
5 25

# do-while loop (executes at least once)
$ awk 'BEGIN {
         n = 1
         do {
             printf "%d ", n
             n++
         } while (n <= 10)
         print ""
       }'
1 2 3 4 5 6 7 8 9 10

# Practical: reverse the fields of each line
$ awk '{ for (i=NF; i>=1; i--) printf "%s%s", $i, (i>1 ? " " : "\n") }'
    employees.txt
Hooghly F 85000 Engineering Alice
Kolkata M 60000 Marketing Bob

# Loop with break and continue
$ awk '{ for (i=1; i<=NF; i++) {
             if ($i ~ /^[0-9]+$/) continue  # skip numeric fields
             printf "%s ", $i
         }
         print ""
       }' employees.txt
Alice Engineering F Hooghly
Bob Marketing M Kolkata

# Accumulate per-department salary using array
$ awk '{ dept_salary[$2] += $3
         dept_count[$2]++
       }
      END {
          printf "%-15s %10s %10s\n", "Department", "Total", "Average"
```

```
            for (d in dept_salary)
                printf "%-15s %10d %10.0f\n", d, dept_salary[d],
                                                dept_salary[d]/dept_count[d]
        }' employees.txt
Engineering      255000       85000
Marketing        127000       63500
HR               113000       56500
```

Listing 126: awk while and do-while loops

Search and Substitute Functions in awk

sub() and gsub() — Substitution

```
# sub(regex, replacement [, target])
# Replaces the FIRST occurrence of regex in target
# If target is omitted, $0 (whole line) is used

# Replace first vowel in the name with '*'
$ awk '{sub(/[aeiouAEIOU]/, "*", $1); print $1}' employees.txt
*lice
B*b
Ch*rlie
D*ana
*ve
Fr*nk
Gr*ce

# gsub(regex, replacement [, target])
# Replaces ALL occurrences

# Replace ALL vowels in name with '*'
$ awk '{gsub(/[aeiouAEIOU]/, "*", $1); print $1}' employees.txt
*l*c*
B*b
Ch*rl**
D**n*
*v*
Fr*nk
Gr*c*

# Replace department abbreviations
$ awk '{gsub(/Engineering/, "Eng"); gsub(/Marketing/, "Mkt"); print}'
    employees.txt
Alice   Eng  85000  F  Hooghly
Bob     Mkt  60000  M  Kolkata

# Remove all digits from a field
$ awk '{gsub(/[0-9]/, "", $3); print $1, $3}' employees.txt
Alice
Bob

# Count replacements (gsub returns the number of replacements)
$ awk '{ n = gsub(/o/, "0")
         print $1, "replacements:", n }' employees.txt
Alice replacements: 0
Bob replacements: 1
Charlie replacements: 0

# Use & in replacement to refer to the matched text
# Wrap matched word in square brackets
$ awk '{gsub(/[0-9]+/, "[&]"); print}' employees.txt
```

```
Alice    Engineering  [85000]  F  Hooghly
Bob      Marketing    [60000]  M  Kolkata
```

Listing 127: awk sub and gsub — search and replace

match() — Searching within Fields

```
# match(string, regex)
# Returns the POSITION (1-based) of first match, or 0 if no match
# Sets RSTART (start position) and RLENGTH (length of match)

# Find names that contain exactly 5 characters
$ awk '{ if (match($1, /^.{5}$/)) print $1, "is 5 chars" }' employees.txt
Alice is 5 chars
Grace is 5 chars
Frank is 5 chars

# Extract matched portion using RSTART and RLENGTH
$ awk '{ if (match($0, /[0-9]+/))
             print $1, "salary digits start at pos", RSTART,
                   "length", RLENGTH,
                   "value:", substr($0, RSTART, RLENGTH)
       }' employees.txt
Alice salary digits start at pos 21 length 5 value: 85000

# Practical: extract only the numeric salary from each line
$ awk '{ match($0, /[0-9]+/); print $1, substr($0, RSTART, RLENGTH) }'
    employees.txt
Alice 85000
Bob 60000
Charlie 92000
```

Listing 128: awk match() — finding patterns in strings

Complete Practical Pipeline Example

```
# Task: Find all Engineering employees in Hooghly,
#       calculate 15% raise, and produce a formatted report

$ grep "Engineering" employees.txt \
  | awk '$5 == "Hooghly" {
             new_salary = $3 * 1.15
             printf "%-12s Current: %6d  After 15%% raise: %6.0f\n",
                    $1, $3, new_salary
             total_old += $3
             total_new += new_salary
             count++
         }
         END {
             print "-------------------------------------------"
             printf "%-12s Current: %6d  After 15%% raise: %6.0f\n",
                    "TOTAL", total_old, total_new
             printf "Extra payroll cost: %.0f\n", total_new - total_old
         }'

Alice        Current:  85000  After 15% raise:  97750
Charlie      Current:  92000  After 15% raise: 105800
Eve          Current:  78000  After 15% raise:  89700
-------------------------------------------
TOTAL        Current: 255000  After 15% raise: 293250
Extra payroll cost: 38250

# Another pipeline: department-wise summary with grep + sort + awk
```

```
$ awk '{print $2, $3}' employees.txt \
  | sort \
  | awk '{ dept=$1; sal=$2
            dept_total[dept] += sal
            dept_count[dept]++
          }
          END {
              for (d in dept_total)
                  printf "%-15s Avg: %6.0f\n", d, dept_total[d]/dept_count[d]
          }' \
  | sort
Engineering     Avg:  85000
HR              Avg:  56500
Marketing       Avg:  63500
```

Listing 129: Complete grep + awk pipeline — real-world data processing

Unit 6 Summary

Key Takeaways — Unit 6: Advanced File Searching with GREP and AWK

1. **grep** searches files line by line for patterns; `-i` ignores case, `-v` inverts, `-n` shows line numbers, `-c` counts, `-r` recurses, `-E` enables extended regex, `-A/-B/-C` show context.
2. **awk** is a pattern-action language; fields are `$1..$NF`, `$0` is the whole line, `NR` is the line number, `NF` is the field count.
3. **print** outputs fields separated by OFS; **printf** gives C-style formatted output with `%s %d %f %-Ns` for alignment and padding.
4. **Comparison operators** (`== != > >= < <=`) and regex operators (`~` and `!~`) are used as patterns to filter rows; `&&` and `||` combine conditions.
5. **Arithmetic operators** `+ - * / % ^` work on fields; variables accumulate values across lines; `+=` etc. are supported.
6. **BEGIN** runs once before input (initialize variables, print headers); **END** runs once after all input (print totals and summaries).
7. **if / else if / else** provides conditional logic inside action blocks; the ternary `condition ? a : b` is also supported.
8. **FS** sets the input field delimiter (or use `-F`); **OFS** sets the output delimiter; reassigning `$1=$1` forces `$0` to be rebuilt with OFS.
9. **String functions**: `length()`, `substr()`, `toupper()`, `tolower()`, `index()`, `split()`, `sprintf()`.
10. **Arithmetic functions**: `sqrt()`, `int()`, `log()`, `exp()`, `sin()`, `cos()`, `rand()`, `srand()`.
11. **Loops**: C-style `for`, `for (k in array)`, `while`, and `do-while` — arrays in awk are associative and perfect for grouping/counting.
12. **sub()** replaces the first regex match; **gsub()** replaces all matches; **match()** finds the position and sets `RSTART/RLENGTH`; `&` in replacement refers to the matched text.

End of Unit 6 — Advanced File Searching with GREP and AWK

Proceed to Unit 7: Processes in UNIX

Unit 6 — Practice Questions

Practice Questions — Unit 6: GREP and AWK

A. Short Answer (2 marks each)

1. What does `grep -v "error" log.txt` do? What flag makes it case-insensitive?
2. Explain the difference between BRE and ERE in `grep`. Which flag enables ERE?
3. What does `grep -r "TODO" .` do? What does adding `-l` change?
4. In `awk`, what is the difference between the `BEGIN` and `END` blocks?
5. What does `grep -c "ERROR" syslog` return? How does it differ from `grep "ERROR" | wc -l`?
6. Explain `awk '$3 > 90 {print $1, $3}' scores.txt`.
7. What is a regex character class? Give examples: `[a-z]`, `[0-9]`, `[^aeiou]`.
8. What does the regex `^root` mean when used in `grep`?

B. Write the Command (2 marks each)

1. Write a `grep` command to find all lines in `/etc/passwd` that start with a letter (not a comment).
2. Write an `awk` command to print the total and average of column 2 from `scores.txt`.
3. Write a `grep -E` command to match lines containing either "cat" or "dog".
4. Write an `awk` command that prints lines where field 4 equals "Hooghly" from colon-delimited `students.txt`.

C. Long Answer (5 marks each)

1. Explain `grep` in detail. Cover: basic syntax, 8 important flags (`-i -v -r -l -n -c -E -w`), and one example for each.
2. Explain `awk` programming: pattern-action structure, built-in variables, field manipulation, arithmetic, and `BEGIN/END` blocks. Give a complete example that reads a CSV, filters rows, and prints a summary.

Unit 7: Processes in UNIX

Unit 7 — Processes in UNIX

Process Concepts · ps command · init & Login · fork() getpid() wait() · Zombie · pipe() & message() · Process States · Scheduler · Swapped Memory · vmstat · top · nice

A **process** is the fundamental unit of execution in UNIX. Understanding how the kernel creates, schedules, and manages processes is essential for writing efficient programs, diagnosing system performance problems, and working effectively as a system administrator.

What is a Process in UNIX?

Definition — Process

A **process** is a **running instance of a program**. When you execute a command, the kernel creates a new process that has its own: Process ID (PID), memory space, open file descriptors, environment variables, signal handlers, and CPU register state. A program is a static file on disk; a process is that program actively executing in memory.

Process vs Program

Aspect	Program	Process
Nature	Passive — a file on disk	Active — executing in memory
Storage	Stored in filesystem (e.g. `/bin/ls`)	Lives in RAM
Identity	Identified by filename	Identified by PID
Count	One binary file	Can spawn many simultaneous processes
Lifetime	Permanent (until deleted)	Temporary (created and destroyed)
Resources	Consumes no CPU/RAM when not run	Owns CPU time, memory, file descriptors

What Every Process Contains

Every UNIX process has its own address space divided into segments:

Segment	Contents
Text (Code)	Compiled machine instructions (read-only, shared)
Data	Initialized global and static variables
BSS	Uninitialized global variables (zero-filled)
Heap	Dynamically allocated memory (`malloc`)
Stack	Function call frames, local variables, return addresses

Each process also has a **Process Control Block (PCB)** maintained by the kernel, containing: PID, PPID, UID, GID, process state, CPU registers, open file table, signal disposition, scheduling priority, and memory maps.

The ps Command — Process Status

Definition — ps

`ps` (*process status*) displays a snapshot of currently running processes. It reads from the `/proc` virtual filesystem. Unlike `top`, it shows a one-time snapshot rather than a live view.

Basic ps Usage

```
# Show processes for the CURRENT terminal only (minimal output)
$ ps
  PID TTY          TIME CMD
 1234 pts/0    00:00:00 bash
 1589 pts/0    00:00:00 ps

# Fields: PID = Process ID, TTY = terminal, TIME = CPU used, CMD = command

# Show ALL processes on the system (BSD style --- most common)
$ ps aux
USER       PID %CPU %MEM    VSZ   RSS TTY    STAT START   TIME COMMAND
root         1  0.0  0.1 167028 11936 ?      Ss   10:00   0:02 /sbin/init
root         2  0.0  0.0      0     0 ?      S    10:00   0:00 [kthreadd]
student   1234  0.1  0.2  22344  9216 pts/0 Ss   10:05   0:00 bash
student   1589  0.0  0.0   9876  1024 pts/0 R+   10:30   0:00 ps aux

# Show ALL processes (UNIX style)
$ ps -ef
UID        PID  PPID  C STIME TTY          TIME CMD
root         1     0  0 10:00 ?        00:00:02 /sbin/init
root         2     1  0 10:00 ?        00:00:00 [kthreadd]
student   1234   850  0 10:05 pts/0    00:00:00 bash
student   1589  1234  0 10:30 pts/0    00:00:00 ps -ef
```

Listing 130: ps — basic process listing

ps Column Meanings

Column	Name	Meaning
PID	Process ID	Unique identifier assigned by the kernel
PPID	Parent Process ID	PID of the process that created this one
USER	User	Username of the process owner
UID	User ID	Numeric user ID
%CPU	CPU Usage	Percentage of CPU time used
%MEM	Memory Usage	Percentage of physical RAM used
VSZ	Virtual Size	Total virtual memory size in KB
RSS	Resident Set Size	Physical RAM currently in use (KB)
TTY	Terminal	Controlling terminal (? = none/daemon)
STAT	Process State	Current state code (see process states)
START	Start Time	When the process was started
TIME	CPU Time	Total CPU time consumed
CMD	Command	Command name and arguments

ps with Different Options

```
# Show processes for a specific user
$ ps -u student
$ ps aux | grep student

# Show process tree (parent-child hierarchy)
$ ps -ejH
$ ps axjf         # ASCII art tree

# Or use pstree (cleaner)
$ pstree
systemd---sshd---bash---ps
       |--cron
       |--nginx---nginx
                --nginx

# Show specific process by PID
$ ps -p 1234
  PID TTY      STAT   TIME COMMAND
 1234 pts/0    Ss     0:00 bash

# Show full command line (not truncated)
$ ps auxww

# Sort by CPU usage (highest first)
$ ps aux --sort=-%cpu | head -10

# Sort by memory usage (highest first)
$ ps aux --sort=-%mem | head -10

# Show only PID and command (custom output with -o)
$ ps -eo pid,ppid,stat,cmd
  PID  PPID STAT CMD
    1     0 Ss   /sbin/init
```

```
     2      1 S    [kthreadd]
  1234    850 Ss   bash

# Show process with threads
$ ps -eLf | head -10

# Show processes of a specific program
$ ps aux | grep nginx
$ pgrep -a nginx        # cleaner version

# Kill a process by name
$ pkill firefox
$ kill -9 $(pgrep nginx)
```

Listing 131: ps — common option combinations

Command	What it shows
`ps`	Current terminal processes only
`ps aux`	ALL processes, BSD style (most used)
`ps -ef`	ALL processes, UNIX style with PPID
`ps axjf`	Process tree with ASCII art
`ps -u user`	Processes owned by a specific user
`ps -p PID`	Details of a single process
`ps -eo ...`	Custom column output

The `init` Process and the UNIX Login Process

The `init` Process (PID 1)

Definition — init

init is the **first process started by the kernel** after the system boots. It always has **PID 1** and is the **ancestor of all other processes** on the system. It never dies — if `init` exits, the kernel panics. On modern Linux systems, `init` is replaced by **systemd**, but the role is identical.

```
# init always has PID 1
$ ps -p 1
  PID TTY      STAT   TIME COMMAND
    1 ?        Ss     0:02 /sbin/init (or /lib/systemd/systemd)

# See the full process ancestry chain
$ pstree -p | head -20
systemd(1)---agetty(456)
           --cron(512)
           --dbus-daemon(487)
           --login(520)---bash(534)---pstree(601)
           --nginx(612)---nginx(613)
                        ---nginx(614)
           --sshd(488)---sshd(545)---bash(547)

# init's responsibilities:
# 1. Start all system services (daemons)
# 2. Manage run levels / targets
# 3. Respawn processes that die unexpectedly
```

```
# 4. Adopt orphaned processes (reparent them to PID 1)
# 5. Handle system shutdown and reboot
```

Listing 132: Inspecting init / systemd

The UNIX Login Process

When a user logs into a UNIX terminal, a precise sequence of processes is created:

Step	Process	What Happens
1	`init / systemd`	Starts `getty` or `agetty` on each terminal
2	`getty`	Opens the terminal device, displays login prompt
3	`login`	Reads username; `getty` execs `login`
4	`login`	Reads password; verifies against `/etc/shadow`
5	`login`	Sets UID, GID, environment (`HOME`, `SHELL`, `PATH`)
6	`shell`	`login` execs the user's shell (e.g. `/bin/bash`)
7	`shell`	Shell reads `.bashrc` / `.profile`, displays prompt
8	User commands	Each command typed creates a child process via `fork()`

SSH Login

For SSH logins, `sshd` (SSH daemon) replaces `getty/login`. After authenticating the user, `sshd` forks a child that execs the user's shell — exactly the same from step 6 onwards.

Process Creation: `fork()`, `getpid()`, `getppid()`, `wait()`

`fork()` — Creating a New Process

Definition — fork()

`fork()` is the UNIX system call for creating a new process. It creates an **exact copy** (clone) of the calling process. The original is the **parent**; the copy is the **child**. After `fork()`, both processes run concurrently, each starting from the instruction after the `fork()` call.

Return Value	Who Receives It	Meaning
`> 0` (child's PID)	Parent process	Fork succeeded; value = child's PID
`0`	Child process	I am the child process
`-1`	Parent process	Fork failed (e.g. too many processes)

getpid() and getppid()

Definition — getpid() and getppid()

getpid() returns the **PID of the calling process**. getppid() returns the **PID of the calling process's parent**. Both are always successful (no error return).

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>    /* fork(), getpid(), getppid() */
#include <sys/wait.h>  /* wait() */

int main(void) {
    pid_t pid;

    printf("Before fork: PID = %d\n", getpid());

    pid = fork();    /* --- FORK POINT --- */

    if (pid < 0) {
        /* fork() failed */
        perror("fork failed");
        exit(EXIT_FAILURE);

    } else if (pid == 0) {
        /* ---- CHILD PROCESS ---- */
        printf("CHILD:  My PID    = %d\n", getpid());
        printf("CHILD:  My Parent = %d\n", getppid());
        printf("CHILD:  Doing child work...\n");
        exit(0);    /* child exits here */

    } else {
        /* ---- PARENT PROCESS ---- */
        printf("PARENT: My PID      = %d\n", getpid());
        printf("PARENT: Child's PID = %d\n", pid);
        printf("PARENT: Waiting for child...\n");
        wait(NULL); /* wait for child to finish */
        printf("PARENT: Child has finished. Exiting.\n");
    }

    return 0;
}
```

Listing 133: fork(), getpid(), getppid() — full working example

```
Before fork: PID = 4521
PARENT: My PID      = 4521
PARENT: Child's PID = 4522
PARENT: Waiting for child...
CHILD:  My PID    = 4522
CHILD:  My Parent = 4521
CHILD:  Doing child work...
PARENT: Child has finished. Exiting.
```

Listing 134: Output of fork example

wait() — Waiting for Child Processes

Definition — wait()

`wait()` causes the **parent process to block (sleep) until one of its children terminates**. It also collects the child's exit status, which prevents the child from becoming a zombie. `waitpid(pid, &status, options)` waits for a *specific* child.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    pid_t pid;
    int status;

    pid = fork();

    if (pid == 0) {
        /* Child */
        printf("Child (PID %d) running\n", getpid());
        sleep(2);           /* simulate work */
        exit(42);           /* exit with code 42 */

    } else {
        /* Parent: wait and retrieve child's exit code */
        pid_t finished = wait(&status);

        if (WIFEXITED(status)) {
            printf("Child PID %d exited normally\n", finished);
            printf("Exit code: %d\n", WEXITSTATUS(status));
        } else if (WIFSIGNALED(status)) {
            printf("Child killed by signal %d\n", WTERMSIG(status));
        }
    }
    return 0;
}
```

Listing 135: wait() and exit status handling

Zombie Processes

Definition — Zombie Process

A **zombie process** (also called a *defunct process*) is a process that has **finished executing but still has an entry in the process table**. This happens when the child exits but the parent has not yet called `wait()` to collect its exit status. The zombie holds no memory or CPU — it is just a record waiting to be acknowledged.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void) {
    pid_t pid = fork();

    if (pid == 0) {
        /* Child exits immediately */
        printf("Child (PID %d) exiting now\n", getpid());
```

```c
        exit(0);
    } else {
        /* Parent does NOT call wait() --- child becomes zombie */
        printf("Parent (PID %d) sleeping --- child is now zombie\n",
               getpid());
        sleep(30);  /* zombie is visible in ps during this sleep */
    }
    return 0;
}
```

Listing 136: Creating and observing a zombie process

```bash
# Run the program above, then in another terminal:
$ ps aux | grep defunct
student   4522  0.0  0.0      0     0 pts/0  Z+   10:30  0:00 [zombie] <defunct>
#                                                          ^ Z = Zombie state

# Zombies cannot be killed with kill -9 (they are already dead!)
# They disappear when the PARENT calls wait() or when the parent exits.

# Check for zombies on the system
$ ps aux | awk '$8=="Z" {print $0}'

# If parent is dead, zombies are adopted by init (PID 1)
# init always calls wait() so adopted zombies are cleaned up quickly
```

Listing 137: Observing and dealing with zombies

Term	Meaning
Zombie	Child has exited but parent hasn't called `wait()` yet
Orphan	Parent has exited but child is still running; adopted by `init`
Daemon	Background process with no controlling terminal (`TTY = ?`)

`pipe()` and Message Passing

`pipe()` — Inter-Process Communication

Definition — pipe()

A **pipe** is a unidirectional communication channel between two related processes (typically parent and child). `pipe()` creates a pipe and returns two file descriptors: `fd[0]` for reading and `fd[1]` for writing. Data written to the write end can be read from the read end.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>

int main(void) {
    int fd[2];          /* fd[0]=read end, fd[1]=write end */
    pid_t pid;
    char message[] = "Hello from parent!";
    char buffer[100];

    /* Create the pipe BEFORE fork */
```

```c
    if (pipe(fd) == -1) {
        perror("pipe");
        exit(1);
    }

    pid = fork();

    if (pid == 0) {
        /* ---- CHILD: reads from pipe ---- */
        close(fd[1]);               /* close unused write end */
        read(fd[0], buffer, sizeof(buffer));
        printf("Child received: %s\n", buffer);
        close(fd[0]);

    } else {
        /* ---- PARENT: writes to pipe ---- */
        close(fd[0]);               /* close unused read end */
        write(fd[1], message, strlen(message) + 1);
        close(fd[1]);
        wait(NULL);
    }
    return 0;
}
```

Listing 138: pipe() — parent sends data to child

```
Child received: Hello from parent!
```

Listing 139: Output of pipe example

```bash
# Every shell pipe | uses the pipe() system call internally
$ ls /etc | grep "conf" | wc -l
# Kernel creates two pipes:
#   ls  ---pipe1---> grep ---pipe2---> wc
# Each | is one call to pipe() + two fork() calls

# Named pipes (FIFOs) work across unrelated processes
$ mkfifo mypipe
$ echo "data" > mypipe &     # writer in background
$ cat mypipe                 # reader --- blocks until writer sends
data
```

Listing 140: Shell pipes are pipe() under the hood

Message Queues

Definition — Message Queue

A **message queue** is a kernel-maintained linked list of messages. Unlike pipes, messages are typed (have a message type number), are not strictly FIFO (receivers can select by type), and persist until explicitly deleted. They are part of **System V IPC**.

```c
#include <sys/ipc.h>
#include <sys/msg.h>

/* Message structure */
struct msgbuf {
    long mtype;         /* message type (must be > 0) */
    char mtext[100];    /* message data */
};

```

```c
/* ---- SENDER ---- */
int main_sender(void) {
    key_t key = ftok(".", 'M');       /* generate unique key */
    int msgid = msgget(key, IPC_CREAT | 0666);  /* create queue */

    struct msgbuf msg;
    msg.mtype = 1;
    snprintf(msg.mtext, sizeof(msg.mtext), "Hello via message queue!");
    msgsnd(msgid, &msg, sizeof(msg.mtext), 0);  /* send */
    return 0;
}

/* ---- RECEIVER ---- */
int main_receiver(void) {
    key_t key = ftok(".", 'M');
    int msgid = msgget(key, 0666);

    struct msgbuf msg;
    msgrcv(msgid, &msg, sizeof(msg.mtext), 1, 0);  /* receive type 1 */
    printf("Received: %s\n", msg.mtext);

    msgctl(msgid, IPC_RMID, NULL);   /* delete the queue */
    return 0;
}
```

Listing 141: Message queue — sender and receiver overview

IPC Method	Direction	Best For
Unnamed pipe	Unidirectional	Parent–child communication
Named pipe (FIFO)	Unidirectional	Unrelated processes, persistent
Message queue	Bidirectional	Typed messages, selective receive
Shared memory	Bidirectional	Fastest IPC; large data exchange
Socket	Bidirectional	Network or local IPC, any process

UNIX Process States and Diagram

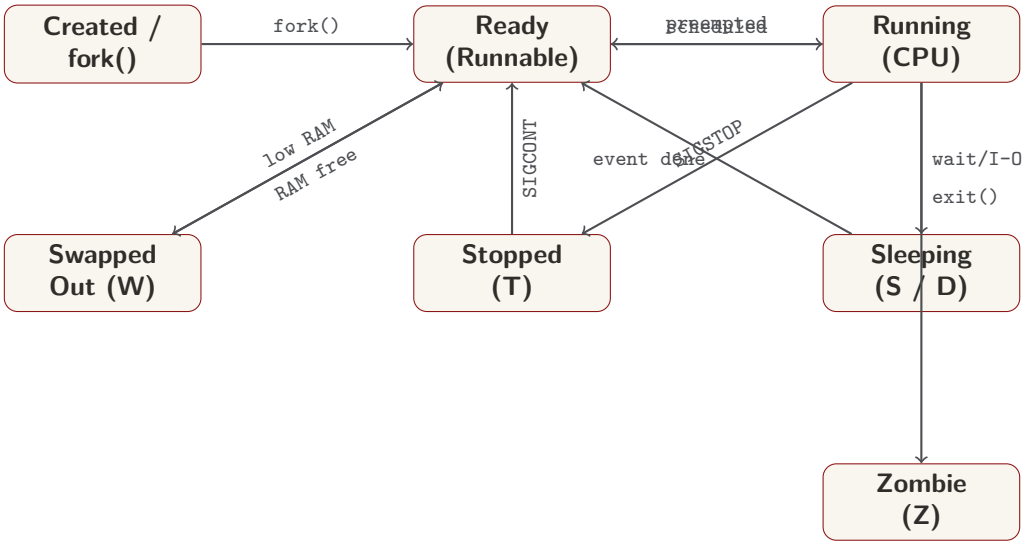
Process States

Every process in UNIX is in exactly one state at any given moment. The kernel's scheduler moves processes between states as they request I/O, receive CPU time, or get suspended.

The Seven Process States

State	Symbol	Description
Running (User)	`R`	Executing in user space right now
Running (Kernel)	`R`	Executing inside a system call
Sleeping (Inter.)	`S`	Waiting for an event; can be interrupted by a signal
Sleeping (Unintr.)	`D`	Waiting for I/O; cannot be interrupted (e.g. disk read)
Stopped	`T`	Paused by `SIGSTOP` or job control (`Ctrl+Z`)
Zombie	`Z`	Finished but parent hasn't called `wait()`
Swapped Out	`W`	Moved to disk swap space (low memory condition)

Process State Transition Diagram



Reading Process State in `ps`

The `STAT` column in `ps aux` uses these codes plus modifiers:

STAT Code	Meaning
`R`	Running or runnable (on run queue)
`S`	Interruptible sleep (waiting for event)
`D`	Uninterruptible sleep (usually I/O)
`T`	Stopped (by signal or job control)
`Z`	Zombie (waiting for parent to call wait)
`s`	Session leader
`l`	Multi-threaded
`+`	In the foreground process group
`<`	High priority (nice value < 0)
`N`	Low priority (nice value > 0)

The UNIX Process Scheduler

Definition — Scheduler

The **scheduler** is the part of the kernel that decides **which process runs next on the CPU**. It runs whenever a process blocks, yields, its time quantum expires, or a higher-priority process becomes runnable. The goal is to maximise CPU utilisation, minimise response time, and ensure fairness.

Traditional UNIX Scheduler

The original UNIX scheduler (System V and BSD) used a **multi-level feedback queue** with the following key ideas:

1. **Time Slicing** — Each process gets a fixed *time quantum* (typically 100ms). When the quantum expires the process is preempted and moved back to the run queue.
2. **Priority-Based** — Lower priority number = higher priority. The scheduler always picks the highest-priority runnable process.
3. **Dynamic Priority** — Priority is recalculated periodically. CPU-bound processes (use lots of CPU) get their priority lowered. I/O-bound processes (sleep a lot) get priority raised. This prevents CPU-hungry processes from starving interactive ones.
4. **Nice Value** — Users can *voluntarily* lower their own process's priority using the `nice` command (higher nice = lower priority = be nicer to others).

Linux Completely Fair Scheduler (CFS)

Modern Linux uses the **Completely Fair Scheduler (CFS)**, introduced in kernel 2.6.23:

Feature	Description
Virtual Runtime	Each process has a `vruntime` — the total CPU time it has consumed, weighted by its priority. CFS always runs the process with the *lowest* `vruntime`.
Red-Black Tree	Runnable processes are stored in a self-balancing red-black tree ordered by `vruntime`. The leftmost node (lowest vruntime) is always next to run — O(log n) scheduling.
Fairness	Over time, every process gets a proportional share of the CPU. A process with nice 0 gets twice the CPU of a process with nice 5.
No Fixed Quantum	CFS uses a *target latency* (typically 6ms) divided among all runnable processes, not a fixed time slice.

```
# See scheduler policy and priority of a process
$ chrt -p 1234
pid 1234's current scheduling policy: SCHED_OTHER
pid 1234's current scheduling priority: 0

# Available scheduling policies:
# SCHED_OTHER   -- default CFS (nice-based, time-sharing)
# SCHED_FIFO    -- real-time, first-in first-out (no preemption)
# SCHED_RR      -- real-time, round-robin with time slices
# SCHED_BATCH   -- for batch workloads (lower priority)
# SCHED_IDLE    -- extremely low priority background tasks

# See nice value (PR = kernel priority, NI = nice value)
$ ps -eo pid,pri,ni,cmd | head -10
  PID PRI  NI CMD
    1  19   0 /sbin/init
  123  19   0 nginx
  456  10   9 backup_job    # nice +9, lower priority

# Real-time priority for a time-sensitive process
$ sudo chrt -f -p 50 1234   # set FIFO priority 50
```

Listing 142: Viewing scheduler information

Swapped Memory

Definition — Swap

Swap space (or swapped memory) is a reserved area of disk that the kernel uses as an **extension of physical RAM**. When RAM is nearly full, the kernel's *memory manager* moves the least-recently-used memory pages of inactive processes from RAM to swap disk — this is called **swapping out** (or *paging out*). When those pages are needed again, they are **swapped in** (paged back into RAM).

How Swapping Works

Step	What Happens
1. RAM fills up	Physical RAM approaches capacity
2. kswapd wakes	Kernel thread `kswapd` identifies least-used pages
3. Pages swapped out	Selected memory pages written to swap partition/file
4. RAM freed	The freed RAM is given to the process that needs it
5. Page fault	If a swapped-out page is accessed, a page fault occurs
6. Pages swapped in	The page is read back from swap into RAM (slow!)

```
# Show current memory and swap usage
$ free -h
                total    used    free  shared  buff/cache  available
Mem:             15Gi   4.2Gi   8.1Gi   512Mi       2.7Gi       10Gi
Swap:             2Gi   128Mi   1.9Gi

# Show swap partitions and files
$ swapon --show
NAME       TYPE  SIZE  USED PRIO
/dev/sda2  part  2G    128M   -2
/swapfile  file  1G      0B   -3

# Show swap usage per process (in KB)
$ cat /proc/*/status 2>/dev/null | grep -E "Name|VmSwap" \
  | paste - - | sort -k4 -rn | head -10

# Create a swap file (if more swap is needed)
$ sudo fallocate -l 2G /swapfile2
$ sudo chmod 600 /swapfile2
$ sudo mkswap /swapfile2
$ sudo swapon /swapfile2

# Disable swap (bring all pages back to RAM)
$ sudo swapoff -a

# Control swappiness (0 = avoid swap, 100 = swap aggressively)
$ cat /proc/sys/vm/swappiness
60
$ echo 20 | sudo tee /proc/sys/vm/swappiness   # less aggressive
```

Listing 143: Viewing and managing swap space

Thrashing

When the system is constantly swapping — writing and reading pages from disk in a tight loop — it is called **thrashing**. The system becomes nearly unusable because disk I/O is orders of magnitude slower than RAM. The solution is to add more RAM or reduce the number of running processes.

The `vmstat` Command

Definition — vmstat

`vmstat` (*virtual memory statistics*) reports on **processes, memory, swap, I/O, and CPU activity**. It gives a system-wide view and is the first tool to reach for when diagnosing performance problems. It can display a one-time snapshot or continuous updates at a specified interval.

```
# Basic syntax
vmstat [options] [delay [count]]

# One-time snapshot
$ vmstat
procs -----------memory---------- ---swap-- -----io---- -system-- ------cpu-----
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st
 1  0 131072 8289280 346200 5213056    0    0    15     8  450  820  5  2 91  2  0

# Continuous output every 2 seconds, 5 times
$ vmstat 2 5

# Human-readable sizes (-S M for megabytes)
$ vmstat -S M

# Show disk I/O statistics (-d flag)
$ vmstat -d

# Show active/inactive memory stats (-a flag)
$ vmstat -a
```

Listing 144: vmstat — reading memory and CPU statistics

Understanding vmstat Output

Section	Field	Meaning
procs	r	Processes waiting to run (run queue length)
	b	Processes in uninterruptible sleep (blocked on I/O)
memory	swpd	Virtual memory used (swap in use, KB)
	free	Idle (unused) physical memory
	buff	Memory used as I/O buffers
	cache	Memory used as page cache
swap	si	Swap-in: KB per second read from swap back to RAM
	so	Swap-out: KB per second written from RAM to swap
io	bi	Blocks received from block device (disk reads)
	bo	Blocks sent to block device (disk writes)
system	in	Interrupts per second
	cs	Context switches per second
cpu	us	Time in user space (%)
	sy	Time in kernel (system) space (%)
	id	Idle CPU time (%)
	wa	Time waiting for I/O (%)
	st	Time stolen from VM by hypervisor (%)

```
# System is memory-starved (look at swap fields):
# si/so > 0 continuously = system is actively swapping (bad!)
# free near 0 + swpd large = RAM is exhausted

# System is CPU-bound (look at cpu fields):
# us + sy near 100% = all CPU time is consumed
# r > number_of_CPUs = processes are queuing for CPU

# System is I/O-bound (look at io/cpu fields):
# wa high (> 20%) = processes spending lots of time waiting for disk
# b > 0 consistently = blocked processes piling up

# Context switches are very high (system field):
# cs > 100,000/sec = potentially too many threads/processes

# Healthy system typical values:
# si = so = 0 (no swapping)
# id > 60% (plenty of idle CPU)
# wa < 5% (disk not a bottleneck)
# r <= number of CPU cores
```

Listing 145: Diagnosing problems with vmstat

The top Command

Definition — top

`top` provides a **real-time, continuously-updating view** of running processes sorted by CPU usage. It shows both system-wide statistics (memory, swap, load average, CPU) and a per-process table. Press `q` to quit.

```
top - 10:45:30 up 2 days, 3:15,  2 users,  load average: 0.52, 0.48, 0.45
Tasks: 187 total,   1 running, 186 sleeping,   0 stopped,   0 zombie
%Cpu(s):  5.2 us,  2.1 sy,  0.0 ni, 91.5 id,  1.2 wa,  0.0 hi,  0.0 si
MiB Mem : 15872.0 total,  8102.4 free,  4308.2 used,  3461.4 buff/cache
MiB Swap:  2048.0 total,  1919.8 free,   128.2 used.  11204.6 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
 1234 student   20   0  456780  61440  20480 R  12.5   0.4   0:15.42 python3
  456 www-data  20   0  216320  18432   8192 S   3.1   0.1   1:02.15 nginx
    1 root      20   0  167028  11936   8960 S   0.0   0.1   0:02.41 systemd
```

Listing 146: top — typical output layout

Interactive top Commands

```
# Launch top
$ top

# While top is running, press:
q            # Quit top
h or ?       # Help screen (all shortcuts)
k            # Kill a process (prompts for PID)
r            # Renice a process (change priority)
u            # Filter by username
M            # Sort by Memory usage (%MEM)
P            # Sort by CPU usage (%CPU) --- default
T            # Sort by TIME+ (total CPU time)
N            # Sort by PID
1            # Toggle: show all individual CPU cores
m            # Toggle memory display format
t            # Toggle task/cpu summary display
f            # Select fields to display
F            # Choose sort field interactively
d            # Change refresh delay (default: 3 seconds)
s            # Set update interval
Spacebar     # Force immediate refresh
z            # Toggle color mode
x            # Highlight the sort column
```

Listing 147: top — interactive keyboard shortcuts

Non-Interactive top Options

```
# Run once (non-interactive, like ps) (-n 1)
$ top -b -n 1

# Run 3 iterations with 2-second delay, save to file
$ top -b -n 3 -d 2 > top_report.txt

# Show processes for a specific user (-u flag)
$ top -u student

# Monitor a specific PID (-p flag)
$ top -p 1234
```

```
$ top -p 1234,5678   # multiple PIDs

# Show threads instead of processes (-H flag)
$ top -H

# Batch mode with 1 iteration, sorted by memory (for scripts)
$ top -b -n 1 -o %MEM | head -20
```

Listing 148: top — command-line and non-interactive options

Header Line	Field	Meaning
Line 1	`load average: a, b, c`	CPU load avg over 1, 5, 15 minutes
Line 2	`Tasks`	Total, running, sleeping, stopped, zombie
Line 3	`%Cpu`	us/sy/ni/id/wa/hi/si percentages
Line 4	`MiB Mem`	Total/free/used/buff-cache
Line 5	`MiB Swap`	Total/free/used/available
`PR`	Priority	Kernel priority (lower = runs sooner)
`NI`	Nice	Nice value (-20 to +19)
`VIRT`	Virtual	Total virtual memory used
`RES`	Resident	Physical RAM currently used
`SHR`	Shared	Shared memory (libs, etc.)
`S`	State	R/S/D/T/Z

The nice Command

Definition — nice and renice

nice launches a command with a modified scheduling *nice value*. **renice** changes the nice value of an already-running process. The nice value ranges from **-20** (highest priority, least nice to others) to **+19** (lowest priority, most nice to others). Regular users can only increase the nice value (lower priority); only root can decrease it.

Understanding Priority and Nice Values

Nice Value	Priority	Who Can Set
`-20`	Highest (most CPU)	root only
`-1`	Above normal	root only
`0`	Normal (default)	Any user
`+10`	Below normal	Any user
`+19`	Lowest (least CPU)	Any user

```
# ---- nice: start a new process with a nice value ----

# Default nice value (0)
$ nice ./my_script.sh

```

```
# Start with nice value +10 (lower priority)
$ nice -n 10 ./heavy_computation.sh

# Start with nice value +19 (lowest possible priority)
$ nice -n 19 ./background_backup.sh

# Only root can use NEGATIVE nice values (high priority)
$ sudo nice -n -10 ./critical_task.sh

# Verify: see nice value (NI column in ps)
$ ps -o pid,ni,cmd -p 1234
  PID  NI CMD
 1234  10 ./heavy_computation.sh

# ---- renice: change priority of a RUNNING process ----

# Lower priority of PID 1234 (increase nice to +15)
$ renice +15 -p 1234
1234 (process ID) old priority 0, new priority 15

# Set nice value for ALL processes owned by a user
$ sudo renice +5 -u student

# Set nice value for all processes in a process group
$ sudo renice +5 -g 1234

# Raise priority (root only) --- lower nice value
$ sudo renice -5 -p 1234
1234 (process ID) old priority 15, new priority -5

# View all processes with their nice values
$ ps -eo pid,ni,cmd --sort=ni | head -15

# Use top's 'r' key to renice interactively:
$ top
# Press 'r', enter PID 1234, enter nice value 10
```

Listing 149: nice and renice — managing process priority

Practical Use of `nice`

```
# Run a nightly backup at low priority so it doesn't slow others
$ nice -n 19 tar -czvf /backup/home_$(date +%Y%m%d).tar.gz /home/

# Compile a large project without slowing interactive work
$ nice -n 15 make -j4

# Run a CPU-intensive data analysis script in background
$ nice -n 10 python3 analysis.py &

# Monitor a high-priority critical process
$ sudo nice -n -20 ./real_time_monitor.sh &

# One-liner: find and lower priority of any process using > 80% CPU
$ ps -eo pid,pcpu --sort=-pcpu | awk 'NR>1 && $2>80 {print $1}' \
  | xargs -I{} sudo renice +10 -p {}

# Check what nice value a process is running at
$ cat /proc/1234/stat | awk '{print "Nice:", $19}'
Nice: 10
```

Listing 150: nice in real-world shell scripts

Unit 7 Summary

Key Takeaways — Unit 7: Processes in UNIX

1. A **process** is a running program instance with its own PID, memory, file descriptors, and state. Every process has a PCB maintained by the kernel.
2. **ps aux** lists all processes; key columns are PID, PPID, %CPU, %MEM, VSZ, RSS, STAT; `ps -ef` shows parent-child relationships; `ps axjf` draws the tree.
3. **init (PID 1)** is the first process started by the kernel; it is the ancestor of all processes. The login sequence: `init` → `getty` → `login` → `shell` → user commands.
4. **fork()** clones the current process; returns 0 to the child and the child's PID to the parent. **getpid()** returns own PID; **getppid()** returns parent's PID.
5. **wait()** blocks the parent until a child exits and collects its exit status, preventing zombies. **Zombie** = child exited but parent hasn't called wait(). **Orphan** = parent exited first, adopted by init.
6. **pipe()** creates a unidirectional channel: `fd[0]` reads, `fd[1]` writes. Shell pipes (`|`) use `pipe()` internally. **Message queues** provide typed, persistent, selective IPC via System V APIs.
7. The **seven process states**: Running (R), Interruptible sleep (S), Uninterruptible sleep (D), Stopped (T), Zombie (Z), Swapped out (W), plus kernel-mode running. CFS transitions between them based on I/O, signals, and scheduling.
8. The **CFS scheduler** uses virtual runtime (`vruntime`) stored in a red-black tree; always runs the process with the lowest vruntime; nice value adjusts the weight.
9. **Swap space** is disk used as overflow RAM; `kswapd` moves least-used pages out; `si/so` in vmstat show swap activity; high continuous swapping = thrashing.
10. **vmstat** shows procs/memory/swap/io/cpu in one line; `si/so > 0` = swapping, `wa` high = I/O bound, `r > CPUs` = CPU-bound; run `vmstat 2 5` for interval monitoring.
11. **top** shows a live updating process table; sort by CPU (P), memory (M), or time (T); `k` kills, `r` renice; `top -b -n 1` for scripting.
12. **nice** launches with adjusted priority (default 0; range -20 to +19; higher = less CPU). **renice** changes a running process. Only root can set negative (high priority) values.

End of Unit 7 — Processes in UNIX

Proceed to Unit 8: Shell Programming

Unit 7 — Practice Questions

Practice Questions — Unit 7: Processes in UNIX

A. Short Answer (2 marks each)

1. Define a process. What information does the PCB (Process Control Block) hold?
2. What is the difference between a zombie process and an orphan process?
3. Explain the `fork()` system call. What value does it return to the parent? To the child?
4. What does `wait()` do? Why is it important for preventing zombie processes?
5. What is the difference between `kill -9` and `kill -15`?
6. Explain the nice value range. Who can set a negative nice value and why?
7. What does `ps aux` show? Explain STAT column values: `R`, `S`, `D`, `Z`.
8. What is swap space? What symptoms indicate a system is thrashing?

B. Trace the Code (3 marks each)

1. How many processes does the following create, and what is printed?
   `fork(); fork(); printf("hello\n");`
2. In a program that calls `fork()`, the child calls `execlp("/bin/ls","ls",NULL)`. What happens to the child's memory space after `exec`?

C. Long Answer (5 marks each)

1. Explain the seven states of a UNIX process. What events cause transitions between states?
2. Describe the Linux login sequence from power-on to shell prompt: BIOS → bootloader → kernel → init/systemd → getty → login → shell.
3. Compare `top` and `vmstat` as monitoring tools. What does each header field mean?

Unit 8: Shell Programming

Unit 8 — Shell Programming

Introduction to Shell · Types of Shell · System & User Variables · Quoting · Command Substitution · `let` · User Input · Command-Line Parameters · `if` Statement · `for` / `while` / `until` Loops · Exit Status

Shell programming is the art of writing scripts — plain text files containing sequences of shell commands — that automate repetitive tasks, manage system administration, and glue UNIX utilities together. A shell script is the UNIX philosophy made tangible: small, composable tools orchestrated by a human-readable program.

Introduction to Shell and Types of Shell

What is a Shell?

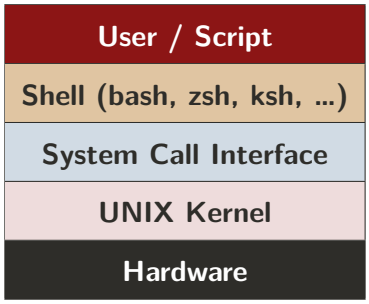
Definition — Shell

A **shell** is a command-line interpreter that acts as the interface between the *user* and the *kernel*. It reads commands (typed interactively or read from a script file), interprets them, and arranges for the kernel to execute them. The shell is itself just another user-space program — there is nothing special about it from the kernel's perspective.

Every time you open a terminal and type a command, you are talking to a shell. The shell performs several roles simultaneously:

1. **Command interpreter** — parses and executes commands
2. **Programming language** — variables, loops, conditionals, functions
3. **Environment manager** — sets and exports environment variables
4. **Process manager** — forks child processes, handles job control
5. **I/O facilitator** — manages redirection and pipes

The Shell's Place in the System



Types of Shell

UNIX has a rich history of shell development. Each shell builds on the lessons of its predecessors.

Shell	Full Name	Key Characteristics
`sh`	Bourne Shell	Original UNIX shell (1979, Stephen Bourne); minimal; POSIX baseline; found at `/bin/sh` on every system
`bash`	Bourne Again Shell	Most common on Linux; superset of `sh`; command history, tab completion, arrays, arithmetic; default on Ubuntu/Debian
`ksh`	Korn Shell	Written by David Korn at Bell Labs; combines `sh` syntax with C shell features; popular in enterprise UNIX (AIX, Solaris)
`csh`	C Shell	C-like syntax; aliases, history mechanism; scripting discouraged (many quirks)
`tcsh`	TENEX C Shell	Enhanced `csh`; better tab completion, history; default on FreeBSD
`zsh`	Z Shell	Feature-rich; superset of `bash`; powerful auto-completion, themes (Oh My Zsh); default on macOS since 2019
`dash`	Debian Almquist Shell	Tiny, fast, strictly POSIX; used as `/bin/sh` on Ubuntu/Debian for boot scripts
`fish`	Friendly Interactive Shell	Modern; auto-suggestions, syntax highlighting; not POSIX-compatible; great for interactive use

```
# Find out which shell you are currently using
$ echo $SHELL
/bin/bash

$ echo $0
bash

# List all shells installed on the system
$ cat /etc/shells
/bin/sh
/bin/bash
/usr/bin/bash
/bin/dash
/bin/zsh
/usr/bin/zsh

# Check the version of bash
$ bash --version
GNU bash, version 5.1.16(1)-release (x86_64-pc-linux-gnu)

# Switch to another shell temporarily
$ zsh           # start zsh; type 'exit' to return to bash
$ ksh           # start ksh

# Change your LOGIN shell permanently (takes effect on next login)
$ chsh -s /bin/zsh
```

```
Password: ****
Shell changed.

# Find the full path of a shell
$ which bash
/usr/bin/bash
$ which zsh
/usr/bin/zsh
```

Listing 151: Identifying and switching shells

sh vs bash in Scripts

Always start scripts intended for portability with `#!/bin/sh`. Use `#!/bin/bash` only when you need bash-specific features (arrays, `[[ ]]`, `let`, etc.). On Debian/Ubuntu, `/bin/sh` is `dash`, which is faster but lacks some bash extensions.

Use of Shell — Writing Your First Shell Script

What is a Shell Script?

Definition — Shell Script

A **shell script** is a plain-text file containing a sequence of shell commands. When executed, the shell reads and executes each command in order, just as if you had typed them at the prompt. The first line — the **shebang** (`#!`) — tells the OS which interpreter to use.

Creating and Running a Script

```
# Step 1: Create the script file
$ nano hello.sh

# Step 2: Add this content
#!/bin/bash
# My first shell script
# Lines starting with # are comments (except the shebang)

echo "Hello, World!"
echo "Today is: $(date +%A, %d %B %Y)"
echo "You are logged in as: $(whoami)"
echo "Current directory: $(pwd)"

# Step 3: Make the script executable
$ chmod +x hello.sh

# Step 4: Run it
$ ./hello.sh
Hello, World!
Today is: Monday, 15 January 2024
You are logged in as: student
Current directory: /home/student

# Alternative: run without making executable (invoke bash directly)
$ bash hello.sh

# Run with sh (POSIX-compatible mode)
$ sh hello.sh

# Run with debug output (prints each command before executing)
```

```
$ bash -x hello.sh
+ echo 'Hello, World!'
Hello, World!
+ date +%A, %d %B %Y
+ echo 'Today is: Monday, 15 January 2024'
Today is: Monday, 15 January 2024
```

Listing 152: Creating, making executable, and running a shell script

```
#!/bin/bash         # Use bash explicitly
#!/bin/sh           # Use system POSIX shell (portable)
#!/usr/bin/env bash  # Find bash in PATH (most portable for bash)
#!/usr/bin/env python3  # Scripts aren't limited to shell!
```

Listing 153: The shebang line — choosing the interpreter

Always Use `set -e` and `set -u`

Add these two lines near the top of every serious script:

- `set -e` — exit immediately if any command fails (non-zero exit)
- `set -u` — treat unset variables as errors
- `set -o pipefail` — a pipeline fails if any stage fails

Together: `set -euo pipefail` — the production-grade safety net.

System Variables and User-Defined Variables

Variables in Shell

Definition — Shell Variable

A **shell variable** is a named storage location that holds a string value. Shell variables have no explicit type — everything is a string, though arithmetic treats strings that look like numbers numerically. Variable names are case-sensitive; by convention, environment variables are UPPERCASE and local variables are lowercase.

User-Defined Variables

```
#!/bin/bash

# --- ASSIGNMENT (no spaces around = !) ---
name="Alice"
age=22
course="BCA"
pi=3.14159

# --- ACCESS with $ prefix ---
echo "Name: $name"
echo "Age: $age"
echo "Course: $course"

# Curly braces for disambiguation (BEST PRACTICE)
echo "Hello, ${name}!"            # Always safe
echo "${name}'s age is ${age}"

# Combining variables
full_info="${name} | ${course} | Age ${age}"
echo "$full_info"

# --- READONLY variables (like constants) ---
readonly MAX_RETRIES=3
```

```
echo "Max retries: $MAX_RETRIES"
MAX_RETRIES=5     # ERROR: bash: MAX_RETRIES: readonly variable

# --- UNSET a variable ---
temp="temporary value"
echo "$temp"
unset temp
echo "$temp"      # prints empty line

# --- Checking if a variable is set ---
if [ -z "$name" ]; then
    echo "name is empty or unset"
else
    echo "name = $name"
fi
```

Listing 154: Declaring and using user-defined variables

Common Mistake — Spaces Around =

```
name = "Alice"     # WRONG: bash treats 'name' as a command!
name="Alice"       # CORRECT: no spaces around =
```

System Variables (Environment Variables)

System variables (also called **environment variables**) are pre-defined by the shell or the OS and carry important information about the environment.

Variable	Example Value	Meaning
$HOME	/home/student	Current user's home directory
$USER	student	Currently logged-in username
$SHELL	/bin/bash	Path of the current shell
$PATH	/usr/bin:/bin:…	Colon-separated command search path
$PWD	/home/student	Present working directory
$OLDPWD	/etc	Previous working directory
$LOGNAME	student	Login name of the user
$HOSTNAME	mylinuxbox	Name of the machine
$TERM	xterm-256color	Type of terminal
$LANG	en_IN.UTF-8	Locale / language setting
$UID	1000	Numeric user ID of current user
$EUID	1000	Effective user ID
$PS1	\u@\h:\w$	Primary prompt string
$PS2	>	Secondary prompt (continuation)
$IFS	(space, tab, newline)	Internal field separator
$RANDOM	(0–32767)	Pseudo-random integer
$LINENO	(current line)	Current line number in script
$$	4521	PID of the current shell
$?	0 or 1	Exit status of last command
$!	4522	PID of last background process
$0	./myscript.sh	Name of the script itself

```
#!/bin/bash
# Demonstrating system variables

echo "--- System Information ---"
echo "User       : $USER ($UID)"
echo "Home       : $HOME"
echo "Shell      : $SHELL"
echo "Hostname   : $HOSTNAME"
echo "Current Dir: $PWD"
echo "Script Name: $0"
echo "Script PID : $$"

# Modify PATH to add a custom bin directory
export PATH="$HOME/bin:$PATH"
echo "Updated PATH: $PATH"

# View all environment variables
env | sort | head -20

# Check if running as root
```

```
if [ "$EUID" -eq 0 ]; then
    echo "Running as root"
else
    echo "Running as regular user: $USER"
fi

# Use RANDOM for a simple number between 1 and 100
dice=$((RANDOM % 100 + 1))
echo "Random number (1-100): $dice"
```

Listing 155: Using system variables in scripts

Exporting Variables

Definition — export

By default, a variable defined in a shell is **local** to that shell process. **Exporting** a variable places it in the *environment*, making it available to all child processes spawned from the current shell.

```
# Local variable (NOT visible to child processes)
greeting="Hello"
bash -c 'echo $greeting'    # prints nothing!

# Exported variable (visible to all child processes)
export greeting="Hello"
bash -c 'echo $greeting'    # prints: Hello

# Declare and export in one step
export JAVA_HOME="/usr/lib/jvm/java-17"
export DB_HOST="localhost"
export DB_PORT=5432

# Make an existing variable available to children
myvar="test"
export myvar

# Remove a variable from the environment
export -n myvar     # unexport (still exists locally, not exported)
unset myvar         # delete completely

# List all exported variables
export -p           # same as: env
```

Listing 156: export — making variables available to child processes

Use of Single Quote, Double Quote, and Backslash

Definition — Quoting

Quoting in shell controls which characters are treated literally and which retain their special meaning. The shell has three quoting mechanisms: single quotes, double quotes, and backslash (escape character).

Backslash — Escaping a Single Character

The backslash (\) removes the special meaning of the *immediately following* character.

```
# Without backslash: $ is special (variable expansion)
$ echo $HOME
/home/student
```

```

# With backslash: $ is treated literally
$ echo \$HOME
$HOME

# Escape a space in a filename
$ mkdir My\ Documents
$ ls
'My Documents'

# Escape special characters
$ echo 2 \* 3 = 6          # * without backslash = glob!
$ echo 2 \* 3 = 6
2 * 3 = 6

# Line continuation: backslash at END of line
$ echo "This is a very long command that " \
>      "continues on the next line"
This is a very long command that continues on the next line

# Common characters that need escaping
# \n  = newline  \t  = tab  \\  = literal backslash
# \$  = dollar   \"  = quote  \!  = exclamation
```

Listing 157: Backslash — escaping special characters

Single Quotes — Full Literal Preservation

Single quotes preserve **every character** literally. No expansion of any kind occurs inside single quotes — not variables, not command substitution, not escape sequences.

```
name="Alice"
path="/home/student"

# Single quotes: NO variable expansion
$ echo 'Hello $name, you are in $path'
Hello $name, you are in $path

# Single quotes: NO command substitution
$ echo 'Today is $(date)'
Today is $(date)

# Single quotes: NO escape sequences (backslash is literal too!)
$ echo 'Line1\nLine2'
Line1\nLine2

# Preserving asterisks, pipes, redirects literally
$ echo 'cp * /backup/ | tee log.txt'
cp * /backup/ | tee log.txt

# Cannot embed a single quote inside single quotes
# Workaround: end the quote, escape, restart
$ echo 'It'\''s a beautiful day'
It's a beautiful day

# Or use $'...' (ANSI-C quoting) for special chars
$ echo $'It\'s a beautiful day'
It's a beautiful day
```

Listing 158: Single quotes — everything is literal

Double Quotes — Controlled Expansion

Double quotes allow **variable expansion**, **command substitution**, and **arithmetic expansion**, but suppress globbing and word splitting.

```
name="Alice"
course="BCA"

# Variable expansion WORKS inside double quotes
$ echo "Hello, $name! You are studying $course."
Hello, Alice! You are studying BCA.

# Command substitution WORKS
$ echo "Current date: $(date +%d-%b-%Y)"
Current date: 15-Jan-2024

# Arithmetic expansion WORKS
$ echo "2 + 3 = $((2 + 3))"
2 + 3 = 5

# Globbing is SUPPRESSED (asterisk is literal)
$ echo "All files: *"
All files: *

# Backslash ONLY escapes: $ ` " \ newline
$ echo "Price: \$100"       # \$ treated as literal $
Price: $100
$ echo "Say \"hello\""      # \" = literal quote
Say "hello"
$ echo "Backslash: \\"      # \\ = single backslash
Backslash: \

# Word splitting is SUPPRESSED (important for filenames with spaces!)
file="My Document.txt"
cat "$file"       # correct: one argument
cat $file         # wrong: two arguments 'My' and 'Document.txt'
```

Listing 159: Double quotes — selective expansion

Quoting Rules Summary

Feature	No Quotes	Single Quotes	Double Quotes
Variable `$var`	Expanded	Literal	Expanded
Command `$(cmd)`	Expanded	Literal	Expanded
Arithmetic `$(( ))`	Expanded	Literal	Expanded
Glob `* ? []`	Expanded	Literal	Literal
Word splitting	Performed	Suppressed	Suppressed
Backslash escape	Active	Literal	Partial

Command Substitution

Definition — Command Substitution

Command substitution allows the output of a command to replace the command itself in-line. The shell executes the command in a subshell, captures its standard output (stripping trailing newlines), and inserts that text into the current command.

Syntax: $(…) vs `…`

```
# ---- MODERN SYNTAX: $(...) --- PREFERRED ----
today=$(date +%Y-%m-%d)
echo "Today: $today"

# ---- LEGACY SYNTAX: backticks `...` ----
today=`date +%Y-%m-%d`
echo "Today: $today"

# Modern syntax is preferred because:
# 1. Easier to nest (backticks cannot be nested easily)
# 2. Backslash handling is cleaner
# 3. More readable

# --- NESTING command substitutions ---
# Get the username of the process with the highest CPU
top_user=$(ps aux --sort=-%cpu | awk 'NR==2{print $1}')
echo "Top CPU user: $top_user"

# Nested: count files in the directory of the current shell's binary
count=$(ls $(dirname $(which bash)) | wc -l)
echo "Files in bash's directory: $count"
```

Listing 160: Command substitution — modern and legacy syntax

Practical Uses of Command Substitution

```
#!/bin/bash

# 1. Capture command output into a variable
kernel=$(uname -r)
hostname=$(hostname)
echo "Running kernel $kernel on host $hostname"

# 2. Use in arithmetic
num_users=$(who | wc -l)
echo "There are $num_users users logged in."

# 3. Dynamic filename with timestamp
backup_name="backup_$(date +%Y%m%d_%H%M%S).tar.gz"
tar -czf "$backup_name" /home/student/
echo "Backup created: $backup_name"

# 4. Conditional based on command output
current_dir=$(pwd)
if [ "$current_dir" = "$HOME" ]; then
    echo "You are in your home directory."
fi

# 5. Build a list from a command
files=$(find . -name "*.sh" -type f)
echo "Shell scripts found:"
echo "$files"

# 6. Store multi-line output
```

```
disk_usage=$(df -h /)
echo "Disk usage:"
echo "$disk_usage"

# 7. In loop iteration
for user in $(cut -d: -f1 /etc/passwd | head -5); do
    echo "User: $user"
done
```

Listing 161: Command substitution in real scripts

Always Quote Command Substitution

Always use double quotes around `$(...)` when the result may contain spaces or newlines: `files="$(find . -name '*.txt')"` — otherwise word splitting will break filenames with spaces.

`let` — Assigning and Evaluating Expressions

Definition — let

`let` is a bash built-in that evaluates **arithmetic expressions** and assigns results to variables. It operates on integers only (no floating point). An alternative, preferred modern syntax is `$(( expression ))`.

Arithmetic Operators

Operator	Meaning	Example
`+`	Addition	`let x=5+3` → `x=8`
`-`	Subtraction	`let x=10-4` → `x=6`
`*`	Multiplication	`let x=4*3` → `x=12`
`/`	Integer division	`let x=7/2` → `x=3`
`%`	Modulus (remainder)	`let x=7%2` → `x=1`
`**`	Exponentiation	`let x=2**8` → `x=256`
`++`	Pre/post increment	`let x++`
`--`	Pre/post decrement	`let x--`
`&`	Bitwise AND	`let x=6&3` → `x=2`
`|`	Bitwise OR	`let x=6|3` → `x=7`
`^`	Bitwise XOR	`let x=6^3` → `x=5`
`~`	Bitwise NOT	`let x=~6`
`<<`	Left shift	`let x=1<<3` → `x=8`
`>>`	Right shift	`let x=8>>1` → `x=4`

```
#!/bin/bash

# ---- Using let ----
let a=10
let b=3

```

```
let sum=a+b
let diff=a-b
let prod=a*b
let quot=a/b        # integer division
let rem=a%b

echo "a=$a, b=$b"
echo "Sum:     $sum"     # 13
echo "Diff:    $diff"    # 7
echo "Product: $prod"    # 30
echo "Quotient:$quot"    # 3 (not 3.33!)
echo "Remainder:$rem"    # 1

# Increment and decrement
let count=0
let count++
let count++
echo "Count: $count"    # 2

let count--
echo "Count: $count"    # 1

# Exponentiation
let power=2**10
echo "2^10 = $power"    # 1024

# ---- Using $(( )) --- MODERN PREFERRED SYNTAX ----
x=15
y=4

echo "Sum:      $((x + y))"      # 19
echo "Quotient: $((x / y))"      # 3
echo "Remainder:$((x % y))"      # 3
echo "Power:    $((2 ** 8))"     # 256

# Assign result of $(( ))
result=$((x * y - 2))
echo "Result: $result"           # 58

# ---- Using (( )) for pure arithmetic (no echo) ----
(( total = 100 * 5 / 2 ))
echo "Total: $total"             # 250

# Useful for loop counters
(( i = 0 ))
(( i++ ))
echo "i = $i"                    # 1

# ---- Comparison inside (( )) ----
# Returns 0 (true) if expression is non-zero
(( 5 > 3 )) && echo "5 is greater than 3"
(( 10 == 10 )) && echo "Ten equals ten"

# ---- declare -i: integer variables ----
declare -i num=42
num=num+8
echo "num = $num"                # 50
num="abc"                        # non-numeric string treated as 0
echo "num = $num"                # 0
```

Listing 162: let and arithmetic expressions

```
#!/bin/bash
# Practical arithmetic: area and simple statistics

# Area of a rectangle
length=12
width=7
area=$((length * width))
echo "Area: ${length} x ${width} = ${area} sq. units"

# Percentage calculation
total=200
obtained=165
percent=$((obtained * 100 / total))
echo "Score: ${obtained}/${total} = ${percent}%"

# Celsius to Fahrenheit (integer only)
celsius=37
fahrenheit=$(( (celsius * 9 / 5) + 32 ))
echo "${celsius}°C = ${fahrenheit}°F"

# For floating-point arithmetic, use bc
echo "Exact F: $(echo "scale=2; $celsius * 9/5 + 32" | bc)"
```

Listing 163: Practical arithmetic in scripts

Taking Input from the User

Definition — read

The **read** built-in reads a line from standard input (the keyboard, or a pipe) and assigns it to one or more variables. It is the primary way to make scripts interactive.

```
#!/bin/bash

# --- BASIC read ---
echo "Enter your name:"
read name
echo "Hello, $name!"

# --- Inline prompt with -p (no separate echo needed) ---
read -p "Enter your age: " age
echo "You are $age years old."

# --- Read multiple variables at once ---
read -p "Enter first and last name: " first last
echo "First: $first, Last: $last"

# --- Silent input (for passwords) ---
read -s -p "Enter password: " password
echo ""                          # newline after silent input
echo "Password length: ${#password} characters"

# --- Read with a timeout (-t seconds) ---
read -t 10 -p "Enter a choice (10 sec timeout): " choice
if [ $? -ne 0 ]; then
    echo ""
    echo "Timed out! Using default."
    choice="default"
fi
echo "Choice: $choice"
```

```

# --- Limit input length (-n characters) ---
read -n 1 -p "Continue? [y/n]: " answer
echo ""
if [ "$answer" = "y" ] || [ "$answer" = "Y" ]; then
    echo "Continuing..."
else
    echo "Aborted."
fi

# --- Read into an array (-a flag) ---
read -p "Enter three numbers: " -a numbers
echo "First:  ${numbers[0]}"
echo "Second: ${numbers[1]}"
echo "Third:  ${numbers[2]}"

# --- Read a file line by line ---
while IFS= read -r line; do
    echo "Line: $line"
done < /etc/hosts
```

Listing 164: read — taking interactive input from the user

```
#!/bin/bash
# Grade calculator using read

echo "==============================="
echo "   Student Grade Calculator    "
echo "==============================="
read -p "Student Name   : " sname
read -p "Marks in Maths : " math
read -p "Marks in English: " english
read -p "Marks in Science: " science

total=$((math + english + science))
avg=$((total / 3))

echo ""
echo "--- Result for $sname ---"
echo "Total  : $total / 300"
echo "Average: $avg%"

if [ $avg -ge 90 ]; then
    grade="A+"
elif [ $avg -ge 75 ]; then
    grade="A"
elif [ $avg -ge 60 ]; then
    grade="B"
elif [ $avg -ge 45 ]; then
    grade="C"
else
    grade="F"
fi

echo "Grade  : $grade"
```

Listing 165: Practical interactive script: student grade calculator

Option	Example	Effect
`-p`	`read -p "Name: " x`	Display prompt inline
`-s`	`read -s x`	Silent (no echo — for passwords)
`-t N`	`read -t 5 x`	Timeout after N seconds
`-n N`	`read -n 1 x`	Read exactly N characters
`-r`	`read -r x`	Raw mode: backslash not treated as escape
`-a`	`read -a arr`	Read words into an array

Command Line Parameters

Definition — Positional Parameters

Command-line parameters (positional parameters) are the arguments passed to a script when it is invoked. They are accessed via special variables: `$1`, `$2`, ..., `$9`, `${10}`, ...along with several special meta-variables.

Positional Parameter Variables

Variable	Meaning	Example (`./script.sh Alice 22 BCA`)
`$0`	Script name (argv[0])	`./script.sh`
`$1`	First argument	`Alice`
`$2`	Second argument	`22`
`$3`	Third argument	`BCA`
`${10}`	10th argument (braces required beyond 9)	—
`$#`	Number of arguments (not counting $0)	`3`
`$*`	All arguments as a single string	`"Alice 22 BCA"`
`$@`	All arguments as separate strings	`"Alice" "22" "BCA"`
`$?`	Exit status of last command	`0` (success)
`$$`	PID of current shell	`4521`
`$!`	PID of last background job	`4522`

```bash
#!/bin/bash
# Save as: greet.sh
# Usage:   ./greet.sh Alice 22 BCA

echo "Script name : $0"
echo "First arg   : $1"
echo "Second arg  : $2"
echo "Third arg   : $3"
echo "Arg count   : $#"
echo "All args    : $@"
echo "All as one  : $*"
echo "Shell PID   : $$"
```

Listing 166: Using positional parameters in a script

```
$ ./greet.sh Alice 22 BCA
Script name : ./greet.sh
First arg   : Alice
Second arg  : 22
Third arg   : BCA
Arg count   : 3
All args    : Alice 22 BCA
All as one  : Alice 22 BCA
Shell PID   : 4521
```

Listing 167: Running the script

Validating Arguments and `shift`

```
#!/bin/bash
# backup.sh --- usage: ./backup.sh <source_dir> <dest_dir>

# Check argument count
if [ $# -ne 2 ]; then
    echo "Usage: $0 <source_dir> <dest_dir>"
    echo "Example: $0 /home/student /backup/"
    exit 1
fi

source_dir="$1"
dest_dir="$2"

# Validate that source exists
if [ ! -d "$source_dir" ]; then
    echo "Error: Source directory '$source_dir' does not exist."
    exit 2
fi

echo "Backing up '$source_dir' to '$dest_dir'..."
tar -czf "${dest_dir}/backup_$(date +%Y%m%d).tar.gz" "$source_dir"
echo "Done."

# --- shift: remove $1, shift all others down ---
# Useful for processing a variable number of arguments
#!/bin/bash
echo "Processing all arguments:"
while [ $# -gt 0 ]; do
    echo "  Argument: $1"
    shift          # $2 becomes $1, $3 becomes $2, etc.
done
```

Listing 168: Argument validation and the shift command

```
#!/bin/bash
# Difference between "$@" and "$*"

print_args_at() {
    echo "Using \"\$@\":"
    for arg in "$@"; do          # each arg as separate, quoted word
        echo "  -> [$arg]"
    done
}

```

```
print_args_star() {
    echo "Using \"\$*\":"
    for arg in "$*"; do          # all args merged into ONE string
        echo "  -> [$arg]"
    done
}

print_args_at "Alice" "Bob Smith" "Charlie"
echo "---"
print_args_star "Alice" "Bob Smith" "Charlie"
```

Listing 169: Iterating over all arguments with $@ and $*

```
Using "$@":
  -> [Alice]
  -> [Bob Smith]        # space preserved!
  -> [Charlie]
---
Using "$*":
  -> [Alice Bob Smith Charlie]   # all merged into one
```

Listing 170: Output demonstrating $@ vs $*

Always Use "$@" to Pass Arguments

Use `"$@"` (quoted) when forwarding arguments to another command — it correctly handles arguments with spaces. `"$*"` merges all arguments into one string, breaking arguments that contain spaces.

Use of `if` Statement

Definition — Conditional Execution

The **if** statement evaluates a condition (a command's exit status) and executes a block of code only if the condition is true (exit status `0`). Every UNIX command returns an exit status; the `test` command (alias `[ ]`) is the most common way to evaluate conditions.

Basic `if` Syntax

```
# ---- Basic if ----
if condition; then
    commands
fi

# ---- if-else ----
if condition; then
    commands_if_true
else
    commands_if_false
fi

# ---- if-elif-else (multiple branches) ----
if condition1; then
    commands1
elif condition2; then
    commands2
elif condition3; then
    commands3
else
    default_commands
```

```
fi
```

Listing 171: if / elif / else — complete syntax

The `test` Command and `[ ]` Brackets

```bash
#!/bin/bash
x=10
y=20
name="Alice"
file="/etc/passwd"

# --- NUMERIC COMPARISONS ---
if [ $x -eq 10 ]; then echo "x equals 10"; fi
if [ $x -ne $y ]; then echo "x not equal to y"; fi
if [ $x -lt $y ]; then echo "x less than y"; fi
if [ $x -le 10 ]; then echo "x less than or equal to 10"; fi
if [ $y -gt $x ]; then echo "y greater than x"; fi
if [ $y -ge 20 ]; then echo "y >= 20"; fi

# --- STRING COMPARISONS ---
if [ "$name" = "Alice" ]; then echo "Name is Alice"; fi
if [ "$name" != "Bob" ];  then echo "Name is not Bob"; fi
if [ -z "$name" ]; then echo "Name is empty"; fi      # -z: zero length
if [ -n "$name" ]; then echo "Name is non-empty"; fi  # -n: non-zero length

# --- FILE TESTS ---
if [ -f "$file" ]; then echo "$file exists and is a regular file"; fi
if [ -d "/tmp" ];  then echo "/tmp is a directory"; fi
if [ -e "$file" ]; then echo "$file exists (any type)"; fi
if [ -r "$file" ]; then echo "$file is readable"; fi
if [ -w "$file" ]; then echo "$file is writable"; fi
if [ -x "/bin/ls" ]; then echo "/bin/ls is executable"; fi
if [ -s "$file" ]; then echo "$file is non-empty"; fi  # -s: size > 0
if [ -L "/bin" ];  then echo "/bin is a symbolic link"; fi

# --- LOGICAL OPERATORS ---
if [ $x -gt 5 ] && [ $x -lt 15 ]; then echo "x is between 5 and 15"; fi
if [ $x -eq 5 ] || [ $x -eq 10 ]; then echo "x is 5 or 10"; fi
if ! [ $x -eq 99 ]; then echo "x is NOT 99"; fi
```

Listing 172: — evaluating conditions]test and [ ] — evaluating conditions

Category	Operator	Meaning	Example
Numeric	`-eq`	Equal	`[ $a -eq $b ]`
	`-ne`	Not equal	`[ $a -ne $b ]`
	`-lt`	Less than	`[ $a -lt $b ]`
	`-le`	Less than or equal	`[ $a -le $b ]`
	`-gt`	Greater than	`[ $a -gt $b ]`
	`-ge`	Greater than or equal	`[ $a -ge $b ]`
String	`=`	Equal (strings)	`[ "$s" = "val" ]`
	`!=`	Not equal	`[ "$s" != "val" ]`
	`-z`	Zero-length string	`[ -z "$s" ]`
	`-n`	Non-empty string	`[ -n "$s" ]`
File	`-f`	Regular file exists	`[ -f file ]`
	`-d`	Directory exists	`[ -d dir ]`
	`-e`	Exists (any type)	`[ -e path ]`
	`-r`	Readable	`[ -r file ]`
	`-w`	Writable	`[ -w file ]`
	`-x`	Executable	`[ -x file ]`
	`-s`	Non-empty file	`[ -s file ]`
Logical	`-a` / `&&`	AND	`[ c1 -a c2 ]`
	`-o` / `||`	OR	`[ c1 -o c2 ]`
	`!`	NOT	`[ ! cond ]`

The `[[ ]]` Extended Test (Bash-Only)

```bash
#!/bin/bash
name="Alice"
num=42

# [[ ]] supports regex matching with =~
if [[ "$name" =~ ^Al ]]; then
    echo "Name starts with 'Al'"
fi

# [[ ]] supports < and > for string comparison
if [[ "$name" < "Bob" ]]; then
    echo "Alice comes before Bob alphabetically"
fi

# [[ ]] is safer: no word splitting or globbing
file="my file.txt"
if [[ -f $file ]]; then    # works even without quotes
    echo "File exists"
fi

# Arithmetic comparison with (( ))
```

```
if (( num > 40 && num < 50 )); then
    echo "num is between 40 and 50"
fi
```

Listing 173:] — enhanced test with regex and safer string handling

Practical if Examples

```
#!/bin/bash
read -p "Enter a number: " num

if [ $num -gt 0 ]; then
    echo "$num is POSITIVE"
elif [ $num -lt 0 ]; then
    echo "$num is NEGATIVE"
else
    echo "The number is ZERO"
fi

# Check if a command exists before using it
if command -v git &>/dev/null; then
    echo "git version: $(git --version)"
else
    echo "git is not installed. Install with: sudo apt install git"
fi

# Check if running as root
if [ "$EUID" -ne 0 ]; then
    echo "This script requires root. Run with sudo."
    exit 1
fi
echo "Running as root. Proceeding..."
```

Listing 174: Complete if-elif-else — number classifier

Use of for, while, and until Loops

for Loop

Definition — for Loop

The **for** loop iterates over a *list of values*, executing the loop body once for each value. The list can be explicit, the result of a command, a glob pattern, or a C-style numeric range.

```
#!/bin/bash

# --- Variant 1: explicit list ---
for fruit in apple banana cherry date elderberry; do
    echo "Fruit: $fruit"
done

# --- Variant 2: numeric range with {start..end} ---
echo "--- Counting 1 to 5 ---"
for i in {1..5}; do
    echo "i = $i"
done

# --- Variant 3: range with step {start..end..step} ---
echo "--- Even numbers: 0 to 10 ---"
for i in {0..10..2}; do
```

```
    echo -n "$i "
done
echo ""

# --- Variant 4: C-style for loop ---
echo "--- C-style (countdown) ---"
for (( i=10; i>=1; i-- )); do
    echo -n "$i "
done
echo ""

# --- Variant 5: iterate over command output ---
echo "--- Shell scripts in /usr/bin ---"
for script in $(ls /usr/bin/*.sh 2>/dev/null | head -5); do
    echo "  $script"
done

# --- Variant 6: iterate over files (glob) ---
echo "--- Text files in /etc ---"
for file in /etc/*.conf; do
    echo "  $(basename $file)"
done

# --- Variant 7: iterate over array elements ---
courses=("BCA" "MCA" "BSc" "MSc")
for course in "${courses[@]}"; do
    echo "Course: $course"
done

# --- Practical: rename all .txt files to .bak ---
for f in *.txt; do
    mv "$f" "${f%.txt}.bak"
    echo "Renamed: $f -> ${f%.txt}.bak"
done
```

Listing 175: for loop — all variants

`while` Loop

Definition — while Loop

The **while** loop executes its body *as long as the condition remains true* (exit status 0). The condition is re-evaluated before each iteration. Use `while` when the number of iterations is not known in advance.

```
#!/bin/bash

# --- Basic while: count down ---
count=5
while [ $count -gt 0 ]; do
    echo "Count: $count"
    (( count-- ))
done
echo "Done!"

# --- while with arithmetic (( )) condition ---
n=1
while (( n <= 5 )); do
    echo "$n squared = $((n * n))"
    (( n++ ))
done
```

```

# --- Infinite loop (break to exit) ---
while true; do
    read -p "Enter 'quit' to exit: " input
    if [ "$input" = "quit" ]; then
        echo "Goodbye!"
        break
    fi
    echo "You entered: $input"
done

# --- Read a file line by line (most common while use case) ---
echo "--- /etc/hosts entries ---"
while IFS= read -r line; do
    # Skip comments and blank lines
    [[ "$line" =~ ^#  ]] && continue
    [[ -z "$line"     ]] && continue
    echo "  $line"
done < /etc/hosts

# --- Process a comma-separated list ---
csv_line="Alice,22,BCA,Hooghly"
IFS=',' read -ra fields <<< "$csv_line"
i=0
while [ $i -lt ${#fields[@]} ]; do
    echo "Field $((i+1)): ${fields[$i]}"
    (( i++ ))
done

# --- Monitor: check until a process appears ---
echo "Waiting for nginx to start..."
while ! pgrep nginx > /dev/null 2>&1; do
    echo "  nginx not running yet, waiting..."
    sleep 2
done
echo "nginx is now running!"
```

Listing 176: while loop — condition-controlled repetition

`until` Loop

Definition — until Loop

The **until** loop is the logical inverse of `while`: it executes its body *as long as the condition is FALSE* (exit status non-zero). It keeps looping *until* the condition becomes true.

```
#!/bin/bash

# --- Basic until: count up ---
count=1
until [ $count -gt 5 ]; do
    echo "Count: $count"
    (( count++ ))
done

# --- until vs while (equivalent loops) ---
# These two loops do the same thing:
i=0
while [ $i -lt 3 ]; do echo "while: $i"; (( i++ )); done

i=0
```

```
until [ $i -ge 3 ]; do echo "until: $i"; (( i++ )); done

# --- until: wait for a file to be created ---
echo "Waiting for /tmp/signal_file to appear..."
until [ -f /tmp/signal_file ]; do
    echo "  Not yet... (sleeping 1s)"
    sleep 1
done
echo "Signal file found! Continuing."

# --- until: retry a command until it succeeds ---
max_attempts=5
attempt=1
until ping -c 1 google.com &>/dev/null || [ $attempt -ge $max_attempts ]; do
    echo "Attempt $attempt failed. Retrying..."
    (( attempt++ ))
    sleep 2
done

if [ $attempt -lt $max_attempts ]; then
    echo "Network is reachable!"
else
    echo "Network unreachable after $max_attempts attempts."
fi
```

Listing 177: until loop — loop until condition becomes true

Loop Control: `break` and `continue`

```
#!/bin/bash

# --- break: exit the loop immediately ---
echo "--- break example ---"
for i in {1..10}; do
    if [ $i -eq 6 ]; then
        echo "Breaking at i=$i"
        break
    fi
    echo "i = $i"
done

# --- continue: skip rest of current iteration ---
echo "--- continue example (skip even numbers) ---"
for i in {1..10}; do
    if (( i % 2 == 0 )); then
        continue      # skip even numbers
    fi
    echo "i = $i"     # only odd numbers printed
done

# --- break N: break out of N nested loops ---
echo "--- break 2 (nested loops) ---"
for i in 1 2 3; do
    for j in a b c; do
        if [ "$j" = "b" ] && [ $i -eq 2 ]; then
            echo "Breaking both loops at i=$i, j=$j"
            break 2
        fi
        echo "  i=$i, j=$j"
    done
done
echo "After nested loops"
```

Listing 178: break and continue — loop flow control

Loop Comparison

Loop	Use When...	Condition Check
`for`	Number of iterations is known; iterating over a list, range, or array	Implicitly: list exhausted
`while`	Repeat while something remains true; unknown iteration count; reading files line by line	Before each iteration (pre-test)
`until`	Repeat until something becomes true; inverse logic of `while`; retry patterns	Before each iteration (pre-test, inverted)

Observing the Exit Status

Definition — Exit Status

Every UNIX command, program, or function, when it finishes executing, returns a small integer to its parent process called the **exit status** (or **return code**). By convention: **0 means success**; any **non-zero value means failure**. The exit status of the most recently executed command is stored in the special variable `$?`.

Reading and Using $?

```bash
#!/bin/bash

# --- Every command sets $? ---
ls /etc/passwd
echo "Exit status of ls: $?"    # 0 (success)

ls /nonexistent_path 2>/dev/null
echo "Exit status of ls: $?"    # 2 (no such file)

grep "root" /etc/passwd > /dev/null
echo "Exit status of grep: $?"  # 0 (found)

grep "xyz_not_exist" /etc/passwd > /dev/null
echo "Exit status of grep: $?"  # 1 (not found)

# --- $? is overwritten by EACH command ---
ls /tmp > /dev/null
echo "ls exit: $?"
echo "echo exit: $?"    # 0 (echo itself succeeded!)
# WRONG: by the time we read $? again it's the exit of the echo

# CORRECT: save $? immediately
ls /tmp > /dev/null
status=$?
echo "ls exit status was: $status"

# --- Standard exit codes ---
# 0   = success
# 1   = general error
# 2   = misuse of shell builtin
```

```
# 126 = command found but not executable
# 127 = command not found
# 128 = invalid argument to exit
# 128+N = fatal error signal N (e.g. 130 = Ctrl+C = SIGINT)
# 255 = exit status out of range
```

Listing 179: Checking exit status with $?

The `exit` Command in Scripts

```
#!/bin/bash
# safe_delete.sh --- safely remove a file with error checking

if [ $# -eq 0 ]; then
    echo "Usage: $0 <filename>"
    exit 1        # non-zero = failure (missing argument)
fi

file="$1"

if [ ! -f "$file" ]; then
    echo "Error: '$file' is not a regular file."
    exit 2        # different non-zero = different kind of failure
fi

if [ ! -w "$file" ]; then
    echo "Error: '$file' is not writable."
    exit 3
fi

rm "$file"
echo "'$file' deleted successfully."
exit 0            # 0 = success (explicit, though 0 is default)
```

Listing 180: exit — setting the script's own exit code

Using Exit Status in Conditionals

```
#!/bin/bash

# --- if directly uses exit status ---
if ping -c 1 8.8.8.8 &>/dev/null; then
    echo "Internet is reachable"
else
    echo "No internet connection"
fi

# --- && (AND): run right side ONLY if left side succeeds ---
mkdir /tmp/mydir && echo "Directory created successfully"
cd /tmp/mydir && touch file1.txt file2.txt

# --- || (OR): run right side ONLY if left side FAILS ---
mkdir /tmp/existing_dir 2>/dev/null || echo "Directory already exists"
command_that_might_fail || { echo "Failed! Cleaning up."; exit 1; }

# --- Combined: try, fall back, or die ---
cp important.txt /backup/ && echo "Backup succeeded" \
                          || { echo "Backup FAILED!"; exit 1; }

# --- Checking exit status in a while loop ---
attempts=0
while ! ping -c 1 google.com &>/dev/null; do
    echo "Attempt $((++attempts)): no connection"
```

```
    sleep 5
    [ $attempts -ge 5 ] && { echo "Giving up."; exit 1; }
done
echo "Connected after $attempts attempts."
```

Listing 181: Using $? and short-circuit operators

Custom Exit Codes and `trap`

```
#!/bin/bash
# Script with cleanup on exit/interrupt

TMPFILE=$(mktemp /tmp/script_XXXXXX)

# trap: execute commands when the script exits or receives a signal
trap 'rm -f "$TMPFILE"; echo "Cleaned up $TMPFILE"' EXIT
trap 'echo "Interrupted! Cleaning up..."; exit 130' INT TERM

echo "Working... (temp file: $TMPFILE)"
echo "some data" > "$TMPFILE"

# Simulate work
for i in {1..5}; do
    echo "Processing step $i..."
    sleep 1
done

echo "Script completed successfully."
exit 0
# When the script exits (normally OR via Ctrl+C),
# the trap on EXIT fires and cleans up TMPFILE
```

Listing 182: trap — handling signals and cleanup on exit

Exit Code	Meaning	Set by
0	Success	`exit 0`
1	General/unspecified error	`exit 1`
2	Misuse of shell commands	`exit 2`
126	Command not executable	Shell
127	Command not found	Shell
128	Invalid exit argument	Shell
130	Terminated by Ctrl+C (SIGINT)	Shell (128+2)
137	Killed (SIGKILL)	Shell (128+9)

The Zero/Non-Zero Convention

This convention is the foundation of shell logic. **Every if, while, until, &&, and ||** uses exit status. When you write `if [ condition ]`, the `[` command runs and returns 0 (true) or 1 (false). This is why commands can be used directly in conditions without any `[ ]`.

Unit 8 Summary

Key Takeaways — Unit 8: Shell Programming

1. A **shell** is a command interpreter and programming language. Common shells: `sh` (POSIX baseline), `bash` (most common on Linux), `zsh` (macOS default), `dash` (fast, minimal). Check current shell with `$SHELL`; change permanently with `chsh`.
2. A **shell script** starts with a **shebang** (`#!/bin/bash`), is made executable with `chmod +x`, and run with `./script.sh`. Use `set -euo pipefail` for robust scripts.
3. **Variables** are assigned with `name=value` (no spaces). Access with `$name` or `${name}`. `readonly` creates constants; `unset` removes variables; `export` makes variables available to child processes.
4. Key **system variables**: `$HOME`, `$USER`, `$PATH`, `$PWD`, `$SHELL`, `$?`, `$$`, `$0`, `$#`, `$@`.
5. **Single quotes** preserve everything literally. **Double quotes** allow variable, command, and arithmetic expansion but suppress globbing and word splitting. **Backslash** escapes one character.
6. **Command substitution** `$(command)` captures the output of a command into a variable or expression. Always quote: `var="$(cmd)"`.
7. **let** and `(( ))` perform integer arithmetic. `$(( expr ))` embeds arithmetic in strings. Use `bc` for floating-point. Key operators: `+ - * / % ** ++ --`.
8. **read** takes user input; `-p` shows a prompt; `-s` silences input (passwords); `-t N` adds a timeout; `-a` reads into an array.
9. **Positional parameters**: `$1..$9`, `${10}`…; `$#` = count; `"$@"` = all args separately (preferred); `"$*"` = all merged. Validate with `[ $# -ne N ]`.
10. **if / elif / else**: condition is a command's exit status. Use `[ ]` for POSIX tests, `[[ ]]` for bash extended tests (regex with `=~`), `(( ))` for arithmetic conditions.
11. **for** iterates over lists/ranges/arrays. **while** loops while condition is true. **until** loops until condition becomes true. `break` exits; `continue` skips iteration.
12. **Exit status** `$?`: 0 = success, non-zero = failure. Use `exit N` to set a script's exit code. `&&` runs on success; `||` runs on failure. `trap` handles cleanup on exit or signals.

End of Unit 8 — Shell Programming
This completes all 8 units of UNIX and Shell Programming — BCA 6th Semester, Techno India (Hooghly Campus).

Unit 8 — Practice Questions

Practice Questions — Unit 8: Shell Programming

A. Short Answer (2 marks each)

1. What is the shebang line? What is the difference between `#!/bin/sh` and `#!/bin/bash`?
2. What is the output of `echo '$HOME'` vs `echo "$HOME"`? Why do they differ?
3. What does `export` do? How is an exported variable different from a local variable?
4. Explain `read -s -p "Password: " pass`. What does each flag do?
5. What is the difference between `"$@"` and `"$*"` when passing arguments?
6. What does `set -euo pipefail` do? Why is it recommended in production scripts?
7. What is command substitution? Give the modern and legacy syntax with an example.
8. Explain exit status. What does a return value of `0` mean? `127`? `130`?

B. Predict the Output (2 marks each)

1. `x=5; let x=x*3+1; echo $x`
2. `for i in {1..5..2}; do echo -n "$i "; done`
3. `echo $((10 / 3))` vs `echo "scale=2; 10/3" | bc`
4. `n=1; until [ $n -gt 3 ]; do echo $n; ((n++)); done`

C. Write the Script (5 marks each)

1. Write `grade.sh`: accepts name and three marks as command-line arguments, computes total and percentage, prints grade (A+/A/B/C/F) using `if-elif-else`. Validate that exactly 4 arguments are supplied.
2. Write a script that reads numbers in a `while` loop until user enters `0`, then prints sum, count, maximum, and minimum.
3. Write a script using a `for` loop to count lines in each `.txt` file in the current directory and print a sorted summary (most lines first).

D. Debug the Script (3 marks)

1. Find and fix **all** bugs:

```
#!/bin/bash
name = "Alice"
if [ $name = "Alice" ]
then echo "Hello $name"
fi
echo "Count: " $((2+3))
```

References

References and Further Reading

Textbooks

[1] Sumitabha Das, *Your UNIX/Linux: The Ultimate Guide*, 3rd ed. McGraw-Hill Education, 2013.
[Primary textbook for this course — covers all 8 units in depth.]

[2] W. Richard Stevens and Stephen A. Rago, *Advanced Programming in the UNIX Environment*, 3rd ed. Addison-Wesley Professional, 2013.
[Definitive reference for UNIX system calls, processes, and I/O.]

[3] Arnold Robbins and Nelson H.F. Beebe, *Classic Shell Scripting*. O'Reilly Media, 2005.
[Comprehensive guide to POSIX shell scripting and UNIX text tools.]

[4] Cameron Newham, *Learning the bash Shell*, 3rd ed. O'Reilly Media, 2005.
[Deep coverage of bash features, variables, loops, and scripting.]

[5] Brian W. Kernighan and Rob Pike, *The UNIX Programming Environment*. Prentice Hall, 1984.
[Classic introduction to the UNIX philosophy and shell programming.]

Online Resources

[6] The Linux Documentation Project (TLDP). *Bash Guide for Beginners* and *Advanced Bash-Scripting Guide*.
`https://tldp.org/guides.html`

[7] GNU Bash Reference Manual. Free Software Foundation.
`https://www.gnu.org/software/bash/manual/bash.html`

[8] The Open Group. *The Single UNIX Specification (SUSv4) / POSIX.1-2017*.
`https://pubs.opengroup.org/onlinepubs/9699919799/`

[9] Linux man-pages project. Manual pages for all standard UNIX commands.
Access locally via `man bash`, `man sh`, `man test`, `man builtins`, etc.

[10] Wooledge Bash Wiki. Community-maintained best-practices wiki.
`https://mywiki.wooledge.org/BashGuide`

[11] ShellCheck — static analysis tool for shell scripts.
`https://www.shellcheck.net`

Standards

[12] IEEE Std 1003.1-2017 (*POSIX.1-2017*). *IEEE Standard for Information Technology — Portable Operating System Interface (POSIX)*. IEEE/The Open Group, 2017.

[13] Filesystem Hierarchy Standard (FHS), Version 3.0. Linux Foundation, 2015.
`https://refspecs.linuxfoundation.org/fhs.shtml`

UNIX AND SHELL PROGRAMMING

Techno India (Hooghly Campus) · Department of Computer Applications
Faculty: Prof. Anik Patra

UNIX & Shell Programming — Quick Reference

All key commands from Units 1–8 in one place · Ideal for last-minute revision

File & Directory Commands

Command	Description	Key Flags
`ls`	List directory contents	`-l -a -h -lt -lS -F -i -R`
`cd`	Change directory	`cd -` (prev) `cd $HOME` (home)
`pwd`	Print working directory	—
`mkdir`	Create directory	`-p` (parents) `-m` (mode) `-v`
`rmdir`	Remove empty directory	`-p` (parents)
`rm`	Remove files/dirs	`-r -f -i -v`
`cp`	Copy files/dirs	`-r -p -i -u -v`
`mv`	Move or rename	`-i -n -v`
`touch`	Create file / update timestamp	`-a -m -d "date" -t STAMP`
`cat`	Display file contents	`-n` (line nos) `-A`
`find`	Search filesystem	`-name -type -size -mtime -exec`
`ln`	Create links	`-s` (symlink)
`chmod`	Change permissions	`u+x 755 644 -R`
`chown`	Change owner/group	`-R user:group`
`stat`	Full file metadata	—
`df`	Disk free (filesystem)	`-h -i -T`
`du`	Disk usage (files/dirs)	`-h -s -a --max-depth`

File Processing Commands

Command	Description	Key Flags / Syntax
`cut`	Extract columns/fields	`-c1-5  -d: -f1,3`
`awk`	Pattern scan & process	`-F:  NR NF $0  BEGIN END`
`paste`	Merge files side-by-side	`-d,` (delim) `-s` (serial)
`sort`	Sort lines	`-n -r -k2 -t: -u -h`
`uniq`	Filter duplicate lines	`-c` (count) `-d` (dupes) `-u`
`wc`	Count lines/words/chars	`-l -w -c -L`
`tr`	Translate/delete chars	`-d` (delete) `-s` (squeeze)
`diff`	Compare files line-by-line	`-u` (unified) `-y` (side) `-q`
`cmp`	Compare files byte-by-byte	`-l` (all diffs) `-s` (silent)
`comm`	Compare two sorted files	`-12` (common) `-23` (file1 only)
`join`	Relational join on files	`-t:  -1 N -2 N  -a 1`
`split`	Split large files	`-l N -b 1M -n 4 -d`
`grep`	Search with pattern	`-i -v -r -n -c -l -E -w`

Utility Commands

Command	Description	Key Flags / Syntax
`date`	Display/format date-time	`+"%Y-%m-%d"  -d "tomorrow"`
`cal`	Display calendar	`cal MM YYYY  -3  -j`
`bc`	Arbitrary precision calc	`-l` (math lib) `scale=N`
`echo`	Print text	`-n` (no newline) `-e` (escapes)
`who`	Show logged-in users	`-a -u -q -H -b`
`whoami`	Current username	—
`pr`	Format for printing	`-N` (cols) `-m` (merge) `-n`
`tar`	Archive files	`-c -x -t -z -j -J -v -f -C`
`gzip`	Compress single file	`-d -k -v -l -9 -1`
`zip`	Archive + compress	`-r -e -9 -d -u`
`unzip`	Extract ZIP archive	`-l -t -d dir -o`

Process Management

Command	Description	Key Flags / Notes
`ps`	List processes	`aux` `-ef` `axjf` (tree)
`top`	Live process monitor	`k` kill `r` renice `P/M` sort
`kill`	Send signal to process	`-9` SIGKILL `-15` SIGTERM
`killall`	Kill by name	`-9 name`
`nice`	Start with adjusted priority	`-n 10 cmd` (range: −20 to +19)
`renice`	Change running process priority	`+15 -p PID`
`vmstat`	Virtual memory stats	`vmstat 2 5`
`bg / fg`	Background / foreground job	`cmd &`

I/O Redirection, Pipes & Shell Operators

Operator	Meaning	Example
`>`	Redirect stdout (overwrite)	`ls > list.txt`
`>>`	Redirect stdout (append)	`echo "hi" >> log.txt`
`<`	Redirect stdin from file	`sort < names.txt`
`2>`	Redirect stderr	`cmd 2> err.txt`
`2>&1`	Merge stderr into stdout	`cmd > out.txt 2>&1`
`&>`	Redirect stdout + stderr	`cmd &> all.txt`
`|`	Pipe stdout to next stdin	`ls | wc -l`
`tee`	Split to screen & file	`cmd | tee log.txt`
`&&`	Run if previous succeeded	`mkdir d && cd d`
`||`	Run if previous failed	`cmd || echo "failed"`
`$( )`	Command substitution	`today=$(date)`
`$(( ))`	Arithmetic expansion	`echo $((5*3))`

Octal	Symbolic	Common Use
777	`rwxrwxrwx`	Avoid (insecure)
755	`rwxr-xr-x`	Scripts / dirs
644	`rw-r--r--`	Text files
600	`rw-------`	Private keys
700	`rwx------`	Private dirs
444	`r--r--r--`	Read-only

Symbol	File Type	Example
`-`	Regular file	`notes.txt`
`d`	Directory	`projects/`
`l`	Symbolic link	`bin -> usr/bin`
`c`	Character dev.	`/dev/tty`
`b`	Block device	`/dev/sda`
`p`	Named pipe	`mypipe`
`s`	Socket	`nginx.sock`

Shell Scripting Quick Reference

Concept	Syntax / Example
Variable assignment	`name="Alice"` (no spaces around =)
Variable access	`$name` or `${name}`
Positional params	`$0` script `$1..$9` `$#` count `"$@"` all args
Special variables	`$?` exit status `$$` PID `$!` last bg PID
Arithmetic	`let x=a+b` `x=$((a*b))` `(( x++ ))`
String test	`[ -z "$s" ]` empty `[ -n "$s" ]` non-empty
Numeric test	`[ $a -eq $b ]` `-ne -lt -le -gt -ge`
File test	`[ -f f ]` file `[ -d d ]` dir `[ -x f ]` executable
if / elif / else	`if [ cond ]; then … elif [ cond ]; then … else … fi`
for loop (list)	`for x in a b c; do echo $x; done`
for loop (range)	`for i in {1..10}; do …; done`
while loop	`while [ $n -gt 0 ]; do …; (( n-- )); done`
until loop	`until [ $n -ge 10 ]; do …; (( n++ )); done`
read input	`read -p "Name: " name` `read -s password`
Command substitution	`today=$(date +%Y-%m-%d)`
exit with status	`exit 0` (success) `exit 1` (error)

VI Editor Quick Reference

Navigation		Insert Mode Entry	
`h j k l`	Left/Down/Up/Right	`i / I`	Insert before cursor / line start
`w / b`	Next / prev word	`a / A`	Append after cursor / line end
`0 / $`	Line start / end	`o / O`	New line below / above
`gg / G`	File start / end	`Esc`	Return to command mode
`:N`	Go to line N		
Editing		**File Operations**	
`x / X`	Delete char fwd/bk	`:w`	Save
`dd`	Delete line	`:q`	Quit
`dw`	Delete word	`:wq`	Save and quit
`D`	Delete to EOL	`:q!`	Quit without saving
`yy`	Copy (yank) line	`:w file`	Save as new file
`p / P`	Paste after/before	`:e file`	Open file
`u`	Undo		
`Ctrl+r`	Redo		
Search & Replace		**Settings**	
`/pattern`	Search forward	`:set nu`	Line numbers on
`?pattern`	Search backward	`:set nonu`	Line numbers off
`n / N`	Next / prev match	`:set ic`	Case insensitive
`:s/o/n/g`	Replace in line	`:set ts=4`	Tab size 4
`:%s/o/n/g`	Replace in file	`:syntax on`	Syntax highlight

Regular Expression Quick Reference

Pattern	Meaning	Example
`.`	Any single character	`c.t` matches cat, cut, cot
`*`	Zero or more of preceding	`ab*c` matches ac, abc, abbc
`+`	One or more (ERE)	`ab+c` matches abc, abbc
`?`	Zero or one (ERE)	`colou?r` matches color, colour
`^`	Start of line	`^root` line starts with root
`$`	End of line	`.sh$` line ends with .sh
`[abc]`	Any one of a, b, c	`[aeiou]` any vowel
`[^abc]`	Not any of a, b, c	`[^0-9]` not a digit
`[a-z]`	Range	`[A-Z0-9]` uppercase or digit
`(a|b)`	Either a or b (ERE)	`(cat|dog)`
`{n,m}`	Between n and m occurrences	`[0-9]{2,4}` 2–4 digits

Glossary of UNIX and Shell Programming Terms

Key terms, definitions, and concepts from all 8 units

Absolute Path — A file path that starts from the root directory `/` and fully specifies the location of a file, e.g. `/home/student/notes.txt`.

AWK — A powerful text-processing programming language and utility (`awk`) for pattern scanning, field extraction, and report generation. Named after its creators Aho, Weinberger, and Kernighan.

Bash (Bourne Again Shell) — The most widely used shell on Linux systems. A superset of the POSIX `sh` shell with extensions including arrays, arithmetic, command history, and tab completion.

Block Device — A file type (shown as `b` in `ls -l`) that provides buffered, block-level I/O access to hardware such as hard disks and USB drives. Located in `/dev/`.

Character Device — A file type (shown as `c`) providing unbuffered, character-by-character I/O to devices such as terminals and keyboards.

Child Process — A process created by another process (its *parent*) via the `fork()` system call. Inherits the parent's environment, file descriptors, and address space.

Command Substitution — A shell feature that executes a command and inserts its output into the current command line. Written as `$(command)` (modern) or `` `command` `` (legacy).

Daemon — A background process that runs without a controlling terminal, typically started at boot to provide system services (e.g. `sshd`, `cron`, `nginx`).

Environment Variable — A named variable stored in a process's environment, inherited by child processes. Set with `export NAME=value`. Common examples: `$PATH`, `$HOME`, `$SHELL`.

Exit Status — An integer returned by a command or script when it terminates. Zero (`0`) means success; any non-zero value indicates an error. Accessed via `$?`.

File Descriptor — A small non-negative integer that the kernel uses to identify an open file. Standard descriptors: `0` = stdin, `1` = stdout, `2` = stderr.

Filesystem Hierarchy Standard (FHS) — A standard defining the directory layout of UNIX-like systems. All files reside under the root `/`, with specific roles for `/bin`, `/etc`, `/home`, `/var`, etc.

Fork — The `fork()` system call creates an exact copy of the current process. The original is the parent; the copy is the child. Returns the child's PID to the parent and `0` to the child.

GREP (Global Regular Expression Print) — A command-line utility for searching text using regular expressions. `grep` prints lines matching a pattern; `egrep`/`grep -E` supports extended regex.

Hard Link — A directory entry that points to the same inode as another filename. All hard links to an inode are equal; data is deleted only when the link count reaches zero.

Here-Document (Heredoc) — A shell construct that provides multi-line input to a command inline within a script. Written as `command << 'EOF' ... EOF`.

i-Node (Index Node) — A data structure on disk storing all metadata about a file (permissions, size, timestamps, data block pointers) except its filename.

Kernel — The core of the operating system, running in privileged mode (Ring 0). Manages processes, memory, file systems, device drivers, and networking.

Kernel Mode — The privileged CPU execution mode in which the kernel runs. Has unrestricted access to all hardware and memory. Contrasted with *User Mode*.

Metacharacter — A character with special meaning to the shell or a utility (e.g. `*`, `?`, `[`, `|`, `$`). Must be escaped or quoted to be treated literally.

Orphan Process — A process whose parent has terminated. Orphans are adopted by `init/systemd` (PID 1), which calls `wait()` to collect their exit status.

Pipe (|) — A mechanism connecting the standard output of one process to the standard input of another, enabling command chaining without temporary files.

POSIX (Portable Operating System Interface) — A family of IEEE standards (IEEE 1003) defining the API, shell, and utilities interface for UNIX-compatible operating systems. Ensures portability across compliant systems.

Process — A running instance of a program, uniquely identified by a PID (Process Identifier). Has its own virtual address space, file descriptors, and execution state.

Process Control Block (PCB) — A kernel data structure containing all information about a process: PID, state, registers, memory map, file descriptors, and scheduling information.

Regular Expression (Regex) — A pattern language for matching text. Basic regex (BRE) is used by `grep`; Extended regex (ERE) by `egrep`/`grep -E`; PCRE by `grep -P`.

Relative Path — A path specified relative to the current working directory, e.g. `../docs/notes.txt`. Does not begin with `/`.

Scheduler (CFS) — Linux's Completely Fair Scheduler (CFS) uses a red-black tree of virtual runtimes (`vruntime`) to select which process runs next, aiming for fairness.

Shebang (#!) — The first line of a script (`#!/bin/bash`) that tells the OS which interpreter to use when executing the file. Composed of `#!` followed by the interpreter path.

Shell — A command-line interpreter that provides a user interface to the OS kernel. Also a programming language supporting variables, loops, conditionals, and functions.

Signal — An asynchronous notification sent to a process to notify it of an event (e.g. `SIGINT` from Ctrl+C, `SIGKILL` to force-terminate, `SIGTERM` to request termination).

Symbolic Link (Symlink) — A special file containing a text path pointing to another file or directory. Has its own inode; becomes a *dangling link* if the target is deleted.

System Call (Syscall) — The controlled interface through which user-space programs request services from the kernel (e.g. `open`, `read`, `write`, `fork`, `exec`).

Terminal / TTY — A character device providing a text interface between the user and the shell. `/dev/tty` is the current terminal; `/dev/pts/N` are pseudo-terminals.

UNIX Philosophy — The design principle articulated by Doug McIlroy: write programs that do one thing well, work together, and handle text streams — favouring composition over monolithic design.

User Mode — The unprivileged CPU execution mode in which user applications run. Cannot directly access hardware; must request services via system calls.

Virtual File System (VFS) — A kernel abstraction layer that provides a uniform interface to different filesystem types (ext4, XFS, tmpfs, etc.), allowing `open`/`read`/`write` to work on any filesystem.

Zombie Process — A process that has terminated but whose exit status has not yet been collected by its parent via `wait()`. Appears as `Z` in `ps` output.

Lab Exercises: Practical Shell Scripts

About These Exercises

The following labs consolidate every major topic in this course. Each is self-contained: a problem statement, a complete working script, expected output, and notes on the key techniques used.

Lab 1 — Hello World and Command-Line Arguments

Objective: Write the canonical first script; accept and display arguments.

```
#!/bin/bash
# Lab 01 -- Hello World + arguments
# Usage: bash lab01_hello.sh <your-name>

NAME=${1:-"World"}      # use first argument or default "World"
echo "Hello, ${NAME}!"
echo "Script name : $0"
echo "Arguments   : $#"
echo "All args    : $@"
echo "PID         : $$"
```

Listing 183: lab01_hello.sh

```
$ bash lab01_hello.sh Alice
Hello, Alice!
Script name : lab01_hello.sh
Arguments   : 1
All args    : Alice
PID         : 4821
```

Listing 184: Expected output

Key Concepts

`$1` first positional argument `$#` argument count `$@` all arguments `$$` current PID
`:-` default-value substitution

Lab 2 — File Information Script

Objective: Display detailed metadata about a file using built-in commands.

```
#!/bin/bash
# Lab 02 -- File information report

if [ $# -ne 1 ]; then echo "Usage: $0 <filename>"; exit 1; fi
FILE="$1"
if [ ! -e "$FILE" ]; then echo "Error: '$FILE' not found."; exit 2; fi

echo "============================================"
echo "  File Report: $FILE"
echo "============================================"
echo "Type        : $(file -b "$FILE")"
echo "Size        : $(du -sh "$FILE" | cut -f1)"
echo "Lines       : $(wc -l < "$FILE")"
echo "Words       : $(wc -w < "$FILE")"
echo "Permissions : $(ls -l "$FILE" | awk '{print $1}')"
echo "Owner       : $(ls -l "$FILE" | awk '{print $3":"$4}')"
echo "Modified    : $(stat -c '%y' "$FILE")"
echo "Inode       : $(ls -i "$FILE" | awk '{print $1}')"
```

```
echo "============================================"
```

Listing 185: lab02_fileinfo.sh

```
$ bash lab02_fileinfo.sh /etc/hostname
============================================
  File Report: /etc/hostname
============================================
Type        : ASCII text
Size        : 4.0K
Lines       : 1
Words       : 1
Permissions : -rw-r--r--
Owner       : root:root
Modified    : 2024-08-15 10:23:11
Inode       : 524289
============================================
```

Listing 186: Expected output

Lab 3 — Arithmetic and Grade Calculator

Objective: Read marks, compute percentage, assign letter grade.

```
#!/bin/bash
# Lab 03 -- Grade calculator

echo "=== Grade Calculator ==="
read -p "Subject 1 (out of 100): " M1
read -p "Subject 2 (out of 100): " M2
read -p "Subject 3 (out of 100): " M3

TOTAL=$((M1 + M2 + M3))
PERCENT=$(echo "scale=2; $TOTAL * 100 / 300" | bc)

if   (( TOTAL >= 270 )); then GRADE="O  (Outstanding)"
elif (( TOTAL >= 240 )); then GRADE="A+ (Excellent)"
elif (( TOTAL >= 210 )); then GRADE="A  (Very Good)"
elif (( TOTAL >= 180 )); then GRADE="B+ (Good)"
elif (( TOTAL >= 150 )); then GRADE="B  (Average)"
elif (( TOTAL >= 120 )); then GRADE="C  (Pass)"
else                          GRADE="F  (Fail)"
fi

echo ""
echo "Total   : $TOTAL / 300"
echo "Percent : $PERCENT %"
echo "Grade   : $GRADE"
```

Listing 187: lab03_grades.sh

```
Subject 1 (out of 100): 82
Subject 2 (out of 100): 76
Subject 3 (out of 100): 90
Total   : 248 / 300
Percent : 82.66 %
Grade   : A+ (Excellent)
```

Listing 188: Expected output

Lab 4 — Loops: Multiplication Table

Objective: Generate a multiplication table using `for` and `while`.

```bash
#!/bin/bash
# Lab 04 -- Multiplication table

read -p "Enter a number: " N

echo "--- for loop ---"
for i in $(seq 1 10); do
  printf "%3d x %2d = %4d\n" "$N" "$i" $(( N * i ))
done

echo "--- while loop ---"
i=1
while [ $i -le 10 ]; do
  printf "%3d x %2d = %4d\n" "$N" "$i" $(( N * i ))
  (( i++ ))
done
```

Listing 189: lab04_table.sh

```
--- for loop ---
  7 x  1 =    7
  7 x  2 =   14
  ...
  7 x 10 =   70
```

Listing 190: Expected output (N=7)

Lab 5 — Functions: Number Utilities

Objective: Define reusable functions for prime check, factorial, and Fibonacci.

```bash
#!/bin/bash
# Lab 05 -- Shell functions

is_prime() {
  local n=$1; [ "$n" -lt 2 ] && echo "No" && return
  for (( i=2; i*i<=n; i++ )); do
    (( n % i == 0 )) && echo "No" && return; done
  echo "Yes"
}

factorial() {
  local n=$1 result=1
  for (( i=2; i<=n; i++ )); do result=$(( result * i )); done
  echo $result
}

fibonacci() {
  local n=$1 a=0 b=1 tmp
  printf "Fibonacci(%d): " "$n"
  for (( i=0; i<n; i++ )); do
    printf "%d " $a; tmp=$(( a+b )); a=$b; b=$tmp; done
  echo
}

read -p "Enter a number: " NUM
echo "Prime?    : $(is_prime $NUM)"
echo "Factorial : $(factorial $NUM)"
fibonacci $NUM
```

Listing 191: lab05_functions.sh

```
Enter a number: 7
Prime?    : Yes
Factorial : 5040
Fibonacci(7): 0 1 1 2 3 5 8
```

Listing 192: Expected output (N=7)

Lab 6 — String Operations

Objective: Demonstrate string length, slicing, replacement, and case conversion.

```bash
#!/bin/bash
# Lab 06 -- String manipulation

read -p "Enter a string: " STR

echo "Original      : $STR"
echo "Length        : ${#STR}"
echo "Uppercase     : ${STR^^}"
echo "Lowercase     : ${STR,,}"
echo "First 5 chars : ${STR:0:5}"
echo "Last  5 chars : ${STR: -5}"
echo "Reversed      : $(echo "$STR" | rev)"
echo "Word count    : $(echo "$STR" | wc -w)"

read -p "Replace what? " OLD
read -p "Replace with? " NEW
echo "After replace : ${STR//$OLD/$NEW}"

read -p "Check contains? " SUB
if [[ "$STR" == *"$SUB"* ]]; then
  echo "'$STR' CONTAINS '$SUB'"
else
  echo "'$STR' does NOT contain '$SUB'"
fi
```

Listing 193: lab06_strings.sh

Lab 7 — Automated Backup with Rotation

Objective: Create a timestamped compressed backup; keep only the last 5.

```bash
#!/bin/bash
# Lab 07 -- Backup with rotation

BACKUP_DIR="${HOME}/backups"
SOURCE_DIR="${1:-.}"
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
ARCHIVE="${BACKUP_DIR}/backup_${TIMESTAMP}.tar.gz"
MAX_BACKUPS=5

mkdir -p "$BACKUP_DIR"
tar -czf "$ARCHIVE" "$SOURCE_DIR" 2>/dev/null

if [ $? -eq 0 ]; then
  echo "Backup created: $ARCHIVE ($(du -sh "$ARCHIVE" | cut -f1))"
else
  echo "ERROR: Backup failed." >&2; exit 1
fi
```

```

# Rotate: delete oldest beyond MAX_BACKUPS
ls -1t "${BACKUP_DIR}"/backup_*.tar.gz 2>/dev/null | \
  tail -n +"$((MAX_BACKUPS + 1))" | xargs rm -f

echo "Total backups: $(ls -1 "${BACKUP_DIR}"/backup_*.tar.gz | wc -l)"
```

Listing 194: lab07_backup.sh

Lab 8 — Process Monitor

Objective: List top CPU consumers; search and optionally kill by name.

```
#!/bin/bash
# Lab 08 -- Process monitor

echo "Top 10 CPU consumers:"
ps aux --sort=-%cpu | head -11 | \
  awk 'NR==1{printf "%-8s %-8s %-6s %s\n","USER","PID","%CPU","COMMAND"}
       NR >1{printf "%-8s %-8s %-6s %s\n",$1,$2,$3,$11}'

read -p "Search process name (Enter to skip): " PNAME
[ -z "$PNAME" ] && exit 0

RESULTS=$(pgrep -a "$PNAME" 2>/dev/null)
if [ -z "$RESULTS" ]; then
  echo "No process named '$PNAME' found."
else
  echo "$RESULTS"
  read -p "Kill all? [y/N]: " CONFIRM
  [[ "$CONFIRM" =~ ^[Yy]$ ]] && pkill "$PNAME" && echo "Sent SIGTERM."
fi
```

Listing 195: lab08_procmon.sh

Lab 9 — Text Processing with grep, sed, awk

Objective: Parse a CSV log file using the three core text tools.

```
#!/bin/bash
# Lab 09 -- grep / sed / awk on a CSV log
# Format: DATE,TIME,LEVEL,MESSAGE

LOG="app.log"
[ -f "$LOG" ] || cat > "$LOG" << 'EOF'
2024-11-01,09:00:01,INFO,Server started on port 8080
2024-11-01,09:02:44,ERROR,Database connection timeout
2024-11-01,09:05:33,WARN,High memory usage 87 percent
2024-11-01,09:06:01,ERROR,Null pointer in UserService
2024-11-01,09:07:20,INFO,Cache cleared
EOF

echo "=== ERROR lines (grep) ==="
grep "ERROR" "$LOG"

echo ""
echo "=== Reformat date DD/MM/YYYY (sed) ==="
sed 's/\([0-9]\{4\}\)-\([0-9]\{2\}\)-\([0-9]\{2\}\)/\3\/\2\/\1/' "$LOG"

echo ""
echo "=== Level summary (awk) ==="
awk -F',' '{c[$3]++} END{for(l in c) printf "%-8s %d\n",l,c[l]}' "$LOG" | sort
```

Listing 196: lab09_textprocess.sh

```
=== ERROR lines (grep) ===
2024-11-01,09:02:44,ERROR,Database connection timeout
2024-11-01,09:06:01,ERROR,Null pointer in UserService

=== Level summary (awk) ===
ERROR    2
INFO     2
WARN     1
```

Listing 197: Expected output

Lab 10 — VI Editor Practice Drill

Objective: Practice the full VI workflow: open, navigate, edit, search, replace, save.

Instructions

Perform each step at the terminal. Do not use arrow keys or the mouse – use only the VI keybindings shown.

```
# Create practice file
$ printf "The quick brown fox\nShell scripting is powerful\nUNIX born at Bell
    Labs 1969\nEvery tool does one thing well\nGrep searches with regex\n" >
    practice.txt

# Open in VI
$ vi practice.txt

# Navigation (Normal mode)
gg   -- jump to line 1        G    -- jump to last line
3G   -- jump to line 3        w    -- next word
0    -- start of line         $    -- end of line

# Editing
i    -- Insert before cursor  Esc  -- back to Normal
dd   -- delete line           3dd  -- delete 3 lines
yy   -- yank line             p    -- paste below
u    -- undo                  .    -- repeat last change

# Search
/UNIX          -- search forward
n              -- next match
?Shell         -- search backward

# Search and replace
:%s/the/THE/g          -- replace all in file
:1,3s/is/was/g         -- replace on lines 1-3

# Save / Quit
:w     -- save       :wq  -- save and quit
:q!    -- quit without saving
```

Listing 198: VI Practice Drill

Lab 11 — Mini Project: Student Record Manager

Objective: Menu-driven script storing student records in a CSV flat file.

```
#!/bin/bash
# Lab 11 -- Student Record Manager  (CSV: RollNo,Name,Marks)

DB="students.csv"; [ -f "$DB" ] || touch "$DB"
```

```

add_student() {
  read -p "Roll No: " ROLL
  grep -q "^${ROLL}," "$DB" && echo "Roll $ROLL exists." && return
  read -p "Name   : " NAME; read -p "Marks  : " MARKS
  echo "${ROLL},${NAME},${MARKS}" >> "$DB" && echo "Added."
}

view_all() {
  [ ! -s "$DB" ] && echo "No records." && return
  printf "%-6s %-20s %-6s %s\n" "ROLL" "NAME" "MARKS" "GRADE"
  while IFS=',' read -r r n m; do
    (( m>=90 )) && g="O" || { (( m>=75 )) && g="A" || \
    { (( m>=60 )) && g="B" || { (( m>=50 )) && g="C" || g="F"; }; }; }
    printf "%-6s %-20s %-6s %s\n" "$r" "$n" "$m" "$g"
  done < "$DB"
}

search_student() {
  read -p "Roll No: " ROLL
  RES=$(grep "^${ROLL}," "$DB")
  [ -z "$RES" ] && echo "Not found." || echo "$RES" | awk -F',' \
    '{printf "Roll:%s  Name:%s  Marks:%s\n",$1,$2,$3}'
}

delete_student() {
  read -p "Roll No: " ROLL
  grep -q "^${ROLL}," "$DB" && sed -i "/^${ROLL},/d" "$DB" \
    && echo "Deleted." || echo "Not found."
}

report() {
  echo "Total : $(wc -l < "$DB")"
  echo "Avg   : $(awk -F',' '{s+=$3;c++} END{printf "%.1f",s/c}' "$DB")"
  echo "High  : $(awk -F',' 'BEGIN{m=0}{if($3>m)m=$3}END{print m}' "$DB")"
  echo "Low   : $(awk -F',' 'BEGIN{m=999}{if($3<m)m=$3}END{print m}' "$DB")"
}

while true; do
  echo ""; echo "=== Student Record Manager ==="
  echo "1-Add  2-View  3-Search  4-Delete  5-Report  6-Exit"
  read -p "Choice: " C
  case $C in
    1) add_student;; 2) view_all;; 3) search_student;;
    4) delete_student;; 5) report;; 6) exit 0;;
    *) echo "Invalid.";;
  esac
done
```

Listing 199: lab11_student_mgr.sh

```
=== Student Record Manager ===
1-Add  2-View  3-Search  4-Delete  5-Report  6-Exit
Choice: 2
ROLL   NAME                 MARKS  GRADE
101    Amitendu Dhusiya     88     A
102    Priya Sharma         95     O
103    Rahul Das            54     C
```

Listing 200: Sample session

Lab Exercises — Summary Table

Lab	Topic	Key Skills Practised
01	Hello World & Arguments	`$1 $# $@`, default values `:-`
02	File Information	`file`, `du`, `wc`, `stat`, `ls -i`
03	Grade Calculator	Arithmetic, `bc`, `if-elif-else`
04	Multiplication Table	`for`, `while`, `printf`, `seq`
05	Number Functions	`function`, local vars, prime/factorial/Fibonacci
06	String Operations	`${#}`, slicing, `^^`, `,,`, `rev`, substring test
07	Backup Script	`tar -czf`, `date`, rotation with `xargs rm`
08	Process Monitor	`ps aux`, `pgrep`, `pkill`, `awk`
09	Text Processing	`grep`, `sed` substitution, `awk` field parsing
10	VI Editor Drill	All VI modes, navigation, `:s///g`, save/quit
11	Student Record Manager	`case`, functions, CSV CRUD with grep/awk/sed

End of Lab Exercises

Index

arithmetic expansion, 188
arrays, 192
awk, 191

case statement, 192
command-line arguments, 187

date, 190

for loop, 189
functions
    shell, 189

grep, 191
gzip, 190

if statement, 188

kill, 191

lab exercises
    arithmetic, 188
    backup, 190
    file information, 187
    functions, 189
    Hello World, 187
    loops, 189
    mini project, 192
    process monitor, 191
    strings, 190
    text processing, 191
    VI editor, 192
ls, 187

ps, 191

sed, 191
stat, 187
string manipulation, 190

tar, 190

vi editor, 192

wc, 187
while loop, 189